

灵枢架构速查手册

目的:开发前先查这里,避免重复建设和异常爆发。每次架构改动必须更新本文件(改了哪、为什么、影响面)。维护规则:新增/改动一个组件 → 更新「组件地图」对应行 + 在文末「变更日志」追一条。

最后更新:2026-06-21(对标前沿 **Agent harness** 追平·第一步「架网」落地:循环不变量 **LingShuLoopInvariants**(纯逻辑 I1-I6 + 退出原因码 + 全局遥测)接进 **LingShuAgentSession** 回合边界;**AgentLoopFuzzTests** 确定性混沌 + **LoopInvariantTests** 纯逻辑守卫 + **Scripts/soak-e2e.sh** 真模型长跑;架网逼出并修复两处历史良构隐患(停滞交还/阻塞同回合的悬空 **tool_call**);**LingShuControlRouter** 载荷序列化拆 + **Payloads.swift** 守住 ≤ 800 。「测无可测」鲁棒性战役开锤(两模式都锤):
NestedLoopFuzzTests 嵌套引擎混沌 → 修 **turnsUsed** 计数倒退(**spine** 同步改增量累加不覆盖);
BatchInterruptLeakTests → 经典引擎 **driveAgentDelivery** 入口复位
batchInterruptRequested 修打断标志粘滞泄漏旁路验收门([verify-gate-bypass-batchinterrupt-leak](#) classic 侧已修);**Scripts/soak-e2e.sh** 加自主模式段(**go_live**/在岗任务/**stop/arm/disarm**,工具名 **lingshu_autonomous**)=通用对话+自主双模式真机长跑。差距 2 薄基线落地:**LingShuVerifierGate**(纯逻辑)按交付物类型 **gate** 贵 LLM 评审——纯代码+确定性门绿 → 跳过 **LLM**(确定性门照跑、只省自评,真机实测轨迹见「跳过 LLM 评审」)、主观交付物才跑;
LingShuHarnessProfile 脑力分 → 连续起步档(薄/中/厚三级,删二元 85 阈值,根治"一道偏题打回厚")。差距 3 **apply_patch** 落地:**LingShuPatchApplier**(纯事务核,多文件多 **hunk**、失败整批回滚、复用 **EditReplacer**)+ **LingShuState**+**ApplyPatch**(工作目录围栏+落盘+自动 **re-read** 校验+产出物登记)。开发阶段全权(用户拍板):**developmentPhaseFullAccess**(**DEBUG** 默认开/**Release** 默认关,**UserDefaults** **lingshu.devFullAccess** 覆盖,
setDevelopmentPhaseFullAccess 切换) → 系统授权门 **requestShellApproval** 直接放行不弹框;唯一例外=未审第三方 **skill** 脚本(**quarantine** 供应链红线仍拦);状态见 **lingshu_status.developmentFullAccess**。详见 **Docs/对标前沿 Agent_harness 追平方案.md**) 2026-06-20(可插拔自我进化 **M0** 内核 **ABI** + **M1** 工具型 + **M2** 传感器型/**Hub** 动态注册 + **M3** 硬件接入/设备发现 + **M4** 执行器型/动作安全门 全落地;能力分层与知识驱动全落地:**P0** 知识纪律「陈述非祈使」**LingShuKnowledgeDiscipline** + 决策知识种子
LingShuSeedKnowledge; **P1** 升级阶梯 **LingShuCapabilityEscalation** 并入验收门;真控后置校验 **LingShuOutcomeVerification**(动作型交付=真实效果,写文档冒充判不过);**P3** 全撤定制:执行器风险/设备发现零关键词、保留名自动派生、已接入大脑判+知识图谱台账)

0. 一句话现状

灵枢的对话主入口已彻底转向「统一 **agent** 循环(模型即编排者)」。**submitTextInput** 的唯一出口就是 **runMainAgentTurn** → 主 **agent** 会话(真模型 + 通用工具 + 隔离并行子会话 + 统一账本 + 验收门)。旧的「启发式前置门 + 一次性路由 + 协同管线」已物理删除(无 **agentBackbonePrimary** 开关、无回退网);路由/直答/澄清/任务续接/并发全由模型在循环里决策。

0.5 自我认知 = 灵枢(大脑 + 四肢),灵枢只是壳

最根本的设计取向(用户拍板 2026-06-15):灵枢是一个能独立做事的"灵枢",不是问答机器人。

- 大脑 = 大模型的推理:思考/分析/拆解/规划/推进/决策/纠错全由模型自己结合上下文完成。任务丢给它=它自己想清楚怎么做并做完(codex 式"没有搞不定的事",除非硬性网络/权限/物理限制)。
- 四肢 = 各项能力(工具):听(ASR)/说(speak 工具→TTS)/读(read_file/fetch_url/VL)/写(write_file)/改代码/跑命令/联网/演示……都是大脑实现意图的手段。
- 壳 = 灵枢工程(本仓库):只负责把大脑接上四肢 + 把感知喂进来 + 把动作落出去。绝不用硬编码启发式控制器替大脑做决策——新能力 = 给大脑加一条"四肢"(工具),不是写一个 if-else 控制器。会议轮次检测这类"反射"可机械化,但"说不说/说什么/怎么推进"必须留在大脑。
- 用户只提供组件(证书/硬件/权限/素材);怎么用它们把事做成是灵枢的事。
- **Loop 模式** = 大脑跑持续的 perceive → think → act 循环(感知=四肢传入,动作=四肢传出),自己推进到目标。典型:丢个 PPT 让它独立演讲 = 它自己读懂→speak 逐页讲→听问题→实时答,全程大脑驱动。

编码机制(壳对、脑可换:任何强模型插上即可真编码):read_file 带行号 + offset/limit 分段读全(不再 8KB 截断,大源码文件能整文件看、行号供定位);edit_file str_replace 精确改(不重写整文件,改大代码首选,带 diff 卡 + 产出物登记 + 可撤销),底层走多策略匹配级联 [LingShuEditReplacer](#)(精确→逐行去空白→块锚点→空白归一→缩进灵活,移植自 opencode);容忍模型缩进/空白小出入,大幅降低"找不到 old_string"误打回;write_file 新建/整重写;run_command grep/装依赖/编译/跑测试。

write_file/edit_file 路径围栏经 **resolveWorkspaceWritePath**(2026-06-21 中台实测修):① 空 path 给清晰可操作报错(模型大内容把工具调用 JSON 撑爆成空参 {} 时,旧逻辑用误导的"写入位置必须在工作目录内"→模型瞎重试 5 次卡死;现明确"缺少 path,内容大改用 heredoc/分段 edit_file");② 相对路径以工作目录为基解析(模型常传相对路径,别一刀切拒);③ 防空内容覆盖(content 空但目标已有内容→拒,防大内容撑空把文件清了);WriteFilePathResolveTests 棘轮。系统提示给了大脑编码手法(先读→edit_file 改局部→跑测试迭代)。底层模型(编码脑)可换(如私有化 GLM),机制对了编码就只是效率/质量问题。见 [LingShuToolExecutor](#) + [LingShuFunctionCalling](#)。已落地的四肢(工具)节选:读写/精确改/命令/联网/find_images/acquire_resource/discover_skill/apply_skill/review_design/spawn_task/update_plan/recall_memory/speak(嘴:主动 TTS)/set_digital_human(身体表现:光球进入聆听/发声/思考/执行/警戒/确认/演示等态,只改表现不决策)/预览四肢(open_preview/preview_next/preview_prev/preview_goto/preview_scroll/close_preview——LingShuPreviewController+LingShuPreviewView,一切→PDF(soffice)→PDFKit,大脑确定性翻页/滚动;HTML 富页面例外(2026-06-20):HTML 经 soffice 转 PDF 会丢 JS/CSS(粒子背景/交互卡片)→改 app 内 WKWebView 原样渲染(controller.isHTML/webView,视图层

LingShuWebPreviewView),preview_scroll/preview_next 走 JS

window.scrollBy 确定性平滑滚、preview_document_text 取

document.body.innerText 一次拿整页正文照讲、present_fullscreen 同窗全屏——根治"HTML 只能丢浏览器+计算机控制滚=慢/卡理解中/前台焦点被 orb 挡滚错"(实测黑猫品种页:app 内打开+读全文+滚动+全屏全通,无浏览器无 computer-control);演示文稿对齐(2026-06-

17):open_preview/preview_next 返回里带【本页 PDFPage.string 实际内容】→大脑照真实内容讲不漂移;present_fullscreen 真全屏放映(sheet 默认无 .fullScreenPrimary 致 toggleFullScreen 失效已修+撑满整屏兜底);系统提示固化"演讲=进全屏→照页讲→退全屏";实测 11 页文稿逐页对得上、slideshow=true=独立演讲+拖动看文档的统一实现;自主模式置顶修复(2026-06-19):自主/在岗时 app 在

后台(化身右上角 orb),makeKeyAndOrderFront(nil) 只在 app 活动时才把演示窗提到全屏最前
→后台时 PPT 开了却压在别的 app 后面(用户实测设置顶没全屏)。修:open_preview 开窗 +
present_fullscreen 进全屏两处都加
NSApp.activate(ignoringOtherApps:true)+orderFrontRegardless() 强制提到所有 app 之前。实测 previewState 能到 isPresented=true → slideshow=true。注:present_fullscreen
在自主态 isStandingPersonOnDuty=true → alreadyManaged → 跳过托管同意门直接全屏)/会议(meeting_converse_*)/计算机直接操作(授权后:screen_capture/
list_ui_elements/click/double_click/move_mouse/type_text/press_key/
scroll——LingShuComputerControl+LingShuState+ComputerControl)/后台守候
watch_until/list_watches/cancel_watch(等长耗时外部条件满足即自动续跑,
LingShuState+BackgroundWatch)/定时调度
schedule_task/list_scheduled_tasks/cancel_scheduled_task(到时间点把指令
交回 agent 循环,LingShuState+ScheduledTasks,接真持久化的
LingShuScheduledTriggerService;根治"大脑没调度工具→即兴伪造 launchd plist 谎称已
设")。加新能力=加一条这样的工具,别写硬编码控制器。待加四肢:present_slide 已并入预览。
/内置多 tab 浏览器(2026-06-
20):browser_open/browser_navigate(URL/back/forward/reload)/browser_tab(new/
switch/close/list)/browser_eval(当前 tab 执行 JS 返回结果=网页自动化核心:读 DOM/点按/取数
据)/browser_read(取整页
innerText)/browser_scroll/browser_fullscreen/browser_close——
LingShuBrowserController(多 WKWebView tab + WKNavigationDelegate 更新标题/URL +
HTML5 全屏 API)+ LingShuBrowserView(独立窗:tab 栏+地址栏+活动 webview,撑满全屏)+
LingShuState+Browser(工具,挂主/派发/自主会话);用于网页演示(免丢 Chrome+计算机控制滚)+ 大脑
做网页自动化测试(打开页→eval 读 DOM/点按/取结果→多 tab)。与 previewController(本地
PPT/PDF 演示)分工。实测:open 本地页→eval 取 title/数 .card/点按钮验 innerText 变化/多 tab list 全
通。

1. 核心范式:统一 agent 循环

用户输入(submitTextInput)

- ├ 自主运行待答(ask_user 卡住)? → handleAutonomousAnswerIfNeeded(回填续跑)
- ├ 自主运行命令? → handleAutonomousRunCommandIfNeeded
- └ (其余一切) → runMainAgentTurn ← 唯一出口,无前置启发式门
 - └ 主 agent 会话(LingShuAgentSession,持久,seed 蒸馏记忆)

循环:模型 ↔ 工

具(write_file/run_command/web_search/apply_skill/spawn_task/ask_user...)

- ├ spawn_task → 派生隔离并行子会话(经 orchestrator,有界并发 N=3)
- ├ ask_user → 卡住(.blocked),等用户 resume
- └ 收尾(.completed) → 【验收门】独立 verifier 核对真实落盘 → 不过则反馈续

跑(≤3 轮)

铁律(违反=回归):

- **maker ≠ checker**:验收一律用独立 **verifier** 会话(评审官人格、独立上下文),不要 self-critique。见 verifyAgentDeliverable。
- 有产出物优先产出物:声称"已生成文件"必须真落盘 + 进产出物清单,否则验收判 **✗**。
- 通用工具,不按能力加补丁:要新能力让模型用 write_file/run_command 组合,不要加 make_ppt/make_crawler 这类专用工具。

- 跨重启续接:主会话用 `seededHistoryMessages()` 从持久化对话 `seed` 上下文,别让它"失忆"。

2. 组件地图(改前先查"想加 X 改哪里")

组件	文件	职责 / 关键入口
Agent 循环核心	LingShuAgentLoop.swift	<code>LingShuAgentSession(actor):send/resume/runLoop;.blocked</code> human-in-the-loop; <code>maxHistoryMessages</code> 上下文窗口(回合边界 <code>trimHistoryIfNeeded,0=</code> 不裁); <code>LingShuAgentTool/LingShuAgentModel</code> 协议; <code>LingShuScriptedAgentModel</code> (测试/演示)。循环不变量(2026-06-21,差距 1·架网):回合边界 <code>(.afterCompaction/.beforeModelCall/.terminal)</code> 调 <code>checkInvariants</code> 记违反到 <code>recordedInvariantViolations</code> +全局遥测,软断言不 crash ;各 <code>return</code> 落结构化退出码 <code>lastExitReason</code> 。架网逼出的两处良构修复:① 停滞交还前给未执行的 <code>tool_calls</code> 补合成结果 <code>(appendSyntheticToolResults)</code> ② 阻塞前先把同回合非阻塞工具执行掉补结果——根治"悬空未应答 <code>tool_call</code> → 续接时网关 400"
循环不变量(架网)	LingShuLoopInvariants.swift (纯逻辑)	差距 1 第一步「架网」。 <code>LingShuLoopInvariants.check(snapshot,at:boundary)</code> 纯函数返回违反:I1 <code>tool</code> 配对(无孤儿/缺 ID)、I2 无未应答 <code>tool_call(.blocked</code> 豁免 <code>pending</code> 那条)、I3 终态后 <code>isRunning=false</code> 、I4 阻塞标志一致(非阻塞终态绝不残留 <code>pending=</code> 会话级 <code>batchInterrupt</code> 泄漏防护)、I5 完成时纠正已采纳、I6 压缩后历史在预算内。 <code>LingShuAgentExitReason</code> 退出原因码; <code>LingShuLoopInvarian</code>

组件	文件	职责 / 关键入口
		tTelemetry 进程级累计计数 (MCP lingshu_status.loopInvariantViolations 暴露、soak 断言恒 0)。守门:LoopInvariantTests(纯逻辑,含"故意注入粘滞标志被抓到")+ AgentLoopFuzzTests(SplitMix64 确定性混沌 ~5000 种子,失败可复现,regressionSeeds[] 棘轮)+ Scripts/soak-e2e.sh(真模型长跑)
真模型适配器	LingShuGatewayAgentModel.swift	LingShuAgentModel 接模型网关,原生 tool_calls 出入,剥 <think>
编排器 + 账本	LingShuAgentOrchestrator.swift	派生隔离子会话(spawn/spawnDetached)、统一账本、事件回灌(onEvent)、续接(resume);drive 收尾后把验收+恢复整段委托给 acceptanceHook(=主状态 verifyAndContinue,撞顶恢复+多轮验收+测试/运行门+停滞交还)——子任务与主线程同一套执行恢复力。派发任务可停 (2026-06-17):存 driveTasks+cancelAllRunning(取消驱动 Task,循环在 Task.isCancelled 边界退出)+activeDriveCount ——修"派发隔离任务后台跑、不计入 hasActiveModelCall → 旧的停止只停主回合、跑飞的 PPT 夺不回 ";cancelCurrentCall 先停派发隔离任务,lingshu_stop 返回 dispatchedStopped
有界并发	LingShuConcurrencyManager.swift	N=3 限流、排队、每线程隔离执行状态。注:spawnDetached 现走硬上限背压(满 3 直接拒绝、不入队),capacity()/hasSpawnCapacity() 暴露给 spawn_task
线程内清单	LingShuTaskPlan.swift	计划/清单数据模型(模型产出的 todo)
语义任务拆分	LingShuTaskSegmenter.swift /	一句话 → N 意图+相关性分组(快

组件	文件	职责 / 关键入口
	LingShuTaskSegmentation.swift	路+模型驱动)。注:类型名 LingShuTaskSegmentIntent(LingShuTaskIntent 已被权限枚举占用)
骨干接线(A+B)	LingShuState+AgentBackbone.swift	mainAgentSession()、runMainAgentTurn、driveAgentDelivery+verifyAndContinue(送 prompt → 跑循环→验收门,主会话/自主运行/答复续跑共用)、agentBuiltinTools(executionPolicy:)(通用工具桥:5 原语+MCP+apply_skill,按 LingShuAgentExecutionPolicy 裁剪)、spawnTaskTool、verifyAgentDeliverable、seededDistilledMemory、recallMemoryTool、identityAnchorMessage
核心循环可插拔 + 嵌套分阶段循环(2026-06-19,新旧循环热切换已落地)	协议 LingShuAgentLoop.swift LingShuAgentSessioning · 工厂 LingShuState+AgentSessionFactory.swift · 新循环 LingShuNestedAgentSession.swift + 纯逻辑 LingShuNestedStagePlan.swift	把"驱动一段会话"抽成协议 LingShuAgentSessioning: Actor(send/resume/continueLoop/injectCorrection/injectBriefing/setTextDeltaSink/isBlocked/turnsUsed/toolInvocations/messages)。两份实现:actor LingShuAgentSession=经典连续循环(.classic,一行未改);actor LingShuNestedAgentSession=嵌套分阶段循环(.nested)。唯一可切换构造点 makeAgentSession(...) 按开关 agentLoopVariant(String raw,持久化 UserDefaults lingshu.agentLoopVariant)返回实现;所有任务型会话(主/自主/派发/spawn)都经它创建;recordIDProvider 参数供 .nested 阶段验收定位记录(经典忽略)。切换 =setAgentLoopVariant(清主/自主 holder 让下回合重建)/MCP

组件	文件	职责 / 关键入口
		lingshu_set_loop_variant{classic\ nested} / lingshu_status.loopVariant。.nested 设计(用户定调):大 LOOP 含多个子 LOOP(阶段)——① 规划(一次模型调用→有序【阶段】,每阶段标 任务/互动;shouldPlanStages 快路启发式:简单请求直通持久 spine 会话=等价经典、零影响,只有有序衔接词+动作 或 互动词+制作词才分阶段;解析失败兜底单任务阶段)② 逐阶段推进:任务阶段起一段经典 LingShuAgentSession 子运行(带前序产物上下文)→复用整套 verifyAndContinue(撞顶恢复+多轮验收+停滞检测+非代码 120s 预算+[验收]通过 trace) 验交付物、不过本阶段内返工;互动阶段不验交付物、开场后让出等主人(完成信号 =isInteractionDone 收口语"没了/结束")③ 大 LOOP 终验=各任务阶段已验+各互动阶段完成,聚合带序号交付并喂回 spine 保连续。全程可打断 (isInterrupted/Task.is Cancelled),继续=从断点阶段续(awaitingResume 状态机,不重头不重新规划);断点续接入口 consumeInterrupt 复位打断标志→内在修掉经典引擎"batchInterruptRequested 粘滞泄漏旁路验收"bug(见 verify-gate-bypass-batchinterrupt-leak)。helper 一次性会话仍直接 LingShuAgentSession()。 实测:584 单测过(+14 NestedLoopTests:规划解析/类型判定/互动收口/断点续接状态机/直通不验/每阶段验收);E2E .nested:多文档"出一个→验一个→终验"✅、做+讲"任务验收/互动不验/没了→终验"✅、断点保存✅;切回 .classic 纯聊天+0 分阶段✅;持久化跨重启✅。机制层(阶段调度/验收/打断/断点续

组件	文件	职责 / 关键入口
		/终验)确定性可靠。三处实战根因修复(2026-06-19,均"主前台空发指令却自发做旧 PPT/卡死"的真凶):① 规划只看真实请求——driveAgentDelivery 对 nested 走 nestedPlanningSendText(guidance+原始 prompt),绝不把"长期记忆·自动召回"块拼进规划文本(否则规划器把召回到的工作目录旧 PPT 当成本次任务凭空重做);长期记忆改由 spine seed + 直通时单独注入 nestedRecallBlock(保留经典"每轮自动召回"能力、不污染规划)。② 外层不重复验收——verifyAndContinue 对 LingShuNestedAgentSession 直接早退(nested 已逐阶段验+终验,外层再验聚合会被 verifier 挑刺→resume→自发起一条"修正"流水线);per-stage 验收传的是内层 classic 会话、不命中早退、照常验。③ per-stage 验收=确定性轻量核对(driveNestedStageAcceptance 查回复声称的产出物文件真存在+非空,extractFilePaths),不复用重型 LLM verifyAgentDeliverable——多阶段共享同一任务记录,LLM verifier 验后阶段时看到前阶段文件会张冠李戴(实测验"广州"却拿"北京建城史"挑刺→无限返工卡死),且慢;确定性核对只看本阶段声称文件→无跨阶段混淆、无 LLM 调用=多阶段从数分钟降到 ~6-9s,与单层引擎同量级。效率实测:直通(简单请求)10s vs classic 14s 同量级;多阶段 2 文档 ~9s。虚假活动澄清:之前误判"真人操作"的自发做 PPT/flow 事件,全是上述 nested bug + standing 回灌/感知;主前台 disarmed 静置 35s 零自发活动。互动阶段调 present_fullscreen 会按系统提示进托管(standing),互动 done 信号在 standing 更干净

组件	文件	职责 / 关键入口
验收 + 执行恢复力	LingShuState+AgentAcceptance.swift	「驱动到验收通过」的统一循环(主/自主/隔离子任务共用):verifyAndContinue= ① 撞顶恢复 recoverFromExhaustionIfNeeded(推进用满 per-run 天花板但有在制品→当检查点补预算续跑,有界 2 次,原地打转不补)② runVerificationLoop(8 轮验收+停滞交还)。 taskHasInProgressWork 判在制品。 artifactBaseline(2026-06-17):验收门只在本回合新增产出物 (currentArtifactCount > baseline)或回复显式声称产出时才触发——常驻在岗会话跨回合复用同一记录,不给基线的话做完 PPT 后"演示/答疑"会被残留的旧 PPT 误拖进验收空转(实测让"演示 PPT"卡死没去演示);在岗/续跑/答复/感知入口都在回合前记基线传入,主会话/隔离子任务基线 0=原行为。非代码交付返工时间预算(2026-06-17):PPT/文档没有确定性测试门,verifier 给的是主观设计意见(每轮不一样)→不触发停滞→返工到 8 轮、叠加云端慢拖成几分钟像卡死。 修:runVerificationLoop 对非代码交付 (deliverableHasCodeArtifact 按扩展名判)设 120s 返工预算,已有真产出物且超时就交付当前版本;代码交付不设(死磕正确性)。单次「看+核」验证器在 DeliveryReview。启动清理:加载到的"执行中"僵尸记录(后台任务不跨重启存活)标"已暂停"。能力升级阶梯(2026-06-20,方案 §2):通用纯逻辑编排 LingShuCapabilityEscalation.swiftrun(goal, rungs, att empt, verify, triggerOf)——对"既能让大脑自由做、又有结构化兜底"的能力,默认从最薄 Rung0(大脑+原语+召回知识)起,verify(真实世界后置校验,maker≠checker)不过/停滞/askUser 才升一级加脚手架

组件	文件	职责 / 关键入口
		(Rung1 引导→Rung2 确定性兜底);升到顶仍不过=诚实交还不假装完成,askUser=立刻交还不浪费升级。脚手架介入频率与脑力成反比(旋钮)。纯逻辑单测 CapabilityEscalationTests(给定 verify 序列断言升级路径)。已并入验收门(2026-06-20):runVerificationLoop 的返工不再"原样重试",改 revisionGuidance(round:critique:) 按返工轮逐级加厚脚手架(Rung0 原样意见→Rung1 结构化引导→Rung2 切确定性兜底);强脑 round0 即过
编排器事件桥接	LingShuState+AgentOrchestration.swift	installAgentEventSink IfNeeded(注入 acceptanceHook=verify AndContinue + 启连接监控)、 handleOrchestratorEvent(子任务→独立记录+对话回灌;含 .interrupted→暂停 /.resumed→续跑)、 briefMainThread、断网重连续跑 resumeSuspendedWork(重连→逐条 orchestrator.resumeInterrupted + 主会话/自主续跑)
断网重连续跑	LingShuConnectivityMonitor.swift · 编排器 suspendedForReconnect/resumeInterrupted/resumeWithInput/driveContinue · LingShuAgentSession.continueLoop()	基础设施故障 ≠ 任务失败:网关重试耗尽 →LingShuAgentModelResponse.failed→循环返回 LingShuAgentRunResult.interrupted(不追加假消息、上下文原样保留)→编排器标.suspended(非 failed)、保留会话、登记待重连。 NWPathMonitor 不可达→可达(去抖 2s)→ continueLoop() 从中断处重发那步模型调用续跑。主会话用 suspendedMainTurn、自主用 suspendedAutonomousRecordID 同理。手动「继续」(submitTaskFollowup)对隔离子任务记录改走 orchestrator.resumeWi

组件	文件	职责 / 关键入口
		thInput(续跑那条隔离会话本身),不再误投主会话
断网主动重试 + 可见进度	LingShuState+NetworkRetry.swift	断网后不被动干等:按退避(5/8/13/21/34/60s)主动重试,主对话框一条原地更新状态气泡显示「第 N 次重试 / 约 Xs 后」;用轻量网关探针 probeGatewayReachable (HEAD 短超时,任意 HTTP 响应=可达)判网络真回来,回来才 fire-and-forget 续跑(任务进「进行中」可见态,不憋到完成才弹)。NWPathMonitor 链路恢复 triggerImmediateNetworkRetry 立即唤醒(重置退避)。语音兜底写死:断网/重连提示走 networkInterruptSpokenLine/networkResumedSpokenLine(不调模型——没网调不动,否则误回退"任务完成")
任务详情多模态面板(2026-06-21,对齐 Codex 右侧)	LingShuWorkspacePanel.swift + LingShuWorkspaceDiffView + LingShuWorkspaceFileTree	任务执行记录子页面右栏从静态 TaskDevToolsPanel 升级为一个面板一键切模式(对齐 Codex):概览(复用 TaskDevToolsPanel:目标/产出物/Git)/审查(git 跟踪的工程:真 git diff hunk 红减绿增,LingShuCodeChangeSummary.repoWorkingDir 持久化仓库根 →LingShuState.resolveGit+runCapturing 就地拉 diff,parseDiff 纯逻辑 WorkspaceDiffParseTests 守;非 git 的新工程(如 run_command 现写出的项目):没 git diff → 把本任务产出的代码/配置文件当「新增文件」列出、点开读真实内容整页绿底展示——根治"写了一堆代码审查却说没改动")/文件(工作区文件树懒加载+筛选→点开 inline 渲染,复用新增 LingShuInlineArtifactPreview 的网页/图片/音视频/QuickLook 多类型)/浏览器(复用 LingShuBrowserChrome)/终端(真·系统命令行 LingShuWorkspaceTerminalView ——用户自己的 zsh,cd 跨命

组件	文件	职责 / 关键入口
		令保持/↑↓历史/clear;Process 执行抽到基础设施 LingShuTerminalShell 守架构守卫"视图不碰 Process")。标签按需开(对齐 Codex):起步只开「概览」,其余经 + 菜单追加、每个可关(概览常驻锚);右栏整列可隐藏(标题栏 sidebar 钮→时间线独占整窗)、可「展开」铺满(折叠 460 侧栏/展开隐时间线给浏览器/终端宽度,宽模式选中自动展开)。把散装能力(预览/浏览器/终端/diff)收拢成统一可切面板。另:参与方过滤栏(子页面顶部)=按真实出场 actor 列 agent(排除 Agent 循环/中枢/系统 内部机制标签 + 用户你),点一个只看它对话(差距 6 可见性落子页面);派发隔离任务的结果/卡住/失败/暂停消息 actor 由子任务 改 灵枢(2026-06-21:派发的隔离 worker 就是灵枢在独立线程干活,其推理已记 灵枢、结果却记 子任务 造成"同一 worker 两个参与方"困惑;统一为 灵枢,生命周期靠 role 区分);定时任务列表(配置→常驻与定时触发)分「运行中/已结束」组 + 每条「执行记录 N」展开看历次到点执行(按 prompt 匹配 taskExecutionRecords) → 点开完整记录
最近产出物记忆 + 运行起来	LingShuState+Deliverables.swift	任务完成有真产出物 → recordDeliverable(任务标题 + 产出物公共父目录 commonParentDir + 完成说明节选)。 recentDeliverablesContext 注入派发隔离任务初始消息 + 主会话 seed → 用户说「运行起来/继续/改一下」时接得上(知道刚做了什么、在哪、怎么跑),不再重新扫盘瞎猜(根治"超级玛丽做完了却问要运行哪个项目")。跨重启从增量存储恢复。 commonParentDir 死循环修复(2026-06-19,真机卡死根因):产出物路径跨不同顶层根(如 /tmp/... 与 /Users/...)时公共祖先缩到 "/",而 ("/" as NSString).deletingLas

组件	文件	职责 / 关键入口
		tPathComponent 仍返 "/" → 旧 while 永真把主线程顶到 100% CPU 卡死(/health 还答但 status/trace 返空=主线程卡死信号;sample <pid> 抓栈定位)。修:到根即止 + 跨根返 nil(CoreBoundaryTests 回归,复发会挂起)
主线程卡死看门狗	LingShuMainActorWatchdog.swift (@unchecked Sendable,纯判定可单测)·挂 LingShuMac.swift .task	独立 Task.detached (绝不挂 MainActor ,否则自身也被卡)周期竞速 MainActor.run{} vs 超时探活:健康时子秒级返回,卡死时排在死循环后永不返回→超时命中。连续 miss (默认 30s×3≈110s ,阈值给宽防误判——健康代码下 MainActor 永不阻塞这么久)→ 记诊断日志 + open 自身强制重启 + exit;shouldRecover/restartsAfterDecay 带重启次数上限(防重启循环)+ 时段衰减清零;重启后读 marker 在对话主动说明"刚从卡死自动重启,说『继续』我接着做"。续作限制:靠 seed 蒸馏记忆 + 最近产出物恢复上下文,不重放卡死那刻正在跑的具体 agent 回合(全内存,Phase 5)。4 单测(CoreBoundaryTests)
增量记忆持久化(Phase 5)	LingShuIncrementalStore.swift (泛型 actor)	参考 MySQL redo log+checkpoint:变更只追加一行 JSON 到 .wal (快,不重写整文件);累计到阈值/定时 (memoryCompactionTimer 5min) compact() 写全量 snapshot + 清空 WAL;启动加载 snapshot + 回放 WAL 恢复(跨 app 重启)。现存最近产出物 deliverableStore ,可扩展存别的记忆。落 ~/Library/Application Support/LingShu/memory/
记忆 v2 知识图谱(2026-06-18)=吸收 Obsidian	Sources/Memory/ (LingShuMemoryNote 原子笔记+Markdown 序列化 / LingShuMemoryGraph 别名归一+双通道召回 / LingShuMemoryGardener	从「embedding 索引的对话日志」→原子笔记+类型+别名归一+双链图谱, dreaming 园丁自维护。真相源=人可读 .md(~/Library/Application Support/LingShu/Memor

组件	文件	职责 / 关键入口
	园丁 / LingShuMemoryVault 磁盘 / LingShuKnowledgeGraph 门面@MainActor)· 接线 LingShuState+KnowledgeGraphDreaming.swift	y/vault/,可 Obsidian 打开/手改/grep)。别名归一复用 LingShuWakeWordMatcher 拼音模糊→治同音污染(林叔/灵枢归一)。混合召回(M1,2026-06-18)=精确实体命中+关键词重叠+语义向量余弦(复用 LingShuLocalTextEmbedding 本地 NLEmbedding 句向量,零网络;治"换种说法"无字面重叠也召回)+顺链图扩展;向量是可重建派生索引(.md 仍真相源)。园丁=并入去重/矛盾消解(权威胜;等权且都用用户明说且矛盾→ .conflict 上报待定不静默择一= M5)/置信衰减/剪枝归档/未链补链/ reinforce 召回反哺= M2 (经 recallText 被召回的 note 置信微增+刷新核验→越用越稳)。召回质量回归测= MemoryRecallEvaluations(M4) 。 M3 收窄版已落(开发期数据可弃→零迁移):事实召回统一走 v2(recallMemoryText 退掉并行 searchColdMemory);「子→主」核心记忆保住+升级=子任务完成 promoteSubtaskKnowledge 把产出蒸馏进常驻 vault(=主线程核心记忆,子线程结束后仍可召回)。 persistedConversationDigest (对话续接)/ TaskMatcher (任务去重)/ deliverableStore (产出物)是不同关注点,故意保留独立、未并入(并入会糊)。旧 LingShuMemoryService 未整删(捕获层留)。接线 additive:state.knowledgeGraph 懒加载;recallMemoryText 并列追加图谱召回(经 recall_memory 工具);dreaming 最前调 consolidateKnowledgeGraph(LLM 抽原子事实→remember→tend,独立一次性会话)。护栏:敏感/空标题拒入、归档有上限、隐私红线(perception-data-zero-retention)

组件	文件	职责 / 关键入口
		不吸音视频/截图)。知识纪律「陈述非祈使」(2026-06-20,方案§3.3):纯逻辑 LingShuKnowledgeDiscipline.swiftclassify——园丁 integrate 落库前过一道:祈使/步骤型候选(有序步骤/「先X再Y」动作链/「…时先…」操作规程/句首祈使词)拒入(返回.skip),知识库只收陈述性事实/教训(根治"知识退化成框架";教训如"写文档≠接入"是事实判断准入)。召回措辞恒为 recallDisclaimer「相关知识·供参考,自行判断是否适用」(recallText 注入),绝不写成规则/必须。零领域关键词、通用。单测 KnowledgeDisciplineTests(正例准入/反例拒入/园丁集成)。决策知识种子(2026-06-20,P0②):Sources/Memory/LingShuSeedKnowledge.swift(数据非代码:CozyLife 本地协议/HomeKit 独占/Bonjour 枚举/设备发现手段/"文档≠接入"教训,全陈述性),knowledgeGraph.seedDecisionKnowledgeIfNeeded() 首启幂等种入(后台跑、不压启动路径),让"知识驱动>框架驱动"有最小可验证起点。前缀缓存安全:召回只进消息后缀/工具结果、dreaming 走独立会话,绝不动 system prompt/早期消息/工具清单那段缓存前缀(MemoryGraph/Vault/E2E 单测+518 全绿;/tmp/lingshu-memory-e2e 留真实.md 产物)
前缀缓存命中率指标	LingShuPrefixCacheMeter.swift(线程安全单例,可单测)·接 LingShuGatewayAgentModel.swift两个调用点	每次模型调用记(输入 token, 命中缓存 token),给累计命中率;日志前缀 prefix-cache: 本轮 hit=.. \ 累计命中率 N%。命中率掉=前缀被打乱的预警(逐轮注入/工具集抖动)。配合 LingShuPrefixCache.swift(策略:DeepSeek/OpenAI 兼容=自动命中;Anthropic=显式

组件	文件	职责 / 关键入口
		cache_control)
朗读内容决策	LingShuState+SpokenReply.swift	spokenReplyText(任务型念摘要/对话汇报念全文)、replyLooksLikeTaskDelivery(纯函数)、briefSpokenSummary、recordSpokenLine(speak 工具调用即留痕到 recentSpokenLines,经 lingshu_status.recentSpoken 暴露)。演示时给每句标真实显示页(2026-06-19):recordSpokenLine 在 previewController.isPresented 时给 speak 文字前缀 [屏显第 X 页] (X=displayedPageNumber=读 pdfView.currentPage 真实显示页,非 pageIndex 变量)→ recentSpoken 呈现"[屏显第 6 页]...第 10 页...", "语音念的页码 vs 画面真实页"对齐与否纯文字客观可判,免音频/肉眼(用户洞察:TTS 念的本质是文字)。配套页码脱节根因修复:LingShuPreviewController.goto 原只命令式 pdfView?.go(to:)(weak, 全屏切换 nil / 紧凑批量来不及 repaint → 变量推进了画面没翻), 改 navigateToCurrentPage () 当帧+下一 runloop 双回正 + 脱节诊断日志(⚠️ 页码脱节)。实测逐页演示 5/5 页 [屏显第 N 页]==念的第 N 页==内容,对齐。不念文件路径(2026-06-19 用户要求):strippedOfFilePaths (纯函数)确定性剥掉绝对路径(整行是路径项→删行/行内夹路径→抹空),任何回复都先过它;任务交付去路径后若已简短(≤140 字,典型"✅完成+路径清单"剥完只剩一句汇报)直接念、不必再调模型摘要。修因:旧 replyLooksLikeTaskDelivery 路径正则漏了 .swift 等代码扩展名→带 .swift 路径的回复没判成任务交付→念了全文含路径(已补全扩展名 + 上面的确

组件	文件	职责 / 关键入口
		定性剥离兜底)
联网搜索子域	LingShuState+WebSearch.swift	webSearchTool(无密钥 DuckDuckGo,可平滑换付费搜索 API)
本地 skill 接入	LingShuState+AgentSkills.swift	applySkillTool(模型按任务调固化技能:用户>策展>内置,返回专家提示+清单)、materializeBundledScript(自带生成器物化,新旧共用;隔离脚本登记运行期隔离表)、matchedSkillHint(命中真 skill 回合广播)
skill 自发现/自安装	LingShuSkillAcquisition.swift (纯逻辑:classify/parseRiskVerdict/rankCandidates/隔离清单持久化)· LingShuState+SkillDiscovery.swift (discover_skill 工具 + 编排:web_search 找候选→下载→分类→reviewScriptRisk LLM 风险审→installDiscoveredSkill 落盘热加载)	自进化 Phase 2 :本地无匹配 skill 时联网找现成高质量技能自动装。安全模型:纯提示直接装;带脚本本过 LingShuSkillSafetyGate 静态门→过门再调大模型风险审→明确无风险才自动装,有风险装但首次运行其脚本强制审批(隔离表 + requestShellApproval 展示风险点,绕过"完全授权")。forceFrontmatterID 命名空间隔离防覆盖内置/策展。红线:绝不静默执行未审来源代码 skill-self-evolution
插件声明式清单+权限(P1,2026-06-18)	Sources/Plugins/ (LingShuPluginManifest / LingShuPluginPermissions 解析 frontmatter perm_* / LingShuPluginPermissionChecker glob+域名作用域匹配·风险评级·越权检测,纯逻辑)·接 LingShuSkillLoader (LoadedSkill.manifest)+ LingShuState+SkillDiscovery (装时亮 scope+风险)	把"skills+捆绑脚本+MCP"往最小权限能力模型收。扩展在 frontmatter 声明 perm_read/perm_write(路径/glob:**跨/、*不跨/、无通配=目录前缀、~展开)、perm_network(域名,*=任意,* .x 含裸域)、perm_shell、perm_system、provides; 缺省=最小权限。riskLevel(系统敏感/跑命令+任意联网或写=高;三者任一=中)。自安装时亮出声明作用域+风险;声明偏高(high)即使代码审 safe 也隔离首次运行(用 manifest 收紧,不只展示)。P2-P5 已落 (2026-06-18,Sources/Plugins/+UI):P2 LingShuPluginToolProvider(插件声明工具+runner,入参 stdin/结果 stdout,经 P3 沙箱

组件	文件	职责 / 关键入口
		跑,产出真 LingShuAgentTool ;统一 MCP+skill; live 注入待有插件真 声明 runner 才激活)·P3 LingShuPluginSandbox (权限→ sandbox-exec SBPL, 只放声明写路径/网络;配置生成已 测,真 confine 要本机验)·P4 LingShuExtensionState Store/Registry (启停+成功 率+ shouldDemote ;启停真 enforcement=CompositeEx pertRegistry.setDisab ledSkillIDs 过滤 + syncExtensionEnablem ent)·P5 LingShuExtensionsPane l (配置→技能 tab ,统一管 skills+MCP;UI 需眼检)。20 单 测过。红线仍 skill-self- evolution 绝不静默执行未审代码
内核 ABI 固化(M0,2026-06-20)	LingShuKernelABI.swift (version + 五协议清单单一真相源)· Docs/灵枢内核 ABI.md ·守门 KernelABIContractTests	把"核心固化→自我编程外围→可 插拔进化"的五大内核协议显式 化、版本化:① 核心循环 LingShuAgentSessionin g ② 工具 ABI LingShuAgentTool ③ 外围 runner 契约 LingShuPluginToolProv ider (stdin/stdout+P3 沙箱)④ 感知输入 LingShuExternalSensor ySource+Reading ⑤ 清单 /权限 LingShuPluginManifest 。契约测试逐字段穿透 + 版本钉 死:协议形状变即编译/断言红。外 围只经这五协议接内核,内核不被 改坏。 M2 已补:感知 Hub 运行时 动态注册口 registerSource/unregist terSource (原在 init 写死)
内置技能注册表(2026-06-27,硬性架构要求)	协议 LingShuBuiltinSkill.swift · 样板 LingShuPresentationSkill.swift ·引擎 LingShuPresentationController. swift (纯逻辑可单测,Hooks 注入)	硬性要求(用户定调):技能分内置 /外部两类,信任边界不同→代码归 属不同。① 内置技能=可信(随 app 出厂签名)→可带原生 Swift 代码,但代码归各自技能模块(实现 LingShuBuiltinSkill 协 议),绝不糊进内核

组件	文件	职责 / 关键入口
		(LingShuState);② 外部技能=运行时来的(discover/author/r ecord) → 不可直接跑原生,必须沙箱/声明式/子进程 runner(见 <u>plugin-manifest-permissions</u>)。内核持 LingShuState.builtinSkills: [any LingShuBuiltinSkill], 一律遍历协议统一调度,内核里不出现任一具体技能的专属 if/调用:tools()(并入工具表,经 previewTools() flatMap)/invocationEntry (@ 菜单,invocablePlugins compactMap)/routeDeclarative(声明式路由)/interceptActiveInput(活动中拦输入=演示答疑,submit + standing 两处遍历)/onCancel(cancelCurrentCall/abortActiveFlow/exit Presentation/ stopAutonomousRun 取消统一遍历,须在 interruptSpeechOutput 之前)/onPause(pauseActiveFlow)/onResume(自主「继续」从演示暂停页续)/mount(host:)(init 把宿主交各技能取内核服务)。演示与答疑是首个迁入的样板(原 LingShuState+Presentation.swift 已删→移进 LingShuPresentationSkill,经 weak host: LingShuState? 取预览/语音/聊天/控制面模型);LingShuState.presentationController 降为仅视图层(预览宿主/进度条渲染)的计算属性 { presentationSkill.controller },编排逻辑不点名。演示「启动」只走显式 @演示(routeDeclarative)+ 大脑 present_documents 工具——2026-06-27 用户定调删掉关键词嗅探启动路由(原 detectPresentationRequest/claimRequest:"句子

组件	文件	职责 / 关键入口
		里只要提到'演示'+带文档路径就误开演",过度误触发)。加/换内置技能=加一个实现协议的模块 + 注册进 builtinSkills ,不碰内核。落地 ABI「核心固化→可信原生外围」。待迁:录制技能(record)仍内联,后续按此样板迁入
设备发现(M3 引导四肢,2026-06-20)	纯逻辑 LingShuDeviceDiscovery.swift (串口/USB/蓝牙/电源 解析 + 驱动缺口分析)· 工具 LingShuState+DeviceDiscovery.swift (discover_devices, 跑只读 system_profiler/ioreg/ls 枚举真硬件)	可插拔进化第一步「发现没驱动的新外设」:枚举本机真接入硬件 + 标哪些没驱动组件(交叉比对已注册源)→ 引导大脑对缺口设备 author_component(component_kind=sensor) 写专用驱动。挂主会话+子任务。实测枚举出真串口/17 蓝牙外设(含 RSSI)/电池控制器。2026-06-20 撤定制(\$4 #9):删 usbserial/ftdi/slab/battery/电池 ... 品牌/品类关键词—— parseSerialPorts 不再猜"这是什么设备"(只过滤 OS 伪端口)、 coverageTokens 改用设备真实名/标识 token 匹配驱动,设备识别全交大脑 classify/probe
自我编程外围组件(M1 工具型 + M2 传感器型 + M3 硬件驱动 + M4 执行器型,2026-06-20)	纯逻辑 LingShuComponentAuthoring.swift (Spec → skill.md 组装/校验,tool/sensor/actuator 三 kind)· 执行安全 LingShuActuatorSafety.swift (reversible/physical;2026-06-20 撤定制:零关键词清单,风险由大脑在 author_component 显式声明 actuator_risk ,不再靠 motor/锁 ... 关键词猜——安全靠 physical 每次确认门,不靠关键词命中)· 编排 LingShuState+ComponentAuthoring.swift (author_component 四肢 + 闭环 + sensor 注册/重启加载/审核启用 + actuator 风险消息)· 门 LingShuState+AgentSkillsquarantineGatedTool (隔离首次审批)/ actuatorGatedTool (physical 每次执行确认)· 传感器源 LingShuRunnerSensorySource .	闭环:需求→大脑自写 runner+清单→校验→①静态门(危险即拒、绝不试跑)→② P3 沙箱 runRunner 真试跑(传感器还验读数可解析)→③ LLM reviewScriptRisk → 无风险且权限不高才自动上线;有风险/权限偏高→ setQuarantine 隔离。 M1 动作型:暴露 live 工具(userSkillProvidedTools),隔离的经 quarantineGatedTool 首次运行强制审批。 M2 传感器型(component_kind=sensor):造 LingShuRunnerSensorySource 周期跑 runner → 读数 JSON 解析进感知链,经 Hub registerSource 动态注册+启用→ perceive 拉得到; loadAndRegisterSensorComponents 跨重启重注册;隔离的不自动启用、

组件	文件	职责 / 关键入口
	swift · LoadedSkill.frontmatter 带 sensor_channel/actuator_risk	<code>approveAndEnableSensor</code> 审核后启用。 <code>situationContribution</code> 改为待办+近期读数都呈现(否则传感器读数被待办挡住)。挂主会话+派发子任务。红线 skill-self-evolution :绝不静默上线/启用未过门代码。 <code>.app E2E</code> (调试钩子 <code>lingshu_author_component/lingshu_call_authored_tool/lingshu_perceive/</code> <code>lingshu_enable_sensor</code> 确定性驱动真实流水线):动作型 纯计算自动上线+真调通✓ 联网隔离+运行期审批拦截✓ 危险静态门拦✓;传感器型 自编系统负载源→动态注册→数据进感知链 → <code>perceive</code> 拉到✓ 跨重启持久化 ✓ 危险拦下不注册✓;M3 硬件驱动 <code>discover_devices</code> 枚举真硬件+缺口→自写电池管理控制器(IOKit)遥测驱动→沙箱读真电池(温度/电芯电压/循环数)→隔离审核→上线→缺口闭合→ <code>perceive</code> 真感知✓(外接 ESP32 串口同机制、待真外设人工验);M4 执行器自编系统音量执行器 (<code>reversible</code>)→首次审批→真改硬件输出(外部核验音量=25)✓ 自编舵机执行器(<code>physical</code>)→每次执行确认门阻塞、不静默触发✓(真舵机/继电器同机制、待真外设)。模型自主选该工具受 deepseek-agentic-ceiling 限,机制无关。 2026-06-20 撤定制 + 真台账:#8 保留名自动派生自实际内核工具目录 (<code>kernelReservedNames(catalogNames:)</code> 把 <code>LingShuFunctionCallingCatalog.builtin</code> 并入,新增内核工具自动覆盖,不手维护清单);#5 执行器上线即把「接入了什么」记进知识图谱 (<code>recordIntegrationKnowledge</code> 一条陈述性原子事实,问"接入了什么"靠召回作答、不靠扫描/固定 <code>schema</code> 台账)
统一外设中枢(2026-06-20)= 所有东西都是外设	模型 LingShuPeripheral.swift (transp	一个已连接外设列表,所有来源(mDNS 网络/串口/USB/蓝牙/电

组件	文件	职责 / 关键入口
	ort 客观 + classification 大脑判,nil=待归类;access 受限词表保路由)· 中枢 LingShuPeripheralHub.swift(NWBrower mDNS + setExternalPeripherals 灌入串口/USB/蓝牙/电源/传感器/组件 + 本机音量控 + applyClassifications)· 接线 LingShuState+Peripherals.swift(refreshPeripherals 汇集所有来源 + classifyConnectedPeripherals 一次性 model 调用让大脑自动分类分组 + peripherals 工具 + adoptPeripheral 按 access 路由)· 面板 LingShuPeripheralsView.swift(配置→连接器→已连接外设,按大脑 displayGroup 分组)	源/传感器/自编组件/本机)汇一处;分类分组不硬编码、由大脑在连接时判(原 classify switch 已删——别用硬编码控制器替大脑决策)。检测通用(Info.plist NSBonjourServices+NSLocalNetworkUsageDescription,Resources + build-app.sh heredoc 两处;首扫需一次「本地网络」授权)控制分适配器:本机(音量,已可控)/开放协议(自写驱动)/HomeKit(需原生 HMHomeManager+entitlement,且苹果家庭单控制者独占)/Matter·米家(需码/token)。MCP lingshu_peripherals_scan/lingshu_peripheral_control/lingshu_author_component。灵枢是 AI 非工具:classification 带 canonical(归一键合并多通道)/alias(语义别名取代产品代号)/what/deviceType/capabilities/integratable;大脑连接时识别(classifyConnectedPeripherals)、label_peripheral 深度探测写回、probePeripheral 主动 investigate;状态流 待探测→可接入→已接入·对话控制(去掉死"可控"徽章);已接入由大脑判 (2026-06-20 撤定制 §4 #4):classifyConnectedPeripherals 带"已有驱动 actuator_target"上下文让大脑判每台 integrated(parseIntegratedPeripheralIDs 纯解析),取代原 markIntegratedPeripherals 子串匹配(已删);面板按 deviceType 分组 canonicalKey 合并;openLocalNetworkSettings 一键直达权限页。真证据:31 条原始外设→大脑语义别名(罗技鼠标/卧室音箱/CozyLife 床头灯)、3 鼠标归一、CozyLife 灯[已接入·可对话控制];对灵枢说「把床头灯打开」→自映射 cozylife 工具→灯真亮(对话控制

组件	文件	职责 / 关键入口
		端到端)。 PeripheralsTests 6
自主运行(执行)	LingShuState+AutonomousRun.swift	<code>authorizeAutonomousRun</code> → <code>launchAutonomousExecution(agent 循环驱动)</code> 、权限级 → 执行策略映射、 <code>handleAutonomousAnswerIfNeeded(ask_user 续跑)</code> 、暂停/停止取消在飞 Task。 objective 空=常驻灵枢(系统提示/kickoff/收尾按空目标分支:在岗示意而非收工)
常驻灵枢(独立运行新形态)	LingShuState+StandingPerson.swift	用户定调 2026-06-16:独立运行=一个"人"不是一次性目标。 <code>goLiveAsStandingPerson</code> (环境自检通过即直接上岗, <code>objective=""</code>)、 <code>isStandingPersonOnDuty</code> 、 <code>handleStandingPersonInputIfNeeded</code> (在岗时对话/语音直接喂在岗执行会话=带权限级与四肢真去做,经 <code>submitTextInput</code> 在 <code>autonomous</code> 命令后接管)。单线程·打断即放弃(用户改定 2026-06-19 ,推翻"插话注入续跑"):在岗自主是单线程——执行中被打断,直接放弃执行中的工作(置 <code>batchInterruptRequested</code> 停批量 + <code>interruptSpeechOutput</code> 掐 TTS + <code>autonomousRunTask.cancel</code>),起一条全新回合处理新指示(<code>resumeStandingSession</code> 内 <code>await previous?.value</code> 等被取消的上一条真停下=单线程不重叠;取消在回合边界生效、工具结果本迭代内补齐故历史良构, <code>resume</code> 安全)。 "要不要继续"交大脑据会话上下文判:打断时发的是 <code>interruptedResumeFraming</code> 包装("你手头的事已停下,新指示:X;若是'继续'就从断点接着做、否则放下改做新的")——会话历史留着,故"继续"=从断点续、其余=改做新指示。不再用 <code>injectCorrection</code> 把新话

组件	文件	职责 / 关键入口
		塞进正在跑的回合(那是"边跑边改"的多线程语义,与单线程相悖、实测易乱)。空闲则直接 resume 起新回合。实测:自我介绍执行中发"停,现在几点" → trace 打断·放弃执行中 → 放弃介绍、答出时间。去污染(修 bug #1):resumeStandingSession 不再前置感知态势——直接问的问题干净地答(要看屏自己 screen_capture),standingPromptWithPerception 已删(它把"介绍一下你自己"污染成"在岗待命")。复用 enterAutonomousRunningState/finishAutonomousRun(已去 private)。配套周期感知循环(下行)让它不只被动应答。上岗即开麦收听(2026-06-16):beginAutonomousActivity 调 startStandingVoiceListening?()(根视图注入,起一个 LingShuVoiceCallController=极简模式同一套:麦克风 → ASR → submitVoiceTranscript → submitTextInput(.voice) → 在岗会话 → 思考 → TTS 回应),endAutonomousActivity 调 stopStandingVoiceListening?() 关麦。「耳·麦克风」与「眼·屏幕」「耳·系统声音」是各自独立的输入通道:麦克风=听主人说话 (VoiceIOManager mic),系统声音=听电脑/会议 (LingShuSystemAudioCapture),屏幕=看 (screencapture → VL)。在岗语音=智能切换唤醒(用户改定 2026-06-17,推翻"在岗一律连续不唤醒"):单独 1:1 跟它说=连续对话不用喊唤醒词;一旦在岗 + (会议中 meetingDetectionState.inMeeting 或多人 multipleSpeakersSuspe

组件	文件	职责 / 关键入口
		cted)→ 要求先喊「灵枢」才应答,没喊只听不答(语音仍进会议纪要/感知,不对会议里别人每句话搭腔)。判据在 LingShuPerceptionActions(ambientGated);喊「灵枢…」过门、stripWakeupWord 剥唤醒词得指令(tail 剥成空时返回原句,只喊"灵枢"=呼叫不丢)。极简通话恒连续(显式 1:1)。灵枢正说话/在跑长回合时,新整句先掐 TTS,再走单线程打断即放弃(见上:取消当前回合+起新回合,不再"插话注入")。执行中忙音(2026-06-19):处理中 (autonomousRunTask != nil hasActiveModelCall loopPhase.isActive)且不在朗读(!isSpeakingOrQueued)→ LingShuCueSound.busyTick() 每 ~2.4s 响一声很轻的低"嘟"(Submarine,音量 0.28,被 AEC 抵消不自触发),告诉主人"在处理、没死";一开口朗读或空闲即 busyStop。两个易错点(都踩过):① 信号必须含 autonomousRunTask != nil——在岗/自主的纯文本回合(对话/对比这类不调工具)既不置 hasActiveModelCall 也不置 loopPhase,漏了它=自主模式没忙音;② 驱动不能挂 UI coreTimer 也不能用独立 Timer——在岗/自主时两者都被 App Nap/窗口遮挡系统暂停(与定时触发器同因),改由 beginAutonomousActivity 那条已抑制 App Nap 的 1s 自驱 Task(LingShuState+AutonomousPerception)每拍调 busyTick(内部节流真响);前台主会话那条仍由 coreTimer 兜。busyTick 不自持 Timer(节流靠 lastBeepAt)。实测控制日志 busy-tone: 起忙音 … play=true。同频切本体色 (2026-06-19,等待不无

组件	文件	职责 / 关键入口
		聊):busyTick 每响一声 busyPulseIndex += 1;digitalHumanSnapshot 处理中(同口径)给 LingShuDigitalHumanSn apshot.pulseIndex,其 accent 据此在 busyPulsePalette(青→蓝 →紫,全息冷色系)间切——本体在 TimelineView 每帧读到、 与"嘟"声同步切色(用 Int 序号 传、不把 SwiftUI 引进状态逻辑 文件)。处理中展示当前阶段 (2026-06-19):orb 的 standingStateChip 在 phase==.processing && loopPhase.isActive 时显 示 loopPhase.rawValue(理解 中/规划中/执行中/验收中)+ 对应 图标,而非笼统"处理中";纯文本回 合(无工具→loopPhase idle)仍 显"处理中"(确无子阶段)。本体下 方文字 (digitalHumanSnapshot .displayText)早已随 LOOP 环节显示阶段。注意 multipleSpeakersSuspe cted 可能被 TTS 外放回灌/系 统声音误判→solo 房间偶尔要求 喊唤醒词,如扰民可降级为仅 inMeeting 触发。唤醒词匹配抽 到纯函数 LingShuWakeWordMatcher 做 宽松同音近音(灵书/铃枢/凌枢…, 修"喊灵枢唤不醒")+ 指令剥 离;requiresVoicewakewo rd 仅在非连续态生效。进极简模 式时停在岗收听(避免双麦,根视图 onChange)
周期感知循环(自主模式 模块 2)= 「眼/耳」通道	纯逻辑 LingShuPerceptionCadence.swift (LingShuPerceptionCa dencePlanner.decide/ LingShuAudioActivityD etector,可单测)· 接线 LingShuState+AutonomousPer ception.swift · 廉价签名 LingShuComputerContro l.frontmostAppToken+w	在岗(完全接管)时定时感知屏幕+ 系统声音 → 维护滚动 perceptionDigest(感知= 输入,要不要行动/做什么由大脑综 合评判,不是写死引导程序)。两段 式心跳(见不变量):① 廉价闸门 (前台签名变没变 + 系统音频电平 突变 LingShuAudioActivityD etector)② planner 节流 VL(min 9s + 无变化也 45s 强制

组件	文件	职责 / 关键入口
	<code>windowTitleSignature</code> · 缩图 <code>downscaledJPEGBase64</code>	刷新 + 大脑在跑回合时让位)③ 截屏缩 1280 长边 JPEG (4MB → 网关 500;缩后 <300KB) → <code>cloudPerceptionClient.analyzeImage</code> → <code>digest</code> ④ <code>digest</code> 只显示在接管态遮罩 (2026-06-17 起不再前置进指令, 避免污染直接提问)。屏幕只静默监测(用户定调 2026-06-16 ,修 bug #1b):自主反应武装 (<code>autonomousAutoReactArmed,opt-in</code>)时世界变化(音频起音 或 前台/界面切换)走 <code>wakeStandingPersonWithPerception</code> → <code>evaluatePerceptionForAction</code> (一次性临时会话,不污染在岗对话)="屏幕异常哨兵",门槛抬高,只有报错/崩溃/卡死/危险操作等真问题才 ACT 出声提醒一句,日常变化一律 IDLE 不打扰。这是「屏幕」感知通道,与「摄像头态势」通道完全独立:屏幕走本扩展 (<code>screenshot</code> → VL → <code>perceptionDigest</code>),摄像头走 <code>VisionIOManager</code> → <code>perceptionGateway.ingestVision/VideoFrame</code> → <code>latestSnapshot</code> (根视图 <code>onReceive(vision.\$...)</code>),两者互不引用、可各自单独运行
外接设备感知框架(第六感, 2026-06-17)=可插拔感知模块	纯领域 Sources/ExternalSensory/(LingShuExternalSensoryModels/LingShuExternalSensorySource 协议 /LingShuANCSProtocol 纯解析/ LingShuPhoneTodoDistiller 纯蒸馏)· 模块 <code>LingShuANCSSensorySource(BLE)/LingShuEventKitSensorySource(EventKit)</code> · 汇聚 <code>LingShuExternalSensoryHub</code> · 接线 LingShuState+ExternalSensory.swift · UI LingShuExternalSensoryView.swift (配置→外接设备 tab)	用户定调:所有外接设备感知=各自独立模块、独立线程(ANCS 用私有 BLE 队列、EventKit 用 store 队列),在汇聚阶段 (<code>LingShuExternalSensoryHub</code>)归一成一套标准输入 <code>LingShuExternalSensoryReading</code> ,经 <code>LingShuSituationContext.ExternalSensoryComponent</code> 与视觉/听觉并列注入大脑综合评判(类比 视觉/听觉/触觉)——只供事实,不替大脑决策。 <code>LingShuExternalSensorySource</code> 协议 + <code>AsyncStream</code> 让无缝切换=启用即起线程订阅、停用即停。上游可插

组件	文件	职责 / 关键入口
		拔:ANCSNotificationSource(iPhone 通知,\$M1-M5 结构已落,M0 BLE 链路待真机配对验证=macOS 作 central 能否订到 ANCS 是头号风险)+EventKitSource(日历/提醒,零配对、实测可用,\$8 最稳起步源),未来加模块只在makeDefault() append。下游数据源无 关:TodoDistiller(读数→关键待办,模型驱动 +heuristicDistill 纯函数兜底)。隐私 perception-data-zero-retention:默认全关、本地只读、正文不落盘、关闭即清空内存;ANCS 配对需 iPhone 上显式确认。 LingShuANCSProtocol(8 字节包/Control Point 请求/Data Source 分片重组)+ 汇聚/蒸馏全有单测(ExternalSensoryTests,23 例过)。UI 已移到 配置→连接器→外设连接器(连接器拆成 MCP 连接器/外设连接器两子 tab)。 M0 实测=纯 Mac ANCS 走不通(iOS 不向第三方 Mac app 开放 ANCS,扫不到可配对设备;macOS CoreBluetooth 也当不了 solicited-peripheral+GATT-client),UX 已改诚实(12s 超时→不可用+重试/打开蓝牙设置按钮);真读 iPhone 通知的现实路线=ESP32/nRF 外接桥跑 ANCS 固件→USB 串口喂 Mac(再加一个 source,下游全复用)。蓝牙名 i18n:ANCS 加 CBPeripheralManager 以 appName(灵枢/Nous)广播,随 language didSet 实时切。蓝牙不可用→警告+自动关闭:CBPeripheralManager 状态非 poweredOn → 源发 .fatal → Hub 弹 warning(UI.alert) +disableSource。多模态合并进大脑(2026-06-17 重构=感知链):用户拍板"感知节奏与大脑节奏解耦"——各感官独立线程持续采集,由

组件	文件	职责 / 关键入口
		LingShuPerceptionChain(Sources/Services/Perception/LingShuPerceptionChain.swift, @MainActor 有界纯内存融合缓冲)以 ~1s 高频把每通道(视觉/听觉麦克风/听觉系统声音=会议 ASR 转写,独立 ambient 通道,门控 LingShuMeetingASR.shared.isRunning;在岗自动起纯听半边 startStandingAmbientListening(系统音频→ASR→链,不走虚拟麦应答),离岗停。会议模式无 UI 按钮、仅 MCP 触发。会议纪要(在岗自动,2026-06-17):LingShuMeetingDetector(纯逻辑反射:前台 app bundle/窗口标题/声音活动判进出会议,exitGrace 60s)+LingShuState+MeetingMinutes.swift。进会 →beginMeetingMinutes 分段累积转写(每 40s rotateMeetingMinutesSegment:落定当前段+重启 ASR,绕开单次 SFSpeech 会话时长上限;append 本身不阻塞,所以不丢音)→离会 →endMeetingMinutesAndGenerate 拼完整转写交在岗会话:大脑生成结构化纪要 →write_file 落档(优先 docx/run_command python-docx,不行则 md)→speak+回复摘要+路径主动推送。检测挂在 perceptionApplySignal (在岗周期感知)。隐私:转写仅内存、生成后清空分段。检测纯逻辑 7 单测过 (MeetingMinutesTests)。系统通知:LingShuNotificationCenter(UNUserNotificationCenter 封装,前台也弹横幅)+ 大脑工具 push_notification(LingShuState+Notifications.swift,挂主+自主会话);纪要生成完会 push 一条横幅(人不在也看得到)。系统权限面板(配置→系统配置底部 LingShuSystemPermissi

组件	文件	职责 / 关键入口
		onsPanel):屏幕录制 (CGPreflight/RequestScreenCaptureAccess)+ 系统通知,可提前主动授予(避免录制/推送时人不在却没授权而失败),点「授予」触发系统授权 + 跳系统设置对应面板/屏幕/外接设备/情境)此刻内容投进链 (perceive 工具已挂主会话 + 自主/在岗会话)(连续相同只刷时间戳不堆积);大脑做决策时调 perceive 工具按时间窗(默认 5s)瞬时拉取已备好的多模态态势——不必当场触发昂贵 VL 再等=快。采样器 startPerceptionChain(1s Task,samplePerceptionChainOnce)只读已缓存态、不触发传感器;活感官门控 (liveSenseSampler 根视图注入,仅 vision.isCameraRunning/voice.isRecording 真在跑才投视觉/听觉,免陈旧冒充实时);屏幕语义昂贵节流→由在岗 VL 单独 note(.screen),formattedWindow 对慢通道放宽并如实标龄(18s 前)。 perceptionDigest(屏+声+外接)仍服务在岗遮罩/哨兵。注:旧 composedPromptHint/SituationContext/query_external_signals 注入路径已弃,统一走感知链。双引擎 ASR 并发 (2026-06-17,修"在岗说话不回复"):麦克风+系统声音都用 SFSpeech 会互相饿死(Apple 不让两个 SFSpeechRecognizer 并发)→ 上岗时若 SenseVoice/sherpa-onnx 可用,把麦克风切到它(独立本地引擎,standingPrevMicProvider 离岗还原),腾出 SFSpeech 给系统声音,两路并发;没装则麦克风留 SFSpeech、不起系统声音 ASR(兜底)。SenseVoice 由 Scripts/install-sensevoice.sh 下

组件	文件	职责 / 关键入口
		载(~280MB,Models/SenseVoice,gitignore),build-app.sh 自动装+打进 .app 并 Apple Development 重签
自主模式 = 化身右上角本体(模块 1,2026-06-17 重定义)	LingShuAutonomousIntroOverlay.swift · 根视图 LingShuPrimarySurfaces.swift	用户定调:自主模式开启=灵枢界面整个消失、化身为右上角一颗半透明悬浮本体,除本体外别无一物(取代旧「边缘辉光+控制条」遮罩,LingShuTakeoverOverlay.swift 已删)。两段:① 进入仪式(isStandingPersonOnDuty false → true)整屏覆暗幕(界面"融化")→ 120 颗青色离子从屏幕各处汇聚右上角凝成本体(冲击波+绽放,纯 Canvas LingShuAutonomousIntroOverlay)→ 约 2.5s 暗幕盖着时切终态;② 终态(autonomousOrbMode,根视图 @State)LingShuAutonomousOrbOnlyView=只画半透明本体 + 右键菜单(暂停/继续、解除自主模式 stopAutonomousRun);LingShuAutonomousWindowController(NSViewRepresentable)把主窗口收缩成右上角小浮窗(116pt)+ 透明背景 + 隐藏标题栏/红绿灯 + .floating,退出自主模式时完整还原(尺寸/不透明/标题栏/红绿灯/层级)。kickoff 招呼语="现在进入自主运行状态..."。务实兜底:右键「解除自主模式」任何时候可回正常界面
灵枢身体表现层	LingShuDigitalHumanModels.swift · LingShuState+DigitalHuman.swift · LingShuDigitalHumanViews.swift	Jarvis 风格光球灵枢。Domain 定义表现态/信号/指令;State 将大脑 set_digital_human 指令或真实状态(听/说/看/思考/执行/异常)归一成 snapshot;View 只消费 snapshot,将左上角灵枢本体头像渲染为动态光球、状态环和信号点。边界:身体层只呈现,不占用对话区,不判断任务、不调模型、不创建线程。发声声纹/嘴部只看真实输出电平(outputLevel),不看 TTS 请求中这种粗状态;网络卡顿时表现同步静默。speak 只触发 TTS,

组件	文件	职责 / 关键入口
		是否出现发声特效由播放电平决定;主/自主/隔离子会话均可调度
通用工具执行	LingShuState+PipelineExecution.swift	runAgenticTool(6 原语经 LingShuToolExecutor,带权限门控,write_file/edit_file 自动登记产出物)、unwrapMCPArguments;agent 工具桥与外部 MCP 共用。 run_command 前接 LingShuShellCommandPolicy(只读免审批 / 系统敏感强制审批)+ 危险判定 LingShuCommandSafety(纯逻辑:拦提权/抹盘/关机/写裸块设备 >/dev/disk...;放行 >/dev/null 等伪设备重定向——2026-06-21 根治旧宽串 "> /dev/" 误拦后台启动服务的家常写法 → SpringCloud 等服务类任务卡在"启动验证"永远交付不了;CommandSafetyTests 棘轮)+ 系统提示给起长时服务指引(后台跑+日志重定向到文件不是 /dev/null、sleep 后读日志验『Started/Listening』再 kill)+ 凭据占位符注入 ({cred:KEY}} → 执行层换真值、输出打码)。旧 pipeline* 调用已删
命令分级策略	LingShuShellCommandPolicy.swift	纯函数(可单测):isReadOnly(grep/find/ls/cat.../git status 复合命令逐段判)= 免审批直放;touchesSystemSensitivePath(删/改/System、/usr、/etc、内核扩展...)= 即使完整授权也强制弹审批。executor 与 state 工具桥共用
凭据四肢	LingShuState+Credentials.swift	方案 B:大脑只见 key、不见明文。 remember_credential(存进 credentialStore AES-GCM 库,不回显)/list_credentials(只列 key 名)/resolveCredentialPlaceholders({cred:KEY}} → 真值,执行层注入)/redact

组件	文件	职责 / 关键入口
		Secrets(输出打码)。明文不进模型上下文/任务记录。模型配置加密导入导出(2026-06-20,可分享/开源/数据安全):纯逻辑 LingShuModelConfigPortability.swift(LingShuModelConfigBundle 打包当前脑/通道/全部密钥 → 口令 PBKDF2-HMAC-SHA256 派生密钥 + AES-GCM 加密,非本机绑定所以换机/给别人能用同口令解开;凭据库本机绑定密钥换机解不开,故这里另用口令)· 接线 LingShuState+ModelConfigPortability.swift(exportModelConfig/importModelConfig 采集→落盘 0600 / 解密→恢复密钥+通道+脑选型+重建会话立即可用;credentialStore.allCredentials/bulkSet)· UI LingShuModelConfigPortabilityView.swift(模型通道页头下「导出加密配置/导入配置」+ 口令弹窗 + Save/Open 面板)· MCP lingshu_export_model_config/lingshu_import_model_config。安全:导出文件零明文(单测+活 E2E 实证)、口令错/篡改 GCM 校验失败拒绝、弱口令(<8)拒绝。单测 ModelConfigPortabilityTests 7、活 E2E 导出→零明文→错口令拒→正口令一键恢复 4/4
计算机操作四肢	LingShuComputerControl.swift (能力层:截屏 screenshot / 鼠标键盘 CGEvent / 滚动 / AX 列可点元素+坐标)· LingShuState+ComputerControl.swift(工具桥)	计划 §9 加工具不加控制器:screen_capture(截屏→VL 描述+OCR)/list_ui_elements (AX 拿元素中心坐标=可靠点击首选)/click/double_click/move_mouse/type_text/press_key/scroll。授权门=computerControlEnabled(设置开关,持久)或完整授权独立运行;动作类再要系统辅助功能授权 (computerControlGate)

组件	文件	职责 / 关键入口
		。坐标统一点、左上原点
后台守候+完成即续	LingShuState+BackgroundWatch.swift	"自动识别需求→无人值守推进"的可靠机制: <code>watch_until</code> (检查命令+满足标志+满足后做的事)挂一个后台 Task 周期轮询(不阻塞对话、 <code>survive</code> 窗口关闭), <code>watchConditionMet</code> 判定满足→ <code>fireWatch</code> 把后续指令注入主 agent 会话续跑;超时则转自查。 <code>list_watches/cancel_watch</code> 管理。授权:只读检查直放,否则一次性审批(轮询不每次弹)。 <code>shellPreauthorized</code> 判预授权
定时调度(时间点触发,四肢)	工具 LingShuState+ScheduledTasks.swift · 底座 LingShuScheduledTriggerService.swift · 接 LingShuState+AutonomousPerception.swift (后台安全驱动)	与 <code>watch_until</code> (等条件)并列的到时间点触发四肢: <code>schedule_task</code> (title/prompt/hour/minute/repeats_daily/可选 date → 算 <code>fireAfter</code>)/ <code>list_scheduled_tasks/cancel_scheduled_task</code> 。底座 <code>LingShuScheduledTriggerService</code> JSON 持久化(跨重启),到点 <code>fireDueTriggers</code> → <code>submitTextInput(trigger.prompt)</code> 把指令交回完整 agent 循环(提醒就开口、任务就动手,远胜静态通知)。根因修复(2026-06-19):① 此前大脑无任何调度工具→任何"提醒我/每天 X 点"请求都即兴写 <code>launchd plist/crontab/shell</code> 脚本谎称"已设自动运行"(实测假象: <code>plist</code> 没 <code>launchctl load</code> 、放错目录、 <code>triggers.json</code> 从未生成),补 <code>schedule_task</code> 工具族 + 主会话/派发任务系统提示明令"定时用 <code>schedule_task</code> ,绝不写 <code>launchd/crontab</code> 假装";② 派发隔离任务原是精简工具集、没挂 <code>schedule_task/watch_until</code> (请求被分诊成"task"派发后即兴伪造的真凶),已把两者补进派发任务工具集;调度四肢现已在全部会话上下文统一挂载=主会话 + 派发隔离任务 + 自主/在岗会话(贾维斯

组件	文件	职责 / 关键入口
		主模式,原也漏挂)+ spawn 子会话(光给主会话加没用——请求落到哪个 session 就得在哪有工具); 各处系统提示同步加引导 (autonomousSystemPrompt 因此拆到独立文件 LingShuState+AutonomousPrompts.swift 守 ≤500 行);③ fireDueTriggers 旧实现要求 now 分钟精确等于 hour:minute 才触发,而驱动它的 UI coreTimer(Timer.publish) 在窗口遮挡/后台被系统暂停(与周期感知被迁出 UI 定时器同因)→到点那分钟没人 tick=永久错过;改为「到点即触发 + 有界追赶窗口(默认 1h,错过那分钟也补、过期太久不补)+ 当次去重」,并在后台安全的自主驱动 Task 里也每秒驱动一次 fire(在岗 UI 定时器冻结也能触发)。纯逻辑 8 单测 (ScheduledTriggerTests)+ 真机实测(每日 9 点 & 一次性到点真触发)
验收门(看+核)	LingShuState+DeliveryReview.swift	verifyAgentDeliverable(真实落盘 U 回复提到且存在的文件→看图审版式 + 据知识核事实,忽略自指/易变事实:文件自报体积/页数/时间戳、KB 四舍五入)+ 代码任务确定性硬门(产出真源码→① codeTaskTestEvidence: 有测试且全绿 ② codeTaskRunEvidence:最近一次构建/运行程序本身不崩(若跑过)—— looksLikeBuildOrRunCommand/ outputLooksLikeCrash/crashSnippet,把崩溃片段回灌逼修到真跑通=执行恢复力。任一不过即使 LLM 放行也判未达标)、 composeDeliveryMessage(返工后通过→另生成干净交付话术)、 isTestableCodePath/looksLikeTestFile/looksLikeTestCommand、

组件	文件	职责 / 关键入口
		extractArtifactContent、visuallyReviewArtifact、runCapturing。真控后置校验(2026-06-20,\$6 verifyOutcome):纯逻辑 LingShuOutcomeVerification.swift—— taskHadActionToolSuccess(扫记录有无真实动作工具成功,据 isActionTool 反推非产出/读取/元工具)+ isDocumentImpersonationSignal(本回合唯一交付是文档、且无真实动作 → 高危),作确定性事实注入 verifier:若用户要的是真实效果(接入/控制/操作设备)而只产出一篇文档 → 判未达标、不收(根治"写指南文档冒充接入");"是否动作型"由 verifier 据意图判,壳零关键词。单测 OutcomeVerificationTests
高质量 PPT(DesignKB)	Resources/DesignKB/(generator.py 多版式生成器 + palettes/typography/layouts.json + rubric.md + icons/lucide 预栅格 PNG)· LingShuDesignKB.swift(定位/读注册表)· LingShuState+DesignResources.swift(find_images Openverse 取图)	自进化 PPT 模块 Phase A:策展 PPT skill 由 DesignKB 设计系统驱动——多版式(封面/章节/要点带图标/大数字/图文/对比/时间线/图表/引言/收尾)+ 配色库 + Lucide 图标 + 原生图表 + 真实配图。apply_skill 把 DesignKB 生成器路径与跑法给模型(就地跑 python3 <DesignKB>/generator.py slides.json out.pptx,自动读同目录素材)。图标预备 Scripts/prep-icons.sh(Lucide ISC,rsvg-convert 栅格白/深 PNG)。根治"背景+文字"。 Phase B(过程内审计):LingShuState+DesignReview.swift 的 review_design 工具——生成后就地自审(soffice 隔离 profile 渲染→VL 按 rubric 打 0-1 分+逐页问题),<0.7 改 slides.json 重生成再审 (designScore/designIssues 落记录)。审计失败带真因 (DeckAuditOutcome:vision Unavailable/rendererMissing/re

组件	文件	职责 / 关键入口
		nderFailed/inconclusive)+ flagDesignAuditUnavailable 醒目告警,不再静默"不可用"。parseDesignScore 返回 Optional(VL 没给数字 =inconclusive,不假失败逼空转)。Phase C(自进化):dreaming 设计轨从历史 designScore+👍👍+失败点 designIssues 提炼"设计经验"写 DesignKB/design-insights.md(可写 overlay, 纯文字、 sanitizePromptOnly 净化),apply_skill 注入提示、 热加载——闭环
自固化(dreaming)	LingShuDreamingConsolidator.swift + LingShuState+Dreaming.swift	自进化 Phase 2:空闲回放已完成任务→candidates(领域成功 ≥3 且通过率 ≥0.6 且未被现有 skill 覆盖)→consolidate 蒸馏纯提示 skill → 落盘用户 Skills 目录 →LingShuCompositeExpertRegistry.reloadUserSkills() 热加载即时生效(免重启)。红 线:sanitizePromptOnly 剥一切代码块/脚本小节,绝不写可执行脚本; scheduleDreamingConsolidationIfIdle(空闲守卫+1h 节流,挂在 finishTaskRecord)
验收判定	LingShuChecklistVerdict.swift	解析"PASS=/FAIL=/结论:通过",allPassed = declaredPass && failedCount==0(verifier 用)
被调用 MCP 服务	LingShuControlServer.swift / LingShuControlRouter.swift (路由分发+工具执行) / LingShuControlRouter+Payloads.swift (只读载荷序列化 status/ledger/task/chat/trace,2026-06-21 拆出守 ≤800)	本机回环 HTTP+JSON-RPC,测试驱动入口(见 §4);lingshu_status.loopInvariantViolations 暴露架网遥测
会议音频(听)	LingShuSystemAudioCapture.swift + LingShuMeetingASR.swift	ScreenCaptureKit 系统音频采集 → SFSpeechRecognizer 转写
消息渲染	LingShuMessageContentView.s	回复分块:代码卡片 +

组件	文件	职责 / 关键入口
	wift	Markdown(标题/列表/表格/行内)
线程(任务清单)	LingShuTaskPoolView.swift	主界面 AppSurface.taskPool (标签**「线程」**;图标 bubble.left.and.bubble.right ,对齐 codex/claude 左侧线程列表概念:主线程=全能中枢,每任务一条线程、子线程=专项工作室上下文更聚焦);进行中/待办 vs 已完成,点开执行记录;热=最近一月(journal 时间窗冷备)、可纳入冷备
任务窗口(codex 式)	容器 LingShuTaskExecutionRecordViews.swift · 卡片 LingShuTaskWindowCards.swift · 开发任务侧栏 LingShuTaskDevToolsPanel.swift · 底栏 LingShuTaskWindowFooter.swift · 交互 LingShuState+TaskWindow.swift	按职责分文件(模块化)。 TaskExecutionRecordSheet(state:) 实时读记录;消息按 LingShuTaskExecutionMessage.detail 渲染分型卡片: TaskToolCallCard/TaskToolResultCard (折叠输出)/ TaskFileDiffCard (+N/-N+审核展开+撤销);答复走 markdown 。 TaskWindowFooter =模型选择+👍👎反馈+窗口内追问(submitTaskFollowup 同记录续跑); undoFileEdit 从 diff 无损还原/删新增
任务窗口布局二分(开发↔交付)	判定 LingShuTaskExecutionRecord.isDevelopmentTask (LingShuTaskExecutionJournal.swift)· 提交动作 commitTaskCodeChanges (LingShuState+TaskExecution.swift)	右侧面板按任务类型分流:开发任务 → TaskDevToolsPanel (「Git 工具」侧栏:改动 +/- 统计聚合自 fileEdit 、分支/仓库、提交改动按钮[只暂存本任务文件+确认弹窗+可 reset]、目标[=一句话总目标 record.goal ,模型 update_plan 时蒸馏的高度抽象概括(如「构建一个清分结算系统」,不复述需求原文);空则回退 title ;状态徽标 + N/N 步·耗时]、分步计划[record.plan 抽象里程碑 + 完成打钩,不绑定具体实现路径——具体怎么做 AI 推进中自己摸索]、产出物[复用 TaskArtifactFileCard ,预览/打开/访达];头部补 repo/branch chip ;左栏对所有任务只留对话叙述(执行计划/任务摘要/产出物都移到右侧)。三级信息架构(用户定调 2026-06-15):①

组件	文件	职责 / 关键入口
		总目标(一句话,右)② 分步计划(抽象里程碑+打钩,右,属规划)③ 具体实现(左栏多 agent 执行对话)。结果/结论不在右侧(在左栏最终答复)。所有任务统一用 TaskDevToolsPanel:顶部目标 + 分步计划;底部文件区——开发任务分「产出物 / 代码管理」两个 tab(避免全堆一列太长、看着乱),非开发任务只「产出物」、无 tab。「代码管理」= Git 工具(改动/分支/改动文件/提交),仅开发任务有。 isDevelopmentTask 严格 = 已捕获的 git 改动里含真·源码文件 (.swift/.py/...);.md/文档/.pptx 即便在 git 仓被改也不算——"读代码写架构总结/PPT"这类非代码流程绝不出现 Git 工具(用户反馈 2026-06-15)。 TaskArtifactFilesPanel 已被统一面板取代(留存未用)
任务窗口纯工具	LingShuLineDiff.swift · LingShuTaskMessageFormatting.swift	模块化拆出的纯算法/工具(不依赖 UI/State,可单测):行 diff(LCS)+ 改前内容还原;录制净化(抹模型名/收敛裸 JSON)+ 工具卡摘要 + 参数美化
附件接入	LingShuState+Attachments.swift / LingShuAttachmentIngestor.swift	 选择 / Cmd+V 粘图 / 拖拽(整框=路径自动转附件)→ 同一 ingest 管线; droppedFilePaths(in:) 纯函数判定(主输入框 convertDroppedFilePathsIfNeeded + 任务窗口底栏 convertDroppedFilePaths(in: draft) 共用,任务窗口拖入也转附件不再落路径文本);attachmentContextBlock 注入模型输入。 LingShuAttachmentTray 多于 3 个时显示 3 个 + 左右箭头滑动 + 计数(主输入框与任务窗口共用)
TTS 朗读	VoiceIOManager+SpeechOutput.swift + LingShuStreamingPCMPlayer.swift	speak → 剥 markdown(保留换行)→ 分段(仅按换行,不再按标点)→ 单段真流式 / 多段连续播放器(首段真流式+其余预取背靠背,无缝); LingShuSpeechSegment

组件	文件	职责 / 关键入口
		ter、openStreamingSegment、AVAudioEngine+PlayerNode
中枢状态	LingShuState.swift · 分诊 LingShuState+Triage.swift	@MainActor 总状态;submitTextInput 出口=续答续接/在岗/分诊。上下文感知分诊(2026-06-17):classifyDispatch 不再只看裸 prompt——buildTriageContext 组装①近上下文(最近 8 条逐字,派发线程消息打 [Tk] 标签)②远上下文(persistedConversationDigest 压缩)③可续任务线程清单(近 40 条聊天里的隔离子任务,各取最近一句+标题,blockedDispatchedRecordID 标"⌚ 正等回答"),分诊器据此分 chat/task/reply(并指明 reply 哪条线程)→ 能回溯到前面隔了几条才问的提问、避免把无关闲聊误当答复。reply → continueDispatchedThread → submitTaskFollowup → orchestrator.resumeWithInput 续那条隔离会话本身(带真上下文),根治"做 PPT 问主题→你答主题→它却跑去聊天=答非所问"。记录按需建:reply 续用线程自身记录、task/chat 才建,被续答/在岗/自主早返回接管的轮次不留空壳记录。串行闸门(2026-06-25「砍掉双线并行」):分诊前一道闸——currentlyExecutingTurn() 判忙(问答线在飞 OR 任务子线程在执行;waitingForUser 已被 prune 剔除=不算忙故答复不死锁)即把新顶层输入入 pendingSerialInputs 串行队列(显示"已排队"气泡、队列区可删),当前回合完全返回后(executeMainTurn defer / promoteQueuedDispatchIfPossible)drainSerialInputsIfIdle 逐条出队重提交(复用气泡)。推翻"1 任务+1

组件	文件	职责 / 关键入口
		问答双线并行"(原 pendingChatTurnIDs/queuedDispatchTasks 跨线并行)→ 同一时刻只一个上下文在跑,避免上下文污染、模型更易认对当前上下文(对齐 codex/claude 单串行)。见 LingShuState+SerialInputQueue.swift 。有行数守卫(≤3500)

3. 关键不变量 / 易爆点(避免异常爆发)

- **Swift 6** 严格并发:跨 actor 的闭包用 `@MainActor @Sendable`;纯函数静态用 `nonisolated static;NSLock` 不能用在 `async` 上下文。
- 签名与 **TCC:.app** 用 **Apple Development** 身份签名(`build-app.sh`,稳定身份才能持久屏幕录制等授权);ad-hoc 会让授权失效/不弹框。虚拟麦克风驱动(未来)需 **Developer ID Application**(已有证书)。
- 跑 **.app**,别跑裸二进制(否则麦克风崩溃+无图标)。重建后必须先退旧实例再 **open**(`open` 不会重载新码)。
- 中文进程名让 `unified log` 检索不可靠 → 诊断写 `/tmp/lingshu-control.log(lingShuControlLog)`。
- 架构守卫测试
`testStateFilesKeepOperationalSubdomainsSplitOut:LingShuState.swift` 不得超 3500 行,超了把子域拆进扩展。
- 顶栏「脑力」评分(**2026-06-20**,替代 **TRUST**):`LingShuBrainScore`(纯逻辑,可单测)衡量当前脑自主性——自主完成一个任务 **+1**(`finishTaskRecord(.completed)` 打点)/每触发一次兜底 **-1**(撞顶补预算 / 验收升级到 `Rung2` 确定性兜底 / 停滞交还,均在 `LingShuState+AgentAcceptance` 打点)/切大模型归零(`brainID=provider | model,applyModelProvider`/导入配置 `reset,rebased` 兜底)。持久化 `UserDefaults lingshu.brainScore`,接线 `LingShuState+BrainScore`,HUD 两处由 **TRUST** 换成「脑力」。单测 `BrainScoreTests` 8、活 E2E(完成+1/换脑归零)过。内置脑力测试 (**2026-06-20**):`LingShuBrainBenchmark`(硬编码 37 题全谱:7 易/15 中/15 难,后者含 5 道 **agentic** 工具题;故意塞已知 LLM 失误题=9.11vs9.9/strawberry-r/认知反射陷阱/三段论有效性/超大数计算,用来照真实差距不是给满分)+ 确定性判分(`reasoning` 看回复;**agentic** 要求"答案对且真调过工具",口算/摆烂不算)+ 难度加权综合分。运行器 `LingShuState+BrainBenchmark`(逐题真发当前脑;**agentic** 题给真工具 +`autoAllowShell`+多轮,据 `session.toolInvocations` 判是否真驱动)· 弹窗 `LingShuBrainBenchmarkView`(挂 `LingShuRootView`,任何界面跑完都弹;入口按钮在模型通道页)· MCP `lingshu_run_brain_benchmark`。答案 **key** 先用 **python** 验过再硬编码(防错杀正确回答)。扩到 42 题:加 5 道极难长链编码题(括号匹配/罗马数字/表达式求值/调

试/多文件工程),由 **CodeCheck** 跑隐藏用例真验代码判分(BENCHDIR 隔离 + 临时切 `codexWorkingDirectory` + `python harness` 输出 `BENCH_PASS` 才算过;答案 `key/harness` 先 `python` 验过),写得出但写错/摆烂=0 分。再扩到 **46 题**(加 4 道前沿硬核:LeetCode-Hard 正则匹配/手写 JSON 解析/N 皇后/大字符串加法,隐藏用例真验,权重 8)。顶栏「脑力」改为可点 **chip**(`LingShuBrainScoreChip` → `popover` 看具体评分拆解 + 上次测评 + 「一键检测脑力分」按钮);实时通话/极简语音入口已删(`LingShuPrimarySurfaces` 的 `waveform` 按钮移除,模式 `infra` 保留但 UI 不可达)。单测 `BrainBenchmarkTests` 9、活 **E2E:DeepSeek 98 分(45/46)**——9 道隐藏用例编码题(含 4 前沿)全过,只错字符串反转(字符级盲区)。重磅结论:在可客观判分的有界任务上 **DeepSeek-chat** 与前沿差距≈0,想"出题让 **DeepSeek** 输给 **Codex/Claude**"未果——印证"同强脑下 **agent** 差距不在有界任务求解,而在 **harness**(鲁棒/上下文/工具精度/长 **horizon**)",见 [deepseek-agentic-ceiling](#)。**TRUST** 旧 **bug** 同日修: **trustScore** 把 **ASR** 写死查 `asr:datanet`,本地 **ASR**(默认)校验在别的键 → 耳朵永不计入 → 通道恒 3/4 → 钉死 **91%**;抽 `LingShuTrustScore`(本地 **ASR/TTS** 始终可用即就绪)修正,`TrustScoreTests` 6 + 活测 **100%**;`lingshu_status` 加 `trustScore/brainScore` 字段。再扩到 **49 题** + 生产/长任务层(2026-06-21,治"有界谜题刷高分"):加 3 道生产任务(库存系统/扩存量代码/多需求数据流),**CodeCheck** 支持 **BENCH_SCORE p t** 部分给分(做一半给一半,`compositeWeighted`),权重 **18/14/14**——真编码+生产任务占总权重~57%(`testBatteryShapeWideSpread` 守"过半"),知识问答刷不高。活 **E2E:DeepSeek 94 分(47/49)**,3 道生产任务全满分**(9/9·7/7·6/6,实证 `partial-credit harness` 正确),错字符串反转 + 手写 JSON(后者有模型波动)。同日另三件:① 会话内语义压缩 `LingShuAgentSession.compactHistoryIfNeeded`(超 `maxHistoryMessages` 把早段蒸馏成滚动「前情提要」再续,替代硬丢中段=对标 `CC auto-compaction`;蒸馏失败回退硬裁剪)。② 一等公民快搜索 `search_text`([LingShuState+SearchTool.swift](#),`ripgrep` → `grep` 兜底,挂主会话+派发子任务)。③ 脑力分当旋钮 `harnessKnobPrefix`([LingShuState+BrainScore.swift](#)):脑在生产主导的测评≥85 分(或运行分≥12)→ 系统提示前置「放权」(`update_plan` 不强制/验收从简),安全红线不放松——让"强脑→薄 **harness**"真生效,且因评分 `measure` 的是生产能力,弱脑刷不到 85 故不会误放权。④ 按难度档水位拆解 `tierBreakdown/TierScore`:综合分本就按难度加权(难题分值高),再把得分按易/中/难/极难分档展示(各档加权% + 全过数)——单一综合分掩盖水位,分档才看清不同脑卡在 哪一层(弹窗四档条 + `MCP tiers` 字段)。跨脑对比:每颗测过的脑存紧凑快照 `LingShuBrainBenchmarkSnapshot`(持久化 `UserDefaults`),弹窗 ≥2 颗脑时并排比各档水位(`upsertBenchmarkSnapshot`,换脑重测即累积);差距具体到"差在哪一难度层"。

- 持久化主会话 + 换脑重建(2026-06-16):`mainAgentSessionHolder` 跨回合持久; **applyModelProvider** 换模型时清空主/自主会话让下回合用新模型重建并重新 `seededDistilledMemory`(换脑即时生效 + 记忆延续)。模型选择 (`modelProvider/modelName/endpoint`)持久化到 `UserDefaults`(`LingShuPreferenceKeys.model*`),跨重启保留所选大脑;`key` 仍走 `credentialStore`(AES-GCM,按 `preset.id` 存)。MCP `lingshu_set_model(provider,model?)` 可 `headless` 切脑(不返显 `key`,只报 `keyConfigured`)。实测:切 **DeepSeek** + 存 `key` → 重启后仍 **DeepSeek** 秒回、身份正确;产出物记忆"运行起来"接上。
- 模型通道=带校验的类型化能力通道(2026-06-16):模型通道页按用途分区——中枢/文本(文本 LLM)、语音合成口/TTS(`LingShuSpeechOutputProviderDescriptor`:数据网关情绪男声/本机系统语音)、感知眼耳(视觉/视频走数据网关 VL、语音识别走本机)。每通道一个**

「校验」实测调用([LingShuState+ModelChannels.swift](#)):中枢=发 ping 拿回复、视觉=发小测试图给 VL 拿语义、TTS=本机始终通/云端看 token、ASR=本机始终通;结果存 `channelValidations[channelKey](brain:<provider>/vision:datanet/tts:<id>...,UserDefaults` 持久,

`LingShuChannelValidation{ok,detail,at})`。过滤铁律:子线程切换 `taskWindowModelProviders=switchableTextProviders()`(只列已配置 key 且校验通过**的文本通道 + 当前在用的),不再平铺全量目录;未配置/未校验的不出现在可切换列表。TTS/感知通道也在模型通道里配 + 校验,模态内部只选校验通过的。UI=暗色科技感 + 子 tab 区分通道类型(中枢·脑/语音·口/感知·眼耳):真视图 [LingShuModelGatewaySurface](#)(设置 hub 的「模型通道」tab 渲染它,不是已删的遗留死视图 `ModelConfigurationViews.swift`),暗色 `LingShuChannelRow`(校验状态徽标 + 校验/使用/修改 chip)+ 暗色弹窗 `LingShuModelChannelSheet`(新增先选供应商再配,密钥不返显)。坑:之前重构误改在未被引用的死视图上(白底浅色主题)→ 渲染不出/风格全白;真视图在 `LingShuSettingsSurfaces.swift`,改 UI 先确认 hub 路由到哪个 view。

- 感知通道:眼/耳拆两通道 + 本地模式开关 + 单通道 **isActive(2026-06-16)**:模型通道页 `ChannelTab` 已是 4 个独立 tab(中枢·脑/语音·口/视觉·眼/听觉·耳),眼耳不再合并。本机有兜底的能力(耳 ASR / 口 TTS)各给一个「本地模式」开关 (`asrLocalModeEnabled/ttsLocalModeEnabled`,`UserDefaults` 持久,`didSet`→`applyASRLocalMode/applyTTSLocalMode` 真切 `voiceManager.transcriptionProvider/speechOutputProvider`)——显式确认走不走本机,取代"本机兜底悄悄移出列表"的隐式降级。单通道模态 **isActive** 修复:视觉/听觉只有一个通道,原来写死 `isActive:false` → 永远显示"未选中/未启用"(误导);现 `isActive=isActiveChannelValidated(key)`(听觉再 && ! `asrLocalModeEnabled`)。配置弹窗不再空白:视觉/听觉编辑弹窗传真实默认 (`perceptionGatewayEndpoint/visionDefaultModel/asrDefaultModel`),而非空的 `channelConfig`——感知走数据网关专项接口,端点/模型本就有确定默认值。
- TRUST/置信度=真实计算,不写死(2026-06-16)**:旧 `trustScore=91` 是写死假值(从不计算),已删。现 `trustScore(computed,[LingShuState+RuntimeStatus.swift])=` 三个真实信号按权重合成:模型连通(0.40)+ 能力通道校验通过比例(0.35,通道=当前脑/视觉/听觉/当前口)+ 近期任务验收通过率(0.25,近 20 条终态:completed/answered=过,blocked/needsRevision=不过);无数数据的维度自动从权重剔除(不凭空拉高/压低)。`trustBreakdown` 给顶栏/核心区 .help tooltip 解释"这百分比怎么来的"。符合 [no-fake-demos](#)。
- 前缀缓存按厂商适配(2026-06-16):省 token 的大头。集中在纯模块 [LingShuPrefixCache.swift](#)——`strategy(for: format)` 按 `requestFormat`(已由 provider/protocol/endpoint 推导)选策略,换模型自动选对、各处无需改代码:OpenAI/DeepSeek/通义/Kimi/MiniMax(chatCompletions)+ OpenAI Responses=.automatic(服务端自动命中,请求体零改动);**Anthropic=.anthropicExplicit**(必须显式打 `cache_control`,否则完全不缓存)。Anthropic 请求体改由 `LingShuModelGateway.anthropicMessagesBody(JSONSerialization)` 构建:给 **system** + 最后一条消息打 `cache_control:{type:ephemeral}` 断点(system=最大最稳前缀;最后一条=缓存到上一轮为止的整段)。可观测:
`LingShuPrefixCache.parseCacheUsage` 跨厂商解析命中量(OpenAI `prompt_tokens_details.cached_tokens` / DeepSeek

- prompt_cache_hit_tokens / Anthropic cache_read_input_tokens) → LingShuRemoteModelReply.cachedTokens, 适配器写 /tmp/lingshu-control.log("prefix-cache hit=...")、recordModelUsage 进用量轨迹。中枢·脑通行副标题显示策略短标(prefixCacheStrategy(for:)) → 缓存·自动/显式/—)。灵枢架构本就 **cache-friendly**(持久 append-only 主会话, system+seed 稳定前缀), 自动缓存供应商免费命中。遗留缺口: Anthropic 工具回合(tool_use/tool_result 原生形态)未发——直连 Claude 跑 agent 工具循环是另一处待补, 与缓存无关。测试 LingShuPrefixCacheTests(8 例)。
- 最终答复真流式 + **TTS** 早接入(2026-06-16): agent 循环的最终答复轮逐字进气泡(取代原"整段拿完再渲染")。机制: LingShuAgentModel 加 respondStreaming(onTextDelta:) (协议默认回退非流式, 脚本模型/不支持流式的供应商不必各自实现); LingShuAgentSession 加 textDeltaSink——只有主会话 setTextDeltaSink 才走流式(子会话/自主/测试无 sink → 仍 respond 非流式, 行为零变更, 风险隔离在主会话); runMainAgentTurn 把 delta 接进 appendStreamingBubbleText(逐字 + 按句早读 streamingSentenceSpeaker——[LingShuState+Streaming.swift] 那套现成流式 UI 终于被接上)。LingShuGatewayAgentModel.respondStreaming 仅 chat/completions 真流式(client.supportsAgentStreaming=requestFormat==.chatCompletions, DeepSeek/兼容), Responses/Anthropic 回退非流式(事件/工具流形态不同, 避免误解析)。client.streamAgent 解析 SSE delta.content(回调)+ 按 index 累积 delta.tool_calls 分片 + 注入 stream_options.include_usage 拿 usage(保留前缀缓存命中可观测)。单次尝试不内部重试(重试会让已逐字气泡重复) → 失败当 .failed → 循环 .interrupted → 走断网挂起/重连续跑(气泡被"网络中断"覆盖, 不重复)。finalizeMainTurn 用 .completed 权威文本覆盖气泡 + concludeStreamedSpeech 补念尾句并设 lastSpokenMessageID 防根视图整段再念(双声线易回归点)。delta 闭包 async 串行 hop MainActor → 保逐字顺序。工具轮 content 基本为空、不进气泡。测试 StreamingAgentLoopTests。坑(markdown 流式易回归, 2026-06-16): streamAgent 取 content 必须直接读 choices[0].delta.content 并保留换行/空白(只过滤空串 ""), 绝不能用 gateway.decodeStreamingTextDelta——它末尾 .nilIfBlank 会把"纯换行 \n 的增量块"当空丢弃(DeepSeek 流式换行常是单独一个 chunk) → 所有换行丢失 → markdown 列表/段落塌成一段(粗体仍在, 因是行内解析)。配套: 渲染器 LingShuMessageContentView.listItem 放宽——行首 - 后紧跟非空格(模型紧凑列表 - **读**)也渲染 bullet(只放宽 -, 不放宽 * 防 *斜体* 误判)。流式渲染本身是每来一个增量全量重解析 markdown(非滑动窗口, 对几百字回复够用)。坑: 加载气泡若带 taskRecordID, [ChatBubbleView] 原本只画紧凑进度行、忽略正在增长的 text → 逐字看不见(只有 finalize 一次性出); 已改为"有流式正文就逐字、还没正文才显示进度行", 且 pending 气泡正文初始留空(去掉占位话, 否则 text 一开始就非空判断失准)。
 - 流式 **TTS** 增量无缝(2026-06-16): 配合真流式, 按句早读升级为增量无缝发声——VoiceIOManager+StreamingSpeechQueue.swift: speakStreamingSentence(每出现一句喂一句) → 并行预取各句 TTS WAV(fetchSpeechAudio, Task.detached) → drainer 按 nextToPlay 严格句序剥 WAV 头取 PCM 背靠背灌进同一个持续 LingShuStreamingPCMLayer(段间不排空到静音=无缝); finishStreamingSpeech 标记结束 → 播完 finishAndDrain; cancelStreamingSpeech(stopSpeaking 调, 打断/新回合)。根治旧 speakQueued 每句独立 speak()、串行等一次完整网络往返 → 句间卡顿。仅云端流

- 式 provider(dataNet 等有凭据)启用,本机/无凭据退回逐句 `speakQueued`。代次 `speechGeneration` 守卫 + 单句失败只跳过不抛错(防本机降级叠加=双声线); `isSpeakingOrQueued` 含 `streamingSpeechDrainTask`(连续通话别把句间误判为说完)。注入:`streamingSentenceSpeaker`(根视图)=`speakStreamingSentence;concludeStreamedSpeech` 念尾句后 `finishStreamingSpeech` 收口。稳健性(2026-06-16 修噪音/卡顿/重复):① 每段独立偶数对齐再 `enqueue`——player 的 `leftover`(半样本暂存)是为单段连续字节流设计的,多段共用同一 player 时某段奇数字节会串到下一段 → 错位半样本 → 之后全噪音(根因);② 采样率一致性校验——首段定基准,后续段不符跳过(否则按首段固定 format 播=音高/噪音);③ 有界预取窗口(`streamingSpeechPrefetchWindow=3,feed` 只登记文本到 `streamingSpeechTexts`、drainer 按窗口启 Task)——不一上来并发几十个/stream 打爆服务端;④ 单句软超时 15s + 重试 1 次(`fetchSpeechAudioResilient,withThrowingTaskGroup` 竞速)——救服务端偶发抖动(实测同一回复有时零跳、有时个别句卡几十秒);两次都超 15s 才跳过那句。双声线根因(已修):根视图整段朗读 `speakLatestReplyIfNeeded` 原用 `last(where:!isLoading)`,本轮气泡 loading 时会落到上一轮回复重念并覆盖去重标记 → 本轮 `finalize` 去重失效 → 又整段 `speak()` 超时降级;改为只看 `messages.last`(loading 就不念)。物理约束:按序无缝播 → 某句慢只能"等或跳",不能既不等又不漏;要绝对不漏需超时句本机补读(音色跳变,未做)。
- 流式增量解析=分层独立模块(2026-06-16,用户要求按模型适配):不同厂商 SSE 结构不同,别再在 `streamAgent` 内联解析(那是换行丢失/think 污染的根本)。① 协议层 [LingShuStreamChunkParser.swift](#):`LingShuStreamChunkParsing` 按 `requestFormat` 选 OpenAIChat/Responses/Anthropic 解析器,SSE 行→结构化 `LingShuStreamChunk{contentDelta(**保留换行**)/reasoningDelta(reasoning_content)/toolCallDeltas/usage/done}`;② 模型层 [LingShuModelReplyAdapter.swift](#)(已有,按 provider/model)做内联 <think> 流式状态机分离。`client.streamAgent` 串两层:协议层取 content → 模型层剥 think → 只正文进气泡(M3 内联 think 流式实时剥、不再逐字污染气泡;DeepSeek-R 的 reasoning_content 也不进正文)。换模型自动选对解析器。测试 `StreamChunkParserTests`(9 例,含"纯换行块不丢"回归)。
 - 目标是唯一成功停止位(goal-driven,不是固定预算):agent 循环的停止条件只有四个——① 目标达成(模型给出最终答复;有产出物则独立 verifier 通过)② 卡住等人(`ask_user → .blocked`)③ 原地打转(连续 `stuckRepeatThreshold=5` 次完全相同工具调用 / verifier 一轮无新产出且意见不变) → 诚实交还 ④ 用户停止。`maxTurns`(主 80/派发 120/spawn 子 80/自主 120——复杂工程单段「读改构建测试修」真能超 40 步,2026-06-16 由 40/25/80 抬高;纯防失控,非预算)与 verifier `verifyCeiling(8)`、撞顶恢复 `recoverCeiling(2)`只是防失控的高位安全天花板,不是目标预算——正常远到不了,绝不能调低当"到点收工"用(那就退回伪 AI)。失败/超时是回灌进上下文的观测,模型据此换方法继续,不是终止信号。撞顶≠失败:per-run 用满天花板但有在制品 → `verifyAndContinue` 把它当检查点补预算续跑(`recoverFromExhaustionIfNeeded`),修到跑通再交付;主/自主/隔离子任务同一套(子任务经编排器 `acceptanceHook` 委托)——根治「复杂工程撞顶/崩溃被直接判异常、不自愈」。
 - 断网重连自动续跑(基础设施故障≠任务失败,2026-06-16):模型网关重试耗尽 → `LingShuAgentModelResponse.failed` → 循环返回 `.interrupted`(绝不追加

- 假"调用失败"助手消息、**messages** 原样停在良构状态,因为故障在模型调用那一跳、工具早 **await** 完落盘=幂等安全)→ 编排器标 **.suspended**(黄色"已暂停"非红色"异常")、保留会话、登记 **suspendedForReconnect**。
- LingShuConnectivityMonitor(NWPathMonitor)**不可达→可达(去抖 2s)→ **resumeSuspendedWork** 逐条
- resumeInterrupted**→**session.continueLoop()**(不注入新消息,重发中断那步)→续跑+验收。主会话(**suspendedMainTurn**)/自主 (**suspendedAutonomousRecordID**)同理。**NWPath**「有链路」≠「网关活」→真探针是续跑那次模型调用,又连不上就再 **.interrupted** 回暂停等下次。手动「继续」修对线程:**submitTaskFollowup** 反查 **agentSubTaskRecords**,隔离子任务记录→**orchestrator.resumeWithInput**(续跑有真上下文的隔离会话本身),不再误投没有该任务上下文的主会话(根治"点继续没反应")。限制:全内存,跨 app 重启的会话丢失=未做(Phase 5,需从持久记录重建上下文+去重)。
- 配套:模型调用瞬时超时/抖动由 **LingShuGatewayAgentModel.respond** 重试 3 次+退避(传输层自愈),都失败才如实回报;**run_command** 超时由工具结果反馈续跑。已删旧全局看门狗 **handleModelTimeout**(单次超时杀全轮的伪 AI 遗留)。
 - 加载气泡状态=**missionTitle**,且显示「当前在做什么」而非笼统"思考中":**LingShuTaskProgressIndicator(stage: state.missionTitle)**。回合期间:起手"理解需求";工具执行中"执行中:<工具显示名>"(**toolDisplayName** 已补全 **update_plan**→制定规划/**review_design**→审版式/**find_images**→找配图/**spawn_task**→派子任务/**preview**→翻页等);工具间隙(模型思考,可长达几分钟)不再置"思考中",而是 **currentActivityLabel**=当前进行中的计划步(模型写的步骤名正是"生成 PPT/写测试"这种活动级标签);**applyTaskPlan** 一更新计划就刷气泡。忘了刷它气泡会停在旧值看着像卡死(2026-06-14);"思考中"笼统不达意已改为活动级显示(2026-06-15)。
 - 停止要真停:**cancelCurrentCall** 必须 **activeAgentTurnTask?.cancel()** + **autonomousRunTask?.cancel()** 并收口所有 **isLoading** 气泡——agent 路不设 **activeThinkingMessageID**,只改状态不取消 Task 等于没停。孤儿加载气泡自愈(2026-06-19,真机卡死根因):**runMainAgentTurn** 的加载气泡靠其 Task 体内 **finalize** 收口;若回合被取消/串行等上一轮而 Task 没跑到 **finalize**,且 **cancelCurrentCall** 在!**hasActiveModelCall** 时提前 **return** 跳过了气泡清扫→气泡永久卡"理解需求 437s"(实测 W3)。修:① **cancelCurrentCall** 的 guard **hasActiveModelCall** else { **reapStaleLoadingBubbles(force:true); return** }(提前返回也清孤儿);② **reapStaleLoadingBubbles(now:force:)** 周期挂在 **tickCoreTimers** 自愈——无在飞回合/自主运行/派发任务驱动、又转圈超 150s 的加载气泡自动收口(派发中的气泡 **dispatchedTaskBubbles** 保护不动)。CoreBoundaryTests 覆盖纯逻辑。
 - 破坏性覆盖自动备份(2026-06-19,选项 B):**runAgenticTool** 里 **write_file** 整体覆盖已存在文件(**op==.modified**)→先把改前内容经 **backupOverwrittenFile** 存一份带时间戳备份到 **~/Library/Application Support/LingShu/backups/** + **lingShuControlLog("guard/overwrite:...")** 醒目日志。不阻塞(保住无人值守自主),但可回滚(任务窗 **undo** 之外多一层文件级备份),防"清空所有源码重写"这类批量破坏覆盖造成不可逆损失。**edit_file** 局部替换不另备份(diff 卡足够)。实测:**write_file** 覆盖 v1→v2,备份含 v1。
 - 口头表述≠字面表述(用户定调 2026-06-19,自主模式只能听):两层——① 语气靠提示:在岗/自主

两段系统提示加"口头表述铁律"(主人看不到任何文字/文件、只能听;像人口头讲述,别用"这是第X页内容:要点A、要点B"念稿腔,演示讲稿=口语讲解不是逐字念幻灯片);②符号靠清洗:`strippedForSpeech(voice.speak` 入口必过)在剥代码块/表格/`*/#`基础上再剥装饰 emoji(✅⚠️🔪...,朗读成"白色对勾"很怪;按 `isEmoji && value>0x238C` 过滤、ASCII 数字/`*/#` 不误伤)+ 破折号转逗号停顿。提示管语气、清洗管符号,合起来=干净口语。VoicePhaseTests 加 2 个口语化清洗单测。

- 语音输出:回复朗读只由根视图 `speakLatestReplyIfNeeded`(监听 `chatMessages`)负责,别在别处再 `speak`(否则双声线)。`voice.speak()` 先 `strippedForSpeech` 剥 markdown(代码/表格/标记/emoji/破折号),再试云端 `dataNetSpeakerTTS`(流式 `.../swds-speaker-tts/stream`,首包即播),token 读自 `Resources/RuntimeConfig/datanet-gateway.token`(gitignore,随包 ditto 进 bundle);token 缺失/过期→静默降级 Mac 本地音——"还是本地音/没声"先查 token。
- 语音通话状态/打断:LingShuVoiceCallController 靠 `state.hasActiveModelCall(=isModelReplying||isModelExecuting)` 显示"灵枢在思考..."。新执行路径模型干活期间必须置 `isModelReplying/enterCoreState(.thinking)(runMainAgentTurn` 首尾置/复位,Task 存 `activeAgentTurnTask` 供打断)。思考中不停麦:继续听,噪音(达不到自适应 VAD 说话门槛)忽略;确认真指令(持续说话→收口)才 `interruptActiveModelCall()` 取消在飞回合并接管。响应中(TTS)停麦防自听,打断需持续电平 $\geq 0.7s$ (短噪音/回声不触发)。
- TTS 分段=换行为主 + 超长行兜底:`strippedForSpeech` 用 `\n` 连接各行(保留换行);`LingShuSpeechSegmenter.segments` 以换行为主切(短行/要点=一段,听感自然),但单行超过 `maxSegmentChars(36)` 就按句末标点(。! ? !?...; ;)二次切(`splitLongLine`,仍超长再按字数硬切)。原因:云端 /`stream` 对长段又慢又只合成首句——纯换行会让长 bullet 变成一个大段→超时/漏读→退本地(2026-06-14 修复)。单段真流式(首包即播);多段走一个连续 `LingShuStreamingPCMPlayer`(player 用 `AVAudioPlayerNode` 背靠背 `scheduleBuffer`,本身无缝)——首段真流式,其余段并行预取(`Task.detached`,窗口 8 深)整段 WAV 灌入同一播放器。预取不是单线程;窗口浅(曾=3)会被短段播放追上→段间饿出长间隔,故加深到 8(2026-06-14)。取舍:单行内多句长文(无换行)会撞上云端 /`stream`「长文本只合成第一句」的限制而漏读后半;模型回复以短行/要点为主极少遇到,真遇到再给超长行兜底(别退回标点拆)。
- 流式起播预缓冲(根治"前面一字一卡、后半段才顺"):LingShuStreamingPCMPlayer 原本 `start()` 即 `node.play()`、首块一到就播——零缓冲下网络/合成块只要晚于实时消耗就 `underrun` 出逐字静音,直到生产追过播放才顺(正是"后半段就好了")。现改为起播预缓冲:`start()` 只启引擎不 `play()`;enqueue 在未起播时累计帧数,攒够 `primeThresholdFrames(≈0.4s)`才 `node.play()`,之后保持播放、后续块无缝续上(`scheduleBuffer` 在节点未播放时也能正常入队)。`finishAndDrain` 对总时长 < 门槛的短句强制起播(`forceStartPlaybackIfNeeded`,同步锁不跨 `await`),否则会卡在预缓冲里不出声。取舍:首声延迟 + $\approx 0.4s$ (门槛可调)换全程不卡。仍见中途卡顿才需加"underrun 重新预缓冲"(暂未做,当前症状是起播段)。未经听感实测确认(2026-06-15,编译通过)。
- 发声「代次」守卫(根治云端 TTS 与降级本机语音重叠/双声线):三条音频路径互相独立(云端流式 `node` / 云端 `AVAudioPlayer` / 本机 `AVSpeechSynthesizer`),`speakWithAppleSpeech` 的 `fire-and-forget` 收口定时器会把过期那条的 `isSpeaking` 误清成 `false` → 队列/会议提前起下一句 → 叠音。现加

VoiceIOManager.speechGeneration:每次 **speak()** 递增并捕获 **gen**;降级(**catch**)前先 **guard speechGeneration==gen**——本任务已被更新发声取代就直接放弃,绝不在新发声上再叠本机降级音;**speakWithAppleSpeech** 入口 + 其收口定时器、**speakWithStreamingSegments** 的 **defer**、取消 **catch** 全部按 **gen** 守卫,过期音频一律不起播/不翻转状态。配套:云端请求超时由 12–40s 延长到 20–60s(**LingShuSpeechOutputGateway** 第 231 行,减少误降级)。未经听感实测确认(2026-06-15,编译通过)。

- **TTS 双声线(易回归)**:多段里后续段失败只 **continue** 跳过,绝不抛错——一旦抛错会冒到 **speak()** 兜底触发本机 Apple 朗读,而云端播放器还在放 → 两声线叠加。**speak()** 兜底分支调本机语音前必须先 **activeStreamingPlayer?.stop()** + **speechAudioPlayer?.stop()**。只有用户取消才停播放器并上抛。
- 首轮预热:启动时后台 **Task { await state.mainAgentSession() }** 预热(含记忆蒸馏),消除首条延迟。
- 验收门触发:可靠信号是本回合真有产出物落盘(**taskExecutionRecords[id].artifacts** 非空);**replyClaimsArtifact**(产出动词+文件线索)作补充。不要只抠回复动词(漏了"已交付"会让验收形同虚设);自我介绍不写文件 → 无产出物 → 不触发,不会误触。
- 编排器会话关掉"回复声称产文件"的验收触发(**trustReplyClaim**,2026-06-19 实测"讲解完处理中卡很久"根因):**replyClaimsArtifact** 只看回复文本"已保存…路径",会被"回复里提到既有文件"误触发。主会话/常驻在岗是编排器——重活都派发给隔离 **session** 各自验收、自己几乎不直接产交付物;其轻量/对话/演示回合(导航/答疑/讲解)的回复一提到既有文件(如正在演示的那个 PPT)就误进验收门 → 本回合无新交付物可验 → **verifier**(也是 **DeepSeek**、对真实存在的文件仍会判"需修正")空转几轮慢调用 → 撞「停滞交还」,表现为**"讲解完后处理中卡很久"。修:**verifyAndContinue/driveAgentDelivery** 加 **trustReplyClaim**(默认 **true**),主会话(**runMainAgentTurn**)与常驻(**resumeStandingSession**)传 **false**——这俩编排器只在本回合真落了新登记产出物**(**producedRealArtifacts**)时才验收;派发隔离子任务 / 自主运行 **kickoff** / 自主续跑 / 会议纪要等执行路径保留 **true**(**run_command** 产出却没自动登记的真文件靠它兜底触发验收)。E2E 实测:同一个会命中 **replyClaimsArtifact** 的在岗回合,修前 [验收] 停滞交还、修后 0 验收条目 + 4s 收尾。
- 演示类回合跳过产出物验收(**turnDidPresent**,2026-06-19 实测"做 PPT+演示卡在结果验证"):做 PPT+演示 这种回合会真落新 **PPT**(**producedRealArtifacts=true**) → 即使 **trustReplyClaim=false** 也触发验收门 → **verifier** 对 PPT 挑刺"需修正" → 返工循环把正在演示的回合卡在「结果验证」、还误导主人(演示 ≠ 交一个待 QA 的文件)。修:**runVerificationLoop** 在触发门前先判 **turnDidPresent=**在给主人看/演示(互动) → 直接跳过验收。**turnDidPresent** 判据:预览正开着(**previewController.isPresented**)或本回合调过 **open_preview/present_fullscreen**。必须含 **open_preview**(2026-06-19 二次实测根因"PPT 打开后卡结果验证")——打开 PPT 走的是 **open_preview**,只认 **present_fullscreen** 会漏掉"已 **open_preview** 开了 PPT、还没进全屏就触发验收"。要单独 QA 这个 PPT 主人会另说"检查一下这个 PPT"(那才是纯文件交付工作型)。E2E:做长城 PPT+全屏演示,近期 [验收] 条目数=0。**LingShuLoopPhase.verify**ing 展示名 2026-06-19 由"验收中"改"结果验证"(**rawValue**,仅展示;**englishName** 不变)。正解是分阶段(用户细化 2026-06-19,标准流程,见 [work-vs-interaction-closure](#)):复合请求(做 PPT+演示+答疑)规划

时拆成顺序阶段——做 PPT=可验收工作(过验收)、演示/答疑=互动(不验收),逐阶段推进。

turnDidPresent 跳过验收是对演示阶段的正确处理;"做 PPT 单独成阶段(经验收)"靠规划提示(两段系统提示已加"复合请求按阶段规划"铁律:做和演示别挤一个回合)。即"演示整个不验收"是一刀切错法,正确是"做那段验收、互动那几段不验收"。验收全程可被唤醒词/打断中止(2026-06-19 修"卡在结果验证喊唤醒词打不断"):runVerificationLoop 的 while 循环每轮先查 **Task.isCancelled || batchInterruptRequested** → 收到打断(唤醒词 barge 置 batchInterruptRequested / 命令打断 cancel 回合)立刻停验收、**setLoopPhase(.idle)**、交还当前结果(recoverFromExhaustionIfNeeded 补预算循环同样加查)。原 bug:verifyAgentDeliverable 是模型调用、循环不查打断→打断/关窗后照样一轮轮转到停滞=卡死打不断。实测:做 PPT 抓到「结果验证」发"停,现在几点"→[验收]打断中止验收→立刻答时间。用户定调流程:规划→准备(理解材料/备稿,验收属此)→执行(演示/答疑互动),全程可唤醒词打断、继续则从断点阶段续。

- 工作 vs 互动——收尾方式不同(用户定调 2026-06-19,行为策略):世间事分两类。工作型(产文件/改代码/查资料等有确定交付物)=做完直接交付收尾。互动型(演示/讲解/答疑/陪聊/带人看东西,本质是和主人来回)=做完绝不直接收尾、也绝不自动关材料,先 **speak**「演示结束,还需要我做什么吗?」并保持材料开着,等主人确认没后续(或转做别的)才 **present_fullscreen(false)+close_preview**。判不准当互动。落点:在岗(空 objective)+ 自主(带 objective)两段系统提示 [LingShuState+AutonomousPrompts.swift](#) 均已写入(演示 LOOP 末页不再自动退/关、改为确认)。注:这是模型在 agent 循环里执行的行为策略,遵守度受 DeepSeek agentic 天花板限制(连续/演示编排本就弱);机制层的"演示完验收空转"是上一条 **trustReplyClaim** 根治的、与此正交。
- 互动回答必须"看得到/听得到"(用户定调 2026-06-19,行为策略):自主/在岗模式主人看不到聊天界面(只有悬浮本体)。实测 bug:演示后问"和豆包对比",详细对比写进了聊天框,但 TTS 只念了句"已对比差异"(回复被 **isTaskDeliveryReply** 因记录残留旧 PPT 误判任务交付→**briefSpokenSummary** 摘要),于是主人既没看到也没听到=白做。原则:互动回答绝不能只写进看不见的聊天框+只念摘要,必须有一条主人此刻能感知的输出——短内容 **speak** 念实际内容,内容多就 **open_preview** 开成可见文档给他看再念要点,怎么做大脑自己定。已写入两段系统提示。分类器已修(2026-06-19):**isTaskDeliveryReply** 去掉"记录里有任何产出物即判交付"那支,只按回复文本本身判(**replyLooksLikeTaskDelivery**:声称产文件/含代码块/含绝对路径)——常驻在岗复用同一记录、跨回合累积旧 PPT,旧逻辑把"和豆包/Cursor 对比"这种纯对话回复误判成交付→只念摘要;现纯对话回复(无文件声称)一律念全文。实测:记录带产出物时间对比类问题,TTS 念全文(非"已对比差异"摘要)。退出演示确定性关(2026-06-19):实测说"退出演示"DeepSeek 口头答应却不调 **present_fullscreen(false)** → 窗没关;**handleStandingPersonInputIfNeeded** 顶部加确定性闸:**isExitPresentationCommand**(退出/关闭/结束演示...纯函数)命中 + 预览开着 → **exitPresentationDeterministically**(停批量+掐 TTS+close+抑制重弹+停回合+出声确认"演示已退出"),不靠大脑。NestedLoopTests 覆盖两者。
- 互动中接任务·队列+主动汇报(用户定调 2026-06-19,通用"像一个人"):互动=接收任务的场合。
① 新输入路由按"互动 vs 执行"分流(**handleStandingPersonInputIfNeeded**):互动中(**previewController.isPresented**)→ 不放弃,**injectCorrection(interactionInputFraming)** 让大脑判断(控制现做/"放下一切马上做"紧急→当场做/其余新任务→**spawn_task** 派后台后接着演示);执行中(非互动)→ 单线程打断即放弃(**interruptedResumeFraming**)。② 子任务完成汇报择机发:**deliverOrQueueSubtaskReport** 一律入 **pendingSubtaskReports** 队列 + 回填

派发气泡并标记已念(抑制 `speakLatestReplyIfNeeded` 自动朗读,免在此刻打断互动);真正汇报由 `drain` 发——互动中:`finishAutonomousRun` 在岗分支
`drainPendingSubtaskReports()` 捎带进下次主线程回复;待机:自主 1s 自驱 `Task`(后台安全)每拍 `deliverPendingReportsIfIdle()`(在岗+非互动+队列非空→贴气泡自动朗读=主动出声汇报)。`isEngagedInInteraction`=演示开着 || 在飞回合 || 在朗读。子任务本身=任务→照常走验收。③ 汇报单线程顺序 + 合并接连完成(用户定调 2026-06-19,"否则太乱"): `isEngagedInInteraction` 含 `isSpeakingOrQueued` → 正念一条报告时新完成的攒着不插队(单线程顺序,念完再报下一批);`deliverPendingReportsIfIdle` 加防抖:最近一次完成后等 ~2.5s 稳定(其间又完成的并入)再一起报,最长 12s 兜底必发;
`drainPendingSubtaskReports` 多条合并成一条带序号("刚才你交代的几件都办好了:1、... 2、...")。E2E:多不相关任务→`spawn_task` 派子任务→[验收]通过→子任务完成·入待汇报队列→主动汇报(待机)真出声(多条合并)。

- 产出物登记(`write_file + run_command`):`runAgenticTool` 里 `write_file` 直接登记;
`run_command` 产出的交付物从命令+输出抽路径补登(`extractRunCommandArtifacts`),否则面板缺文件/验收误判"不存在"反复打回(曾的 PPT 死循环)。扩展名白名单含文档/媒体 + 源码/配置/构建产物
(`.java/.xml/.yaml/.json/.py/.gradle/.jar...` 带 \b 防 cpp/jsx 前缀截短;2026-06-21 补——原来只认 pptx/docx 等文档,致 SpringCloud 等 `run_command(heredoc)` 写的工程文件全不进产出物面板;误登记由 `mtime` 过滤兜住,`RunCommandArtifactTests` 棘轮)。执行前快照文件是否已存在→标 新增 / 修改(`LingShuArtifactOperation`);记录右侧文件面板按新增/修改分组。注:`lingshu_task_records MCP` 返回的是 `artifactCount`(整数)不是 `artifacts` 数组,核对产出物数看 `artifactCount` 或走 `lingshu_task_detail`。
- 边做边想(执行流可读):`LingShuGatewayAgentModel.onReasoning` 上报模型发起工具调用时的旁白,主/自主会话经 `recordAgentReasoning` 落"分析"行进任务记录(像 `codex` 的「分析→动作」)。系统提示要求每步先一句话讲清要做什么、且用 `web_search` 别手写 `curl|grep` 反复抓(控成本)。
- 任务记录冷热:`LingShuTaskExecutionJournal` 按时间窗分热/冷——热=最近一个月(`updatedAt≥now-30d`,再设 300 条上限防爆),更早转冷备(`archivedTaskExecutionRecords`)。归档页可一键纳入冷备。参考上下文冷备策略。
- 流式 TTS 喂 PCM 必须在后台线程,不挂 `@MainActor`(在岗音频卡顿根因,2026-06-16):`AVAudioEngine` 渲染本就在实时音频线程,但喂 PCM(WAV 解析 + 偶数对齐 + 建 `float buffer + enqueue`,含两次逐样本循环)若在主线程做,主线程一忙(尤其常驻灵枢在岗时的周期感知/VL)就喂不上下一段 → 引擎欠载 → 段间卡顿/逐字静音。普通对话主线程闲所以顺,一在岗就卡。修法:把每段重活抽成
`VoiceIOManager.renderStreamingSegment(nonisolated,后台线程跑),drainer(startStreamingSpeechDrain)` 只在主线程做廉价协调(取句序/更新播放器引用/电平回灌闭包),`await Task.detached`
{ `renderStreamingSegment(...)` } 把重活全部 off-main;
`LingShuStreamingPCMPlayer` 本就 `@unchecked Sendable`+锁,后台 `enqueue/start` 安全。配套:周期感知 VL 调用(~400KB base64 的 JSON 序列化)也包进 `Task.detached off-main`。两条 TTS 路径都要修(易漏的第二处):① 主会话流式 `delta sink` → `speakStreamingSentence` → `startStreamingSpeechDrain(path1)`;②

- voice.speak**(整段回复一次性朗读)→**speakWithStreamingSegments(path2)**——自主/在岗会话没 **delta sink**, 回复走的是 **path2**。path2 旧实现 **openStreamingSegment** 用 **URLSession.bytes** 逐字节在多线程 **for try await byte in bytes** 边收边喂(数十万次多线程 **await** + 多线程 **enqueue**)=「普通对话顺(path1 已修)、一上岗就卡(path2 没修)」的真凶。已弃 **openStreamingSegment**, path2 改与 path1 同款:整段 WAV 后台拉取 + **renderStreamingSegment off-main**(代价:首段从「首包即播」变「整段拉完再播」,首声 +~0.5–1s,换全程不卡;后续段 8 路预取无缝)。所有 **enqueue(pcm16:)** 现都只在 **renderStreamingSegment(nonisolated)**里、经 **Task.detached** 调,无一在多线程。实测:在岗多句 TTS + 并发 VL,无跳段/无降级、感知 seq 平滑。铁律:任何会被多线程负载饿到的实时音频喂数据,都放后台线程,多线程只协调;改音频先确认 **path1+path2** 两条都覆盖。
- 高频 **@Published** 别拖累重视图(滚动抽搐 + TTS 卡顿根因):
VoiceIOManager.inputLevel/outputLevel 在收音/播放时 ~20Hz 刷新 (**VoiceIOManager+AudioLevels**)。任何 **@ObservedObject var voice** 的视图会随之 20Hz 重渲——若该视图含重内容(**markdown** 聊天列表 **N** 条气泡),每秒重算 20 次 → 滚动条上下抽搐 + 多线程过载 → 喂 TTS PCM 的 **MainActor** 任务被饿 → 播放卡顿。铁律:重列表(聊天滚动)抽成只订阅 **state** 的子视图(**LingShuChatScroll, LingShuDialogueViews**), 与 **voice** 高频电平解耦(**SwiftUI** 见输入 **state** 未变即跳过子视图 **body**)。电平只该被真正用它的小视图(**LingShuMinimalVoiceView** 波形)+ **VAD** 逻辑(直接读属性,不靠订阅)消费。新增订阅 **voice** 的视图务必先想:它会不会被 20Hz 拖着重渲重内容。
 - 音频/文字 **desync**:新一轮 **runMainAgentTurn** 开头调 **interruptSpeechOutput?()** (根视图注入=**voice.stopSpeaking()**)掐掉上一条还在放的 TTS,否则上一条(多段)音频会盖到新轮。
 - 多轮串行(关键):**LingShuAgentSession** 是 actor,send 在 **await model.respond** 挂起时会被下一条 **send** 重入 → 多条 **user** 消息堆进同一上下文 → 模型把多轮揉一起答(串台)+ 并发模型调用(疑似 **GatewayError** 真因)。**runMainAgentTurn** 用 **await previousTurn?.value** 把多轮严格串行——快速连发的输入逐条处理,各答各的。改执行路径时务必保持这个串行。
 - 子任务并发硬上限=3:模型 **spawn_task**=背压拒绝,用户连发派发=排队(2026-06-19 区分):
spawn_task(模型自己派生)派前查 **runningCount() < capacity()**、**spawnDetached** 内 **hasCapacity** 原子兜底——满 3 直接拒绝并回报模型(防模型一次甩几十个子任务 runaway,这是对的)。但用户经分诊派发的任务(**dispatchIsolatedTask**)满 3 时旧逻辑也直接拒"重发一次"=体验差;现改为自动排队:**spawnDetached** 返 **false** 无副作用(不注册 **session**) → 起一个轮询 **Task** 每 2s 重试 **spawnDetached**、有空位自动派出(**bubble** 保持加载=排队中),最长约 10 分钟超时才放弃。两条路径区别对待:模型 runaway 要挡、用户连发要排队。改并发上限只动 **LingShuAgentOrchestrator(maxConcurrent:)**。
 - 派发/spawn 隔离任务必须继承父上下文 **shell** 预授权(2026-06-19,真机"演示卡在'执行中:跑命令'根因):**agentBuiltinTools** 的 **.standard** 策略 **allowShell** 公式=!
requireHumanApproval || sessionShellAlwaysAllowed,不认在岗/自主的完整授权(**autonomousRun.isActive && .full**)。而 **dispatchIsolatedTask**(用户分诊派发)和模型 **spawn_task** 子会话原都写死 **.standard**——于是在岗时派发的任务跑第一条非只读 **shell**(大脑常以"先检查文件是否存在"开场)就 **allowShell=false** → **requestShellApproval** 弹审批框,而框被全屏演示盖住/无人点 → 永久挂"执行中:跑命令"(用户实测 2 分钟没反应、唤醒词也打不断)。修:加

- dispatchedTaskExecutionPolicy([LingShuState+BackgroundWatch.swift](#),= shellPreauthorized ? .autoAllowShell : .standard),两处派发都用它=继承父预授权(与 autonomous 自身 autonomousExecutionPolicy(.full→.autoAllowShell) 一致)。安全不破:autoAllowShell 下系统级敏感删改仍命中 forceConfirm(LingShuState+PipelineExecution line84/90 systemSensitive→强制弹)、只读命令本就免审批。**E2E** 实测:同一条之前卡死 2 分钟的演示指令,修后 t10s 打开预览、t15s 进全屏、idx 1→2→3 正常翻页推进,不再卡。
- 上下文裁剪(防旧任务污染新请求):常驻主会话 maxHistoryMessages=80,在 send 回合边界 trimHistoryIfNeeded 裁掉最旧的非系统历史(系统/身份/seed 永留,裁后不留孤儿 tool 结果)。配套系统提示明确"别把历史里无关的旧产出物当本次素材"。根治"被问做自我介绍 PPT 却把几轮前的别的 PPT/测试文件塞进交付物"。
 - 交付话术与返工文本解耦:验收经返工(round>0)才通过时,maker 最后一轮文本是"逐条修正"的内部 QA 记录,直接抛给用户=驴唇不对马嘴。verifyAndContinue 此时改调 composeDeliveryMessage(无工具小会话,据原始请求+真实产出物清单生成干净交付说明);首轮即过(round=0)仍用模型原话。占位收尾兜底(2026-06-21,清分系统实测):模型最后一步若是个静默 run_command(输出写进文件、stdout 空)→ 收尾回复退化成「✓run_command: (无输出,退出码 0)」把"无输出"丢给用户。verifyAndContinue 末尾加 isPlaceholderDelivery 检测(命中「(无输出」「已发起工具调用」「裸 ✓run_command」等占位)+ 任务真有产出物 → 同样走 composeDeliveryMessage 据产出物补一段像样交付。PlaceholderDeliveryTests 棘轮。
 - verifier 忽略自指/易变事实:产出物自报的文件体积/字节/页数/时间戳(每次重生成都变、与内容无关)+ KB 四舍五入(41KB vs 40KB)一律不算事实错误,否则陷入自指返工死循环 (verifyAgentDeliverable 维度②提示已写明)。
 - 数据网关 token 统一解析(VL/感知与 TTS 同口径):dataNetGatewayToken()——① 主通道就是网关→当前 apiKey;② 凭据库 credentialStore(灵枢配置数据库,AES-GCM 加密落盘,用户写入的权威来源);③ 随包 Resources/RuntimeConfig/datanet-gateway.token 兜底。cloudPerceptionClient 改走它(之前只读 credentialStore、不读随包 token → VL 说"没配"而 TTS 却能用的不一致已修)。写凭据:MCP lingshu_set_credential(provider,token) → credentialStore.setAPIKey(持久)。VL 图片审计走 base64(analyzeImage(imageBase64:)) → /v1/perception/swds-vision-fast 的 image_base64)。
 - 设计审计兼容描述型 VL:本网关 Qwen-VL 多返回 semantic_suggestions(summary/tags 描述)而非 score= 数字;auditDeckDesign 先 parseDesignScore 取数字,取不到则 deriveDesignScoreFromDescription(命中版式问题关键词→0.5+问题,干净→0.8),避免一律 inconclusive。审计带任务主题给 VL 以判配图切题。
 - 单图视觉理解走 swds-vision-fast + model=qwen2.5-vl(实测此组合单图 base64 可用、语义扎实);swds-vision-deep 是视频专用端点(实测拒收单图 base64),只用于 analyzeVideo 视频 + 周期性态势感知。改图源/模型只动 analyzeImage。
 - 产出物只登记真改动:run_command 执行前快照命令里提到的已存在文件的 mtime,事后只登记*

- *新建(原本没有)或 mtime 变了(真改了)**的交付型文件;只读/列(ls/cat/file,mtime 没变)→不登记(产出物面板别塞只看过的文件)。write_file 仍直接登记。
- **TTS** 不念"总用时"后缀:strippedForSpeech 跳过以 🗣 开头的行(纯展示,念出来怪)。
- **PPT** 默认用 **DesignKB** 配色,不套联网模板(关键教训):实测自动 acquire_resource 联网套通用 **Office** 模板会把模型想要的深色专业风 overpaint 成寡淡白底、版式更乱——通用 web 模板远不如 DesignKB 自带 palettes 好看。故 PPT skill 提示默认选 **DesignKB** 深色主题 (**midnight/graphite/royal**),不联网找 .pptx 模板;模板底只在用户明确提供品牌 .pptx 时用。
- 模板主题接管=显式 **opt-in**(不是默认):generator.py 的 extract_template_theme(从模板 theme1.xml 抽 clrScheme+fontScheme 推导 palette)只在 theme=="template" 或 {"palette_source":"template"} 时触发;普通 theme:"midnight" 等一律按 DesignKB 配色,模板不覆盖(否则模型选的深色被通用模板顶成白底=回归)。字体仅在能渲中文(ea/CJK)时采用,否则保 PingFang SC。清模板样例页用 drop_rel(只摘 sldIdLst 会残留 part→撞 slide1.xml 名)。
- **acquire_resource** 兜底仍在(但 **PPT** 默认不用):DuckDuckGo HTML 常被限流(202),模板站多 JS 非直链。LingShuResourceRegistry.curatedFallbackSources 内置已核实开源许可的 GitHub raw 直链(pptx-template=python-pptx MIT)。机制保留给"用户要基于某模板"的场景;自动 **PPT** 流程不再默认拉模板(见上条教训)。
- **review_design** 强制相关性裁决 + 真迭代:描述型 VL 只"描述"图、不主动判相关性 → 跑题配图曾拿 0.8 蒙混过关(deriveDesignScoreFromDescription 无问题关键词命中默认 0.8)。VL prompt 现要求第三行 relevant=yes/no/na,imageJudgedIrrelevant 命中 relevant=no/"配图不相关"→强制压到 ≤0.45 + 出 **lowPage**,逼模型删图改 icons 重生成。**PPT** skill 提示把"逐页迭代到每页达标"列为硬步骤(别一审完就交)。抽象主题(**AI**/软件/服务)默认 **icons+chart+色块**、不配照片——模型看不到图、免费图库对抽象词常返垃圾,治本是不配照片而非靠审计兜。
- **skill** 自发现安全模型(**discover_skill**):纯提示直接装;带脚本=静态门(LingShuSkillSafetyGate)→LLM 风险审(reviewScriptRisk 无工具小会话)→parseRiskVerdict **fail-safe**(只有明确 RISK=none 才自动装,低/高/含糊一律 risky)。risky → 装但 setQuarantine,materialize 时把脚本路径登记进 quarantinedScriptPaths,requestShellApproval 命中即强制弹审批(绕过 sessionShellAlwaysAllowed)+ 展示风险点,首次审批后 clearQuarantine 解除。forceFrontmatterID 把来源 id 改 discovered-* 命名空间,防 id: curated-ppt 覆盖内置/策展。绝不静默执行未审源代码。
- 从用户反馈学(子线程纠正→**dreaming**,最有意义的进化):用户在任务窗口对设计/**PPT** 任务给的纠正/追问是最高信号改进点。captureDesignFeedbackForDreaming(LingShuState+Dreaming,挂在 interjectCorrection+submitTaskFollowup)把纠正记进该记录 designIssues(标「用户反馈:」)并立即 learnDesignFeedbackNow(minSamples=1,绕过空闲节流)走 dreaming 蒸馏进 DesignKB 设计经验 overlay → apply_skill 下次注入 → 用户说过一次的问题不再犯。consolidator 提示把「用户反馈:」列为最高优先级铁律。只对设计任务捕获;红线仍 sanitizePromptOnly。
- 图标对比铁律(深色底别用黑图标):generator.py 的 place_icon 只用与背景对比的变体

(深色主题→-white,浅色→-dark);对比变体缺失就返回 False 让调用方画 ACCENT 亮色圆点兜底——绝不退用同色调 .png(那会在深色底成看不见的黑图标)。预栅格图标须成对供白/深变体(Scripts/prep-icons.sh)。

- 聊天列表=有界滑动窗口的 **VStack**(空白+卡顿双根因):**LingShuChatScroll** 用 **VStack**(非 **Lazy**,根治"行没滚进视野不渲染→空白要拖滚动条")+ **windowSize** 滑动窗口(只渲染最近 **N=40** 条,suffix(windowSize);"加载更多"按钮 +40 并按需 loadOlderChatHistoryIfNeeded)。为什么两者都要:**VStack** 全量 realize 解决空白但消息一多就卡 → 窗口把渲染量压住保流畅。别回退 **LazyVStack**(空白坑),别去掉窗口(卡顿)。git status 抓 git。
- 代码交付=独立 **tab**,且只算本任务自己的改动:**TaskArtifactFilesPanel** 是两个 **tab** ——「产出物」(文件)/「代码改动」(git);后者仅 record.codeChanges != nil 才出现(非代码需求连 tab 都没有)。captureCodeChanges 只看本任务 **artifacts** 里的源码文件(isCodeLikePath 滤掉 pptx/图片/pdf/assets),再 gitChangeSummary(limitTo:) 把 porcelain 与本任务文件求交——绝不扫整库(否则"问个时间"也列出仓库一堆历史脏文件,曾的 bug)。已提交的不在 porcelain → 不计;本任务文件全已提交 → nil。git 路径多候选兜底(launchd 精简 PATH 下 /usr/bin/git shim 会失效)。强制绑定 **git**(用户定调 **2026-06-21**):captureCodeChanges 先把工程根(commonParentDir(taskCodePaths))经 ensureGitRepo 自动 **git init**(不在 git 工作树才建,避免嵌套;不自动 commit → 新文件留未跟踪=审查里红绿可见),再以工程根为 workingDir 拉 gitChangeSummary——根治"新工程没 git → 审查说没改动"。ensureGitRepo 有 isSafeToInitGit 护栏: /·/tmp·/Users·家目录·桌面/文档等过宽目录本身绝不 **init**(只对工程子目录绑),GitBindSafetyTests 棘轮。未提交分支名经 symbolic-ref 兜底(刚 init 显 main 不再误显「detached HEAD」)。清分系统实测(**2026-06-21**)逼出两处 codeChanges 失准并修:① 算工程根剔除 /tmp、/var/folders 等 scratch 输出(一个跑到 /tmp 的结果文件会让公共父目录跨根缩到 / → nil → 工程根判丢);② git status --porcelain 加 --untracked-files=all——默认会把整个未跟踪目录折叠成 ?? proj/ 一条(不列单个文件),与本任务文件路径求交永远落空 → codeChanges 恒空;-uall 列出每个文件才能命中(实测在父级是 git 仓的 ~/app 下新建工程时复现,修后 repoName=app branch=master files=[proj/x.py...] 正确)。
- 不为可选脚手架工具交还(**codex** 式"没有搞不定的事"):LingShuAgentLoop.optionalScaffoldTools(现含 update_plan)——卡在它们上(连续 5 次相同调用)绝不 .maxTurnsReached 交还,而是清签名史 + 注入纠偏"它只是可选计划工具、不是任务本身,跳过它直接 write_file/run_command 做事"。update_plan 解析失败的返回也改成 steer 跳过而非死磕格式。只有卡在真任务动作上才诚实交还,且给"结果+原因+下一步"不是空喊走不通。治"问写个 hello.py 因 update_plan 格式反复失败而放弃"。
- 过度自测收敛(**2026-06-16**,超级玛丽暴露):stuck 检测只抓"完全相同"调用,抓不到"每次略不同的测试空转"(DeepSeek 把超级玛丽做好后又跑了 35min node test_engine.js 反复验、不宣布完成 → 独立 verifier 一直没触发)。runLoop 加过度自测门: mutatingToolNames=write_file/edit_file;已产出过文件后(sawMutation)连续 overValidationNudgeAt(10)步不再改任何文件(只测试/查看) → 注入纠偏"工作已完成,别再空转,直接给最终交付";到 overValidationForceAt(20)仍空转 → 强制 .completed(在工具执行后判,保证 messages 良构)交独立验收(checker),而非无界自测 / 被撞顶误判异常。纯探索期(还没写过文件)不触发;周期性改文件的真进展任务

turnsSinceMutation 永远到不了门槛,不误伤。

- 贾维斯式启动按钮(性能:外壳静态、只动小反应堆):
JarvisLaunchButton(LingShuAutonomousRunViews)的渐变/边框/阴影/文字是静态的,不进 TimelineView;只有左侧 JarvisReactor(弧环+呼吸)自带一个低帧 20fps TimelineView 动那一小块——避免每帧重渲整颗按钮拖卡(已去扫光+阴影脉动,保留呼吸+弧环,计划 §7)。常驻灵枢无需目标,按钮始终可点(isEnabled=true,文案「让灵枢上岗」)。
- 独立运行=常驻灵枢,不再要求目标(用户定调 2026-06-16,推翻旧"目标必填"):idleBody 删了目标输入框,选权限级 → 点「让灵枢上岗」(goLiveAsStandingPerson)→ 环境自检通过即直接上岗(objective="", runbook=nil、phase 直接 running)。上岗后由对话/语音自然驱动(handleStandingPersonInputIfNeeded 把输入喂在岗会话);一段处理完保持在岗不收工(finishAutonomousRun 空目标分支)。在岗收尾必须每个终态都有空目标特例(2026-06-19 修"问天气撞顶后退岗+无回复"):finishAutonomousRun 原只有 .completed 判空目标保持在岗,.maxTurnsReached(在岗一句话查询如外部源不可用反复重试撞顶)漏了→ 走目标驱动失败分支 endAutonomousActivity() 把在岗停了(standing=False)+ 该给主人的答复成了"⚠️独立运行步数上限",主人遂"听到回合内 speak 的 TTS、却看不到聊天回复、灵枢还悄悄下岗"。修:.maxTurnsReached 加空目标特例——把已有最好结果(lastText,空则兜底句)当在岗回复贴聊天 + 保持 .running 在岗、不 endAutonomousActivity。实测:问天气 standing 全程 True、聊天出真天气气泡。🔗上传降为「可选素材」。仍保留目标驱动路径:handleAutonomousRunCommandIfNeeded(对话里"进入独立运行模式 + 目标")仍走 prepareAutonomousRun(目标非空→runbook 规划)。独立运行默认「完整授权」=完整电脑控制:只读命令免审批直放、写动作直接执行、不再要授权,唯一例外=删/改系统级敏感文件强制弹审批 (LingShuShellCommandPolicy.touchesSystemSensitivePath → request ShellApproval(forceConfirm:),绕过 sessionShellAlwaysAllowed;无人值守则安全拒)。
- 周期感知=专用驱动 Task,不靠 UI Timer + AX 放后台 + 抑制 App Nap(自主模式 模块 2 关键教训,2026-06-16):常驻灵枢接管时灵枢自己必然在后台(它在操作别的 app),此时① SwiftUI 的 Timer.publish(根视图 coreTimer → tickCoreTimers)会因窗口被遮挡/后台被暂停(实测心跳冻结,seq 不再涨);② 系统 App Nap 进一步暂停后台活动;③ AX IPC(AXUIElementCopyAttributeValue 取焦点窗口标题)是同步阻塞调用,放主线程每拍跑会饿死同在主线程的 TTS 喂 PCM 任务 → 在岗时音频卡顿(实测根因)。修法:beginAutonomousActivity()(进入运行/在岗态调,幂等)= ProcessInfo.beginActivity(.userInitiated) 抑制 App Nap + 起一个专用 autonomousPerceptionDriverTask(注意:不是 @MainActor),循环里:await perceptionHeartbeatOnMain()(主线程廉价:守门+采音频电平+锁存起音+判到点,绝不碰 AX/截屏,只返回前台 app token)→ 到点才在后台线程算 LingShuComputerControl.windowTitleSignature(pid:label:)(慢 AX)→ await perceptionApplySignal(signature:)(主线程轻量:planner 决策+启 VL/唤醒)。endAutonomousActivity()(暂停/停止/目标驱动收尾)释放令牌 + 取消驱动。tickCoreTimers 不再调周期感知(UI Timer 后台被停 + 主线程 AX 饿音频,双坑)。frontmostContextSignature() 拆成主线程 frontmostAppToken()(读 NSWorkspace,快)+ 后台 windowTitleSignature()(AX,慢)。perceptionDebugLine 非 @Published(只 MCP 读,不触发重渲)。④ 灵枢说话时暂停感知重活(「一字一字」卡顿真因,2026-06-16):perceptionHeartbeatOnMain 见

- `voiceManager?.isSpeakingOrQueued` 即 `return nil`——TTS 播放期间截屏子进程 + 全屏图缩放(**CGContext**) + **VL** + **AX** 全停,否则这些 CPU/IO 突发会抢占实时音频渲染线程(**AVAudioEngine** 渲染在 RT 线程,被 CPU 饿到就出现逐字卡顿,即使 PCM 喂数据已 `off-main` 也救不了),说完即恢复(只剩廉价心跳采音频,seq 仍涨)。实测:整段 TTS 期间 debug 持续「让位 TTS」、无 **VL**。**MainActor** 本身在后台不冻(MCP 仍即时响应)——冻的是 UI Timer,故专用 **MainActor** 心跳可靠。
- 周期感知 **VL** 必须缩图(否则网关上游 **500**,**2026-06-16**):全屏 Retina 截图 PNG ~4MB,直接 base64 喂 `analyzeImage` → 数据网关上游模型连续 3 次失败→**HTTP 500**(`requestFailed(500, "上游模型连续 3 次调用失败")`)。 `screen_capture` 工具偶尔侥幸过(上游轻载)。修法:
`LingShuComputerControl.downscaledJPEGBase64(pngPath:maxDimension:1280,quality:0.6)`(`NSImage` → `CGContext` 缩放 → `JPEG`)缩到 <300KB,VL 稳定返回真实屏幕语义,且更快更省。周期感知关键帧用完即删(零留存)。**screen_capture** 工具也已加缩图兜底(**2026-06-16**):它是给大脑「看清屏幕后精确点击/输入」的,缩太狠丢 UI 细节,故走体积分级 + 失败重试(`LingShuState.analyzeScreenForControl`):PNG ≤1M(`screenCaptureOriginalByteCeiling,shouldSendOriginalScreenshots` 判,纯函数可单测)= 非 Retina/小窗口 → 原图保最大细节;超 1M(全屏 Retina ~4MB)或原图调用失败 → 缩到 **1600** 长边 **JPEG q0.7** 重试一次(比周期感知 1280/q0.6 更精)。工具回给模型的 PNG 路径始终是原图(只缩 VL 上送的 base64)。
 - 周期感知节流分层(成本/延迟):VL 单次 1–3s,绝不能每拍都跑。纯逻辑
`LingShuPerceptionCadencePlanner.decide`(可单测)按
`LingShuPerceptionCadenceConfig`(tick 4s / VL 硬下限 9s / 强制刷新 45s / 唤醒冷却 30s)决策:先廉价闸门(前台签名 `frontmostContextSignature` 变没变 + 音频电平突变 `LingShuAudioActivityDetector` 滞回分类)再决定花不花 VL;大脑在跑回合(`autonomousRunTask != nil || hasActiveModelCall`)时让位不重复 VL/不打破;自主反应唤醒默认 **opt-in**(`autonomousAutoReactArmed`,安全默认关),且唤醒=「屏幕异常哨兵」只提醒真问题(**2026-06-17**,见上)。`digest` 只显示在接管态遮罩,不再前置进指令(**2026-06-17** 去污染,`standingPromptWithPerception` 已删)。
`composedPromptHint/livePerceptionContextProvider` 是旧管线遗留已无人调用,感知不自动进 agent 回合。
 - 自主反应唤醒=临时评估器,绝不污染在岗对话/聊天(否则真提问被带跑偏,实测 **bug**):`wakeStandingPersonWithPerception` 不再直接用在岗会话 `resume` 一个"[自主感知]..."回合——那会让每次屏幕变化都生成一句"屏幕变化…保持在岗待命",既刷屏又灌满在岗会话上下文,导致真问"介绍一下你自己"也只回"在岗待命"(而干净主会话答得很好=灵枢污染问题,非模型问题)。改:`evaluatePerceptionForAction` 用一次性 **throwaway**
`LingShuAgentSession(maxTurns:1,无工具)` 评估只回 **ACT: .../IDLE;IDLE**(日常界面变化的 99%)只留一条轨迹、不进聊天、不碰对话上下文;只有 **ACT** 才用在岗会话真去做并正常呈现。
 - 常驻在岗 **system prompt** 必须强制"全力正面回应",禁止待命套话:旧 `prompt` 过度强调"安静在岗、不臆造任务" → 模型对真请求也只回"您动口我动手/随时吩咐"敷衍。现
`prompt(autonomousSystemPrompt 空目标分支)`立铁律:主人一开口就完整正面回应(问题答全、任务做完),绝对禁止用"在岗待命/随时吩咐"代替真回答;"安静不臆造"只适用于真没输入的空闲。`kickoff` 也只打一句招呼、不反复"待命"。实测:同问题在岗回复从"随时吩咐"套话 → 变成

完整 5-6 句答案(与干净主会话一致)。

- 语音收听:**TTS** 时不停麦 + **AEC** 回声消除 + **barge-in** 打断:常驻在岗会自动开麦 (`startStandingVoiceListening=LingShuVoiceCallController`)。**startRecognition** 开 `inputNode.setVoiceProcessingEnabled(true)`(系统 **AEC**)——TTS 播放时麦克风不自听灵枢自己的声音,故说话时持续收音;
`LingShuVoiceCallController` 的 TTS 分支不再 **stopRecognition**,主人持续说话 $\geq 0.7s$ (过 `bargeInThreshold`,AEC 残留短噪不触发)即 `stopSpeaking()` 打断 TTS、接住主人这句正常提交。AEC 失败则 `try?` 退回(识别仍可用)。注:**barge-in** 体验需真人对着说话验证,自动化测不了。自适应收口窗口(2026-06-19,修"喊灵枢后没说完就进处理中"):
`LingShuVoiceCallController` 的静默收口阈值原写死 `silenceHold=1.0s` → 主人停顿 $> 1s$ 就被切、话没说完就提交。改为据本句已累计有效说话量自适应(纯函数 `silenceHold(speechTicks:...)`,可单测):刚打断/喊完唤醒词、还没说实质内容 (`speechTickCount < substantiveSpeechTicks \approx 12` tick $\approx 1s$) → 给 5s 长窗口等主人开口;已说出实质指令 → 命令结束静默 3s 才收口提交。`speechTickCount` 每个过 `speakThreshold` 的 tick +1、`resetUtterance` 清零。`VoicePhaseTests` 覆盖窗口决策。转写收口健壮化(2026-06-19,修"喊灵枢说指令像没听到"):底层 `VoiceIOManager.evaluateUtteranceSilence` 原只靠 `lastPartialAt`(ASR `partial` 稳定 2s)收口——嘈杂/连续识别下背景噪音持续刷新 **partial** → **lastPartialAt** 永不稳定 → 指令永远卡 **partial** 不 **finalize**、不提交(实测:喊"灵枢介绍一下你自己",转写出来了但全程 `final=false` 没提交;同时音频 VAD 在噪音里也收不了口)。修:加音频静默收口——tap 记 `lastLoudInputAt`(电平过 `loudInputThreshold=0.10` 即"主人在出声"),
`shouldFinalizeUtterance`(纯函数,可单测)= 转写稳定 (`silenceFinalizeSeconds=2s`)或 主人停止出声 (`audioSilenceFinalizeSeconds=3s`,不被噪音 `partial` 拖住),任一即 `forceFinalizeUtterance`。3s 与端点容忍一致(中途停顿 $\leq 3s$ 不切)。`VoicePhaseTests` 覆盖。连续 TV 大噪音仍是难点(远场 ASR),但"安静房间但 ASR churn"已修。
- 只读命令免审批(计划 §3):`runAgenticTool` 里 `LingShuShellCommandPolicy.isReadOnly(command)` 命中 → `effectiveAllowShell=true` 跳过审批弹窗(`grep/find/ls/cat/head/tail/wc/file/git status...`,复合命令每段都只读才算)。写/删/网络/提权/重定向/命令替换一律不算只读照常拦。保持单一通用 `run_command`,不新增工具。
- 侧栏=本轮真实进展(计划 §2,去伪指标):`LingShuCallChainPanel` 改绑 `LingShuState+RuntimeStatus` 的实时派生值——
`hasLiveProgress/currentTaskRecord`(独立运行取其记录,否则模型在跑时取最近更新记录)/`currentToolDisplay`(尾部 `toolCall`)/`autonomousRunningStep`(`runbook` 当前步)/`currentPlanProgress/currentRoundElapsed/recentExecutionMessages`(执行轨迹尾部)。已删静态聚合遥测(在线/运行 agent、执行/监工线程、巡检轮次等伪指标)及其 4 个子视图;真空闲给有意义空态。
- 凭据零明文(计划 §5,方案 B):`remember_credential` 存 `credentialStore`(AES-GCM,不回显)、`list_credentials` 只列 key 名;大脑用凭据只写 `{{cred:KEY}}` 占位符,`runAgenticTool` 在执行层对 `command/url` 调 `resolveCredentialPlaceholders` 换真值、执行后 `redactSecrets` 给输出打码。明文不进模型上下文、不进任务记录(记录保留占位符)。常量

userCredentialPrefix/credentialPlaceholderPattern 须
nonisolated(executor/测试要用)。

- 代码任务确定性硬门=测试门 + 运行门(计划 **\$4** + 执行恢复力 **2026-06-16**):系统提示把"写测试跑全绿"+"程序真跑起来不崩"列为编码硬步骤;**verifyAgentDeliverable** 对产出真源码(**isTestableCodePath**)的任务加两道确定性门,任一不过即使 LLM 放行也判未达标并回灌原因:① 测试门 **codeTaskTestEvidence**——扫记录配对 **toolCall** → **toolResult**,命中测试运行器(**swift test/pytest/npm test/go test...**)或执行了测试文件、且该次 **run_command** 成功=全绿;② 运行门 **codeTaskRunEvidence**——扫构建/运行程序本身的命令(**looksLikeBuildOrRunCommand** 且非测试运行器),看最近一次是否干净(退出码成功 且 输出无高置信崩溃签名 **outputLooksLikeCrash:traceback/段错误/panic/fatal/编译失败...**);最近一次跑崩=未达标,把崩溃片段(**crashSnippet**)回灌逼修到真跑通(没跑过程序则只由测试门把关,不单独拦)。治"建到能跑却崩了→拿异常当交付"。测试文件经 **write_file** 自动登记为产出物。
- 计算机操作四肢(计划 **\$9**):**LingShuComputerControl**(截屏/CGEvent 鼠标键盘/滚动/AX 列元素)+ **computerControlTools()**(始终挂主+自主会话,授权在 **call-time** 判=开关即时生效)。授权 **computerControlGate:computerControlEnabled**(设置开关,持久 **UserDefaults**)或完整授权独立运行;动作类再要系统辅助功能授权(没有则 **requestAccessibilityTrust** 弹一次系统提示)。可靠点击 = 先 **list_ui_elements(AX 元素中心,点坐标)**再 **click**,别从截图猜坐标。坐标统一点、左上原点(避 Retina 2x 偏移);Swift 6:**kAXTrustedCheckOptionPrompt** 用字面值绕全局 **var** 并发报错。
- **AGI** 定位,不自设边界、不推竞品(身份铁律):被问能力/与 **CodeBuddy/Copilot/Codex/WorkBuddy** 比较时,绝不列"我不做/我不擅长 X、那是某产品的活儿"清单,绝不建议用竞品;讲"这类需求我会怎么落地"。代码任务(读库/改码/跑测/提交)是本职,别说"交给某某 Code"。唯一例外=诚实红线:确未具备的能力如实说在完善、但不点名竞品不罗列短板、不假装已做到。写在 **mainAgentSession** 系统提示身份段。
- 滚动跟随别用动画(回弹根因):聊天自动到底用 **proxy.scrollTo(lastID, anchor:.bottom)** 不加动画——对正在增长的"思考中/流式"末条做 **withAnimation scrollTo** 会过冲再回弹(用户看到"抽一下");另在 **onChange(missionTitle)** 补一刀(末条 **loading** 时阶段变化也重新顶到底,保证"思考中"气泡始终可见)。详见 **LingShuChatScroll**。
- **dreaming** 自固化红线:Phase 2 离线固化(**LingShuDreamingConsolidator**)只自动落纯提示 **skill**——**sanitizePromptOnly** 结构性剥掉一切围栏代码块与脚本/代码/生成器小节,绝不自动写可执行脚本(供应链红线,见 [skill-self-evolution] 记忆);带生成器的 **skill** 仍只能人工授权 + 安全门。只固化"成功 ≥ 3 且通过率 ≥ 0.6 且现有 **skill** 未覆盖"的领域,不抢策展 PPT。
- 任务窗口结构化录制 + 净化:工具活动不再灌裸字符串——**runAgenticTool** 录 **LingShuTaskExecutionMessage.detail(.toolCall/.toolResult/.fileEdit,write_file** 执行前抓改前内容算行 **diff**);窗口据 **detail** 渲染 **codex** 卡片。所有进窗口的文本过 **sanitizeRecordText**:抹底层模型名(**MiniMax/Qwen...**,身份铁律)+ 把裸 **tool_calls** JSON 收敛成一句话(杜绝截图里那种 **{"role":"assistant","name":"MiniMax AI",...}** 泄露)。 **detail** 字段 **Optional** → 旧持久化记录解码缺省 **nil**,向后兼容。
- 窗口内追问续跑:**submitTaskFollowup** 把追问当同一条记录的续跑

- (runMainAgentTurn(taskRecordID:)),执行流追加进窗口、走持久主会话(带上下文);最终答复也 append 成 .result 消息(时间线读起来连贯)。
- 朗读内容裁剪(任务型只念摘要):[LingShuState+SpokenReply.swift](#) spokenReplyText——任务型交付回复(有产出物 / 声称产出文件 / 含代码块/路径,常中英混杂)只念一句口播摘要(briefSpokenSummary 小会话生成);对话型/汇报型(干净中文正文)念全文。判定 replyLooksLikeTaskDelivery(纯函数可测)。根视图 speakLatestReplyIfNeeded 异步取 spokenReplyText 再 voice.speak;感知/记忆仍用全文。
 - 子任务→主线程简报(信息同步≠上下文同步):子任务完成/卡住/失败 → briefMainThread([LingShuState+AgentOrchestration.swift](#)) → session.injectBriefing,在主会话回合边界作为 system 提示注入摘要(≤200 字),主线程下次作答即知悉,不搬子任务完整 transcript(对齐 codex 的 subagent 汇报)。每条子任务本身是单线程 actor 会话。
 - 任务窗口附件:TaskWindowFooter 走与主输入框同一套 presentAttachmentPicker/pendingAttachments/LingShuAttachmentTray;submitTaskFollowup 与 interjectCorrection 都前置 attachmentContextBlock() 再 clearAttachments() (与 sendPrompt 同口径)。
 - 先计划后执行=LOOP 标准(决不能丢):多步任务落地前模型先调 update_plan 列 3-7 步计划,再逐步执行、每步更新 status(LingShuState+TaskPlan.swift);计划存任务记录 plan 字段、窗口顶部 TaskPlanCard 渲染成 todo(进行中/已完成实时变)。系统提示硬性要求(mainAgentSession),主+子会话都挂 update_plan;纯对话/简单问答跳过。注:旧 LingShuTaskPlan.swift 是无人调的遗留模型,真计划走记录 plan 字段。
 - 回复带总用时:runMainAgentTurn 起点记 turnStartedAt,收尾给气泡末尾追 总用时 X(formatElapsed);极简语音模式不加(会被 TTS 念);任务记录/记忆存干净 text 不含此后缀。
 - 极简对话模式=纯对话,不派生子任务:isMinimalVoiceMode 时——① spawnTaskTool handler 直接拒绝派生(回 steering 文案让模型本会话作答);② runMainAgentTurn 的 guidance 换成「对话模式」提示(口语直答、不写文件/跑命令/套交付模板、不走固化 skill)。聊天诉求(如"做个自我介绍")不再被当任务执行。其余模式不变。
 - 流程纠正(看到跑偏即时干预):LingShuAgentSession.injectCorrection 置 pendingCorrection,runLoop 在回合边界(工具结果已补齐 / 模型刚出文本)采纳为最高优先级 user 注入——不打断在飞工具、不丢上下文、不产生半截 tool_calls(比"停止后重发"平滑)。模型自认收尾但刚收到纠正 → 不收尾,带纠正继续(纠偏"假收尾")。runLoop 加 Task.isCancelled 检查让"停止"真停。入口:任务窗口底栏执行中输入即"纠正"(interjectCorrection)、MCP lingshu_interject(可带 recordId)。派发隔离子任务也能纠偏(2026-06-16):interjectCorrection 按 recordID 反查 agentSubTaskRecords → 命中则 orchestrator.injectCorrection(id:) 注入那条隔离子会话本身(主/自主会话的 interject 够不到编排器里的子会话——否则空转的派发 maker 没法从外部叫停)。实测:跑"斐波那契前 20 项"中途注入"改前 8 项+中文序号" → 落盘文件与答复均按纠正交付;单测:编排器注入纠正到隔离子会话,resume 后采纳。语音 barge-in 必须同步置 batchInterruptRequested(2026-06-19,修"演示中打断 PPT 疯狂翻页"):barge-in 在 partial 就掐 TTS(LingShuPerceptionActions 两处:唤醒词打断 / 纯唤醒打断),但

`batchInterruptRequested` 原只在 **final** 收口才经在岗输入置位。那个 `partial` → `final` 间隙里, `run_steps` 的 `speak` 步因 TTS 被掐而瞬间返回 → 连发 `preview_next` 狂翻页。修:两处 `barge-in` 掐 TTS 时立刻同步置 **`state.batchInterruptRequested = true`**, 批量在下一步边界即停。 **post-STT MCP 入口:** `lingshu_voice_text`(掐 TTS + `submitVoiceTranscript`) 忠实复现"一句语音指令转写收口后"的下游, 供无音频时完整演示打断/翻页——实测"翻到第 12 页演示" → `pageIndex` 真到 12。物理中断兜底(2026-06-19, 因语音打断在演示中不可靠): 实测演示中喊"灵枢跳转到第五页"被丢弃——控制日志 **`voice/barge: wake=false busy=true=ASR`** 没把唤醒词识别进来, 命中严格唤醒门 (`LingShuPerceptionActions` line246 `strictWakeMode && !containsWake && !armedWindowOpen` → `drop`)。语音管线天生脆, 故加两条不依赖 ASR 的物理中断 ([LingShuState.swift](#) `pauseActiveFlow/abortActiveFlow`): ① 动鼠标 → 软暂停: 全屏演示中 `LingShuManualTakeoverMonitor`(`local+global NSEvent` 监听 `mouseMoved/keyDown/scroll`, 武装后 0.8s 起判) 检测到键鼠 → `pauseActiveFlow`(置 `batchInterruptRequested` 停 `run_steps` 自动推进 + 掐 TTS, 真停不只退全屏——旧逻辑只 `setSlideshow(false)`、批量在对话框后面继续偷偷翻), 退全屏弹"停止/继续演示"; "继续演示" → 重进全屏 + `submitVoiceTranscript("从当前页继续")`。② 关任一演示窗 → 硬中断: `LingShuPreviewView` 的 `windowWillClose` 在用户态(`isPresented==true`, 区别于代码主动 `close` 已先置 `false`) 调 `controller.onUserClosedWindow`(根视图 `onAppear` 接到 `abortActiveFlow`) → 停批量+回合+派发任务(`cancelCurrentCall`)+掐 TTS+关预览+置 **`previewController.suppressAutoReopenUntil = now+5s`**: 5s 内 `open`/进全屏一律拒绝, 根治"手动退出后大脑下一步又把 PPT 弹回来"(关窗瞬间批量还有一步在飞的竞态)。实测: 动鼠标 `flow/pause`、点停止 `flow/abort` 真触发(真机用户实点验证)。 **`cancelCurrentCall`**(停止按钮/`lingshu_stop`) 在全屏演示中也一并关预览+设防重弹(原只 `abortActiveFlow` 关 → 停止后预览窗残留, 已统一)。纯唤醒词("灵枢")打断真停在飞回合(2026-06-20, 修"喊灵枢有'叮'声却打不断理解中"): `LingShuPerceptionActions` 纯唤醒分支原只置 `batchInterruptRequested`(只停 `run_steps` 批量), 不 `cancel` 任务; 而 `screen_capture/scroll` 这类模型调用循环靠 `Task.isCancelled` 才退 → 停不下来。修: 纯唤醒打断(`busy` 含 `loopPhase.isActive`, 理解中必触发) 改调 `interruptInFlightForWakeWord()` (`cancel activeAgentTurnTask+autonomousRunTask` + 置 `batchInterrupt+setLoopPhase(.idle)`, 保持在岗)。鸡生蛋: 系统授权弹窗灵枢点不了(2026-06-20): 被要求"给你授权"时它没 AX 权限就点不了那个授权弹窗(点弹窗本身要权限) → 反复 `screen_capture` 死循环卡理解中; 修: `computerControlGate` AX 失败时确定性打开『辅助功能』设置页 + 返回终止指令"鸡生蛋、别循环、口头让主人手动授权", 自主系统提示加同铁律。

- 反馈喂回 **dreaming**: 任务 🍷(`taskRecordFeedback`) → 该任务不进 `dreaming` 固化样本 (`succeeded=false`), 别从用户不满意的输出里学经验。
 - 开发者身份固化: Roy Zhao(写死在 `mainAgentSession` 系统提示 + `identityAnchorMessage`)。
 - 身份口径: 自我介绍只说"灵枢, 由我的开发者打造", 不提底层模型(可换); 靠主会话系统提示 + `seed` 末尾身份锚点压制历史里的旧错误自述。
 - 历史 **seed** 副作用: `seed` 里回放过的"已生成"假完成会污染模型行为 → 这是验收门必须存在的原因之一。
-

4. 测试 / 运行速查

```
# 构建 .app(Apple Development 签名)
bash Scripts/build-app.sh debug
# 重启(必须先退旧实例)
osascript -e 'tell application "灵枢" to quit'; pkill -f "dist/灵枢.app/Contents/MacOS"; open "dist/灵枢.app"
# 全量测试(单元/集成:脱网 + 脚本化模型 + 真 executor)
swift test
# 真·端到端冒烟(真模型 + 真运行 app:构建→启动→经 MCP 发真任务→按盘上真产出物断言→拆)
bash Scripts/smoke-e2e.sh # 需联网 + 模型 token, 不进离线 CI; PASS/FAIL + 退出码
```

两层测试: `swift test` = 单元/集成(脱网确定性, 脚本化模型 + 真 executor 真读写/真跑命令);

`Scripts/smoke-e2e.sh` = 真模型端到端冒烟(证明引擎从头跑通一个任务)。自主开发任务由策展技能 **curated-engineering**(LingShuCuratedSkillRegistry) 教大脑走"写码→build→单测→e2e 冒烟→红的自己修到全绿→交付"的完整闭环(配合测试门 + `smoke-e2e.sh`)。

MCP 控制服务(默认 `127.0.0.1:8917`, env `LINGSHU_MCP_PORT/LINGSHU_MCP_TOKEN`):

```
curl -s 127.0.0.1:8917/health
# 提交指令(走 agent 主入口)
curl -s -XPOST 127.0.0.1:8917/ -d
'{"jsonrpc":"2.0","id":1,"method":"tools/call","params":
{"name":"lingshu_send_prompt","arguments":{"text":"..."}}}'
```

工具: `lingshu_status`(含 `taskRecordCount/recentTaskRecords/产出物`)、`lingshu_send_prompt`、`lingshu_get_chat`、`lingshu_get_trace`、`meeting_*`、`agent_demo_start/agent_resume/agent_ledger`、`agent_run`(真模型单跑)。任务窗口/干预驱动(免点 **button**, 可脚本化验证): `lingshu_interject`(流程纠正)、`lingshu_voice_text`(**post-STT** 语音入口: 掐 TTS+submitVoiceTranscript, 模拟一句语音指令转写收口后的完整下游, 含演示中途插话; 无音频时完整测语音打断/翻页)、`lingshu_stop`(停止)、`lingshu_autonomous`(驱动自主模式/常驻灵枢: `action=go_live` 上岗接管 / `stop` 停止并夺回 / `pause` / `resume` / `arm` / `disarm`); `lingshu_status` 新增 `standingPersonOnDuty/autoReactArmed/perceptionDigest/perceptionDebug` 供脚本断言)、`lingshu_task_records`(列记录)、`lingshu_task_detail`(取 `codex` 卡片时间线: `toolCall/toolResult/fileEdit+diff`, 免开窗看)、`lingshu_task_followup`(窗口内追问)、`lingshu_task_feedback`(👍👎)、`lingshu_undo_edit`(撤销文件改动)、`lingshu_attach`(按路径加附件)/`lingshu_clear_attachments`(清空附件)、`lingshu_clear_context`(清空主对话上下文/新会话)、`lingshu_main_session`(读主会话上下文尾部, 验证简报/纠正注入)。实测: `fib` 任务 `task_detail` 拿到工具卡 + 内联纠正; `feedback` 设/清通; `write_file` 任务 `undo` 真删文件 + 卡标 `undone`。

- 清空上下文 / 新会话: `clearMainContext()` ([LingShuState+ClearContext.swift](#))——停在飞回合(`cancelCurrentCall`)→`mainAgentSessionHolder=nil`(懒重建, 下回合仍 `seed` 长期记忆)→`chatMessages/executionTrace` 回初始问候/待机态→`currentAgentTurnRecordID=nil`。只清当前对话上下文, 不动任务线程/执行记录, 不删长期记忆。入口: 输入坞(LingShuInputDock)的 `square.and.pencil` 按钮(带确认弹窗)+ MCP `lingshu_clear_context`。灵枢此前无清空上下文能力, 只能重启 app——本次补齐(2026-06-15)。

5. 已知缺口 / 待办(避免重复造)

- (已完成)记忆归一:① seed 改模型蒸馏摘要 (seededDistilledMemory/distillConversationMemory),不原样回放历史助手输出——根治"身份/假完成被模仿"的污染;会话内当轮仍 verbatim;实测重启续接(品红)+身份正确。② 旧 prepareMainThreadMemory 前置注入已移到回退路径(agent 路不再空跑;trace 不再有误导的"主线程记忆")。③ 加 recall_memory 工具(模型主动召回长期记忆,走 memoryService.searchColdMemory)。
- (已完成)run_command 审批续接:agent 路与旧管线同走 runAgenticTool→requestShellApproval(withCheckedContinuation 挂起→中文弹窗→续跑),工具 handler await 期间整条会话自然挂起,无需 ask_user 式 .blocked。原 §5 此条为陈旧记录。
- (已完成)工具集扩全 = MCP + 本地 skill:① 外部 MCP/连接器工具 (connectorRegistry.discoveredTools)已并入 agentBuiltinTools,复用 definition(forMCPTool:) 的 arguments_json 信封,runAgenticTool 的 MCP 分支唯一处 unwrapMCPArguments 解包(新旧共用)。② 本地固化 skill(组合注册表:用户>策展>内置)经 apply_skill 工具暴露给所有 agent 会话 ([LingShuState+AgentSkills.swift](#)):取专家提示+评审清单,并把 skill 自带生成器(已过安全门)物化到工作目录(materializeBundledScript,与旧管线共用)。命中真 skill(id 前缀 skill-)时回合开头 matchedSkillHint 广播可发现性;取用仍由模型经 apply_skill 决定。只读策略不挂 apply_skill(物化=写盘)。
- 会议端到端对话(进行中):① S1 会议对话闭环已落地+实测: LingShuMeetingConversationController(系统音频 →LingShuMeetingASR→检测对方说完一句 →submitTextInput(source:.meeting) 喂 agent 全能力→TTS)+ LingShuState+MeetingConversation + MCP meeting_converse_start/stop;实测启动即 isCapturing=true。② S3 输出路由已落地:LingShuAudioRouting(Core Audio 枚举输出设备 + 按名定向)+ LingShuStreamingPCMPPlayer.applyPreferredOutputDevice(把 TTS 引擎定向到虚拟麦)。③ S2 HAL 虚拟麦驱动 + 自安装: Drivers/LingShuAudioDriver/(AudioServerPlugIn loopback,v1 clang 编译通过)随 app 包发布(build-app.sh 自动编译+拷进 Resources),运行时灵枢自安装—— LingShuAudioDriverInstaller.installIfNeeded() 一次管理员授权 (osascript)拷到 /Library/Audio/Plug-Ins/HAL/+重启 coreaudiod,用户不跑脚本(拍板:装了灵枢就有这能力);meeting_converse_start 时自动触发。驱动属性模型已补全 (PlugIn/Device/输入流/输出流 + loopback IO + 零时戳),编译/签名/自安装均通过(2026-06-15 上机)。上机迭代(2026-06-15,真因已锁定为「公证」):证书 Developer ID Application: Yang Zhao (KM7N84AC9Y) 已导入钥匙串、私钥在本机、find-identity 有效。修了两个 bug:① build-app.sh 的 --deep 不遍历 Contents/Resources/*.driver →驱动一直 ad-hoc(现单独签驱动再签 app);② installIfNeeded 见已装就跳过→残留旧签名驱动永不升级(现比对主执行文件字节,签名变就重装),startMeetingConversation 也别再用 isInstalled() 短路。实测自安装跑通:驱动落 HAL、codesign --verify --strict 通过、

Authority=Developer ID + TeamID KM7N84AC9Y、coreaudiod 重启。但设备仍不出现——`spctl -a -t install` 判:我们 Unnotarized Developer ID → rejected,BlackHole Notarized Developer ID → accepted;其余全同 (MH_BUNDLE 双架构、工厂符号 `_LingShuAudioDriver_Create` 导出、Info.plist 的 `CFPlugInTypes=HAL UUID、root:wheel、hardened runtime`)。结论(纠正旧判断):**coreaudiod** 静默只加载「已公证(notarized)」的 **HAL** 插件——**Developer ID** 签名是必要非充分,真正最后一跳=公证。`build-app.sh` 已加可选公证步(先签驱动→`notarytool submit --wait+stapler staple` 驱动→最后签 app):提供凭据即一条命令打通——`LINGSHU_SIGN_IDENTITY="Developer ID Application: Yang Zhao (KM7N84AC9Y)" LINGSHU_NOTARY_PROFILE=<profile> bash Scripts/build-app.sh debug(或 LINGSHU_APPLE_ID+LINGSHU_APPLE_TEAM_ID+LINGSHU_APPLE_APP_PW)`。需用户提供的新组件=公证凭据(`xcrun notarytool store-credentials` 存一个 profile,要 Apple ID + app 专用密码 或 App Store Connect API key)。最新续(2026-06-15,公证已打通但仍非真因):凭据已配(profile `lingshu-notary`),驱动实测 公证 **Accepted + stapled + installed + Info.plist** 补全标准键(**SupportedPlatforms/InfoDictionaryVersion/MinimumSystem**),**coreaudiod** 重启——设备仍不出现、**coreaudiod** 零日志,而同目录 **BlackHole** 正常加载。结论再修正:签名/公证/**Info.plist** 三道门全部解决且验证通过,**device** 不现的真因在更深处=驱动 **AudioServerPlugIn** 实现层(**LingShuAudioDriver.c** 的属性模型/设备发布),只有过了前面所有基础设施门才暴露。下一步=对照 **BlackHole** 源码 / **Apple NullAudio** 样例 + 开 **coreaudiod** 调试日志,逐项核驱动实现。一次性硬门(证书/公证/签名)已永久解决,剩下是纯驱动代码调试。进阶:**DriverKit .dext** 系统扩展(免密码框,需 **DriverKit entitlement**)。④ S4 自主运行"自己演示 PPT" 待做。用户拍板:用自建签名 **HAL** 驱动(非 **BlackHole**),Apple 账号已开能力。

- 现场提问注入「当前脑回路」=「边讲边答」的接线(2026-06-15):架构定调——汇报主持人就是大脑本身(主线程/自主循环),不另设独立组件(否则就是往模型综合评判里加装饰器)。缺口只是接线:会议 ASR 收口后原本写死 `submitTextInput(.meeting)` → 主会话,自主演示时大脑在自主会话里、听不到;且 `tick()` 见 `hasActiveModelCall` 就丢弃。现修:
`MeetingConversationController.tick()` 在 `autonomousRun.phase==.running` 时不再短路丢弃,收口的发言改调 `state.injectMeetingQuestion(utterance)` ([LingShuState+MeetingConversation.swift](#))——注入正在跑的那条会话(自主优先,否则主),复用会话 `injectCorrection` 的步骤边界注入机制,但语义是「现场提问」([现场提问] ..., 先口头答再接着汇报),不做"纠正"措辞、不进 **dreaming** 设计反馈、不双投。于是同一个大脑边讲边答。入会操作不专门编排:靠通用「看」(`screen_capture/视觉`)+「做」(`click/type/scroll`)两个四肢操作任意会议软件,不写 per-app 适配。剩余打通缺口:① 虚拟麦设备出现 (`StreamConfiguration` 修复待验)+ 自动触发扩到自主演示开始时;② 端到端从未上机跑通(+音频起播修复未听感验)。
- (已完成)自进化 **skill · Phase 2(dreaming 对齐)**+ 热加载:空闲离线回放已完成任务→蒸馏纯提示 skill 落盘([LingShuDreamingConsolidator](#) + [LingShuState+Dreaming](#)),实测通过率作质量分。**LingShuCompositeExpertRegistry** 改 class + `reloadUserSkills()`,固化后免重启即时生效。红线:只落纯提示(`sanitizePromptOnly` 结构性剥代码/脚本),绝不自动执行未审来源代码。仍待:质量分回流到策展库。

- (已完成)自进化 **skill** · 联网自发现自安装(**discover_skill**):补齐"自主找最好 skill 自动装"老需求。本地无匹配→联网找候选→分类(纯提示/静态门拦/需风险审)→带脚本走静态门 + LLM 风险审→安全装 + 热加载;高风险脚本隔离、首次运行强制审批(详见 §3 不变量 + [LingShuSkillAcquisition](#) / [LingShuState+SkillDiscovery](#))。现实限制:DuckDuckGo HTML 被限流 → 联网发现命中率低,机制完备但实战常空(同 `acquire_resource`);换付费搜索 API 可显著改善(只动 `webSearchLinks`)。仍待:可信 `registry` 源 + `star`/采用真实信号排序(现仅 URL 启发式)。
- (已完成)自主运行引擎:授权后不再只翻状态——
`authorizeAutonomousRun` → `launchAutonomousExecution` 用统一 `agent` 循环驱动目标(`driveAgentDelivery` + 全工具集 + 独立 `verifier` + `orchestrator`)。权限级映射执行策略(`LingShuAgentExecutionPolicy`):观察=只读 / 代理=标准(`shell` 经审批门,无人值守安全拒) / 完整授权=直接放行。关键词模板 `runbook` 降为建议性上下文。暂停/停止取消在飞 `Task`; `ask_user` 卡住→下条输入回填续跑(`handleAutonomousAnswerIfNeeded`)。默认形态=常驻灵枢(2026-06-16 推翻"目标必填":上岗即"人",对话/语音驱动;目标驱动仅保留给对话命令路径)、默认完整授权=完整电脑控制(只读免审批/写直放/系统敏感强制审批),详见 §3 不变量。
- (模块 1+2 已完成,真机验证)自主模式=完全接管 + 周期感知:① 模块 1 完全接管态遮罩 (`LingShuTakeoverOverlay`,边缘辉光+悬浮控制条+一键停止夺回,不硬屏蔽)——实测上岗即出现、停止即清。② 模块 2 周期感知循环(`LingShuPerceptionCadence` 纯逻辑+12 单测、`LingShuState+AutonomousPerception` 接线)——实测灵枢全程后台仍持续感知(专用驱动 `Task`+抑制 `App Nap`)、缩图后 VL 返回真实屏幕语义、`digest` 填充并按 45s 强制刷新。③ 上岗即开麦收听(`startStandingVoiceListening`=极简模式同一套 `LingShuVoiceCallController`:麦→ASR→在岗会话→思考→TTS;实测上岗 `voiceListening=true`、"麦克风已接入",停止即关)。MCP `lingshu_autonomous` 可脚本化驱动(`lingshu_status` 含 `standingPersonOnDuty/voiceListening/perceptionDigest`)。待办:模块 3 会议纪要(滚动转写累积+结构化纪要落盘,扩展现有 ASR);④ 模块 4 通用视觉操作可靠化(看屏+AX 定位+点击/输入的失败自查/重试/接管兜底,GUI 自动化脆=靠迭代)。另:`screen_capture` 工具(给大脑看屏)也已加缩图兜底(体积分级原图/1600 JPEG + 失败重试,`analyzeScreenForControl`,见上「周期感知 VL 必须缩图」条);自主反应唤醒目前 `opt-in` 且只认音频起音(屏幕事件唤醒未做)。
- (已完成)旧启发式管线物理删除:`submitTextInput` 已收敛为唯一出口 `runMainAgentTurn`(模型即编排者),不再有 `agentBackbonePrimary` 开关。已物理删除:启发式路由(`requestRoutedCodexReply/requestAPIGatewayRouteReply/requestBackground*/reconcileRoute/ensure*/分类器`)、协同管线 (`LingShuState+Collaboration` 仅留 `fireScheduledTriggersIfDue`)、直答快路(`LingShuState+DirectChat`)、本地直答(`LingShuState+LocalAnswers`)、意图澄清卡 + 任务续接卡(`LingShuState+TaskResume`)、`pipeline*` 调用 (`LingShuState+PipelineExecution` 仅留共享 `runAgenticTool/unwrapMCPArguments`)、伪 `demo` `runEngineeringValidationSuite`。 `LingShuState.swift` 3495 → 1047 行。续接/澄清/并发统一由 `agent` 循环(持久记忆 `seed` + `ask_user` + 串行)处理。脚本化 `demo(startDemoMission/runEngineeringValidationSuite)` 及其菜单/通知也已删(伪 AI 不留)。残留待清:任务段队列 (`markTaskSegmentFinished/startNextQueuedTaskIfAvailable` 现对空队列

空转)、forcedThreadID 形参(已忽略)。

- (已完成)差距 4/7 可替换 **harness** 模块:按"协议+多实现+单一切换点"落地两个可无损替换的模块——工具调度 [LingShuToolDispatch.swift](#)(LingShuToolDispatching:串行/并行 **TaskGroup**,生产默认并行降延迟、结果恒同序;差距 7-A)+ 历史压缩 [LingShuHistoryCompaction.swift](#)(LingShuHistoryCompacting:经典条数兜底/token 预算分层 + 蒸馏事实入知识图谱近无损召回,生产默认 token 分层;差距 4 超越)。核心循环 LingShuAgentSession 加注入属性 toolDispatcher/historyCompactor/factSink(缺省=经典零变更);切换点 makeAgentSession → harnessModules 按持久化开关 (lingshu.toolDispatch/lingshu.historyCompaction/lingshu.contextTokenBudget)选实现,可一键无损切回。循环不变量 **I6** 升级为按压缩契约校验 (LingShuCompactionBudget .messageCount/.tokens;token 模式查"除单条最大消息外 body token≤预算×2"——架网 fuzz 实战逼出的契约错配修复)。验收: ToolDispatchTests+HistoryCompactionTests+LoopInvariantTests(token 模式)+AgentLoopFuzzTests 300 种子并行+分层混沌零违反;全量 833 测试 0 失败。
- (已完成)差距 5 MCP 可替换 **transport**:把传输层抽成 LingShuMCPTransport 协议 (+LingShuMCPTransportFactory 按 config 选)[LingShuMCPTransport.swift](#)—— **stdio**(子进程,原行为)+ **HTTP/SSE**(LingShuMCPHTTPTransport:Streamable HTTP 逐帧 POST、application/json 与 text/event-stream/SSE 响应解析、Mcp-Session-Id 会话头)=把"吃现成生态"从本机扩到远程/托管 **MCP server**(跨厂商共享的同一批)。client 改名 LingShuMCPClient、注入 transport(只管 JSON-RPC 语义);config 加 transport/url 且向后兼容(旧 servers.json 无字段→解码为 .stdio);registry 加 addHTTPServer。验收:MCPTransportTests(SSE/JSON 压实/帧拆分纯解析 + 配置兼容 + 注入假 transport 测 client 解析);全量 850 测试 0 失败。真假分级:HTTP 纯解析核已单测✅;对真远程 **MCP server** 的 **live E2E** 待一个真 **server** 验⚠️;持久双工连接(现每次调用一发一收,含 initialize)=后续 transport 层优化、契约不变。
- (已完成)差距 7-B 工具目录延迟加载(按脑力自适应):可替换模块 [LingShuToolCatalog.swift](#)——全部工具 **handler** 始终注册可执行,只把给模型的 schema 收成「通用高频核心集(12 原语/高频)+ search_tools 元工具」;模型要核心外能力先 search_tools(query) → 相关性排序返回用法并激活(引用型 LingShuExposedToolSet 锁保护,核心循环每回合算活跃集→下回合即见,动态扩张)。核心循环只多"算 activeTools 一行"+ 注入 exposedToolNames(nil=全暴露零变更)。按脑力 **gate**(用户拍板):切换点 toolCatalog(for:) 读 lingshu.toolCatalog(adaptive 默认)——仅 **lean** 档(强脑)延迟加载、balanced/guided(如当前 DeepSeek)走全量=日常零回归;eager/deferred 可强制。验收:ToolCatalogTests(分词/排序/暴露集/build/search 激活 + 端到端真 **session**:激活前模型看不到长尾、search_tools 后看得到且可执行、循环恒良构);全量 858 测试 0 失败。
- (完全版骨架 #2,2026-06-22)统一自适应 **harness** 配置:抽 [LingShuHarnessConfig.swift](#)(Sendable,封脑力自适应:并行调度/token 分层压缩/factSink/按脑力延迟加载;makeSession=唯一"建带完整自适应 harness 经典会话"入口)。makeAgentSession 的 **classic** 分支与 **nested** 内层(ensureSpine/makeInner)共用同一份 **config**(经 currentHarnessConfig() 一处读开关+脑力档)→ 修掉 **nested** 直接 **new** 裸会话、绕过 **harness** 的双底盘不一致(此前确认的真问题);旧 harnessModules/toolCatalog 并入 config 删除。

LingShuNestedAgentSession 加 harness: 注入(nil=回退裸会话兼容)。验收: NestedHarnessTests(config 纯方法 + nested spine 实锤吃到延迟加载、长尾被延迟、无 harness 回退)+ 全量 905 测试 0 失败。这是"完全版灵枢"长线工程的地基(后续都按"锁接缝+现状做默认实现"叠加,详见记忆 [harness-parity-roadmap](#))。

- (完全版接缝全铺,2026-06-22) 除 #2 外:#3 [LingShuArtifactVerifier](#)(按类型确定性验收注册表,已接 nested 阶段验收)· #5 [LingShuPermissionMatrix](#)(资源域×风险×模式→裁决,红线硬编码)· #1 [LingShuModelChannel](#)(续接策略,改写上下文强制降级 prefixStable)· #4 [LingShuMemoryFacade](#)(多源召回合并门面)· #6 [LingShuCapabilityProvider](#)(能力枚举注册表)。协议+默认实现就绪。⚠接线状态校正(独立验收 2026-06-22 实测):#1/#2/#3/#8 当时已真接进运行路径;#4/#5/#6 当时仅协议+单测、运行路径未接(详见下条「地基夯实」已补接)。
- (地基夯实·接线补齐 + 鲁棒性修复,2026-06-22 独立验收后) 三项落地,全量 930 测试 0 失败:
 - #4 接进运行路径:recall_memory 工具改多源召回——知识图谱(主 actor)+ 本机文件索引(后台 detached)→ LingShuMemoryMerge 归一去重排序(拆到 [LingShuState+Recall.swift](#) 守 ≤500)。
 - #6 接进运行路径:新增只读工具 list_capabilities([LingShuState+Capabilities.swift](#))——LingShuStaticCapabilityProvider 包 MCP 工具/固化技能 → LingShuCapabilityRegistry.merge 汇总枚举给大脑(挂进 agentBuiltinTools)。
 - #5 接成 SafetyKernel:requestShellApproval 的非 quarantine 裁决收口到 LingShuPermissionMatrix.decide(纯函数 shellApprovalDecision,dev-full 保留为显式 override;红线由矩阵硬保证)。真值表等价守卫 ShellApprovalKernelTests(逐项==改造前,零行为回归)。
 - 打断恢复孤儿 tool_call 不变量泄漏修复(真凶):LingShuAgentSession 续接入口 (send/resume/continueLoop)repairOrphanToolCalls()——飞行中取消留下的未应答 tool_call 在不变量检查前补齐合成结果(与网关 sanitizeToolCallSequence 同口径、发生在会话层)。确定性真阳性回归 BatchInterruptLeakTests.testOrphanToolCallRepairedOnResumeNoInvariantLeak(关掉修复即红:未应答 tool_call)。修前 live soak 打断恢复 0→3→8 违反 + 级联误报自主段;soak 脚本同步改滚动检查点(每段判增量、不级联,见 Scripts/soak-e2e.sh)。详见 Docs/回归_打断恢复不变量泄漏 _7a6744f.md、记忆 [verify-gate-bypass-batchinterrupt-leak](#)。
- #1 真接进 live 适配器:LingShuGatewayAgentModel 据 LingShuModelChannelStrategy 决定续接(网关响应带 continuationToken=原生续接 id;native 通道+干净追加才链,改写上下文降级 prefixStable,DeepSeek 等无状态恒 nil 零变更)。.app 多轮"记 4271 → 续接答对"无回归。
- #8 编辑审查闭环(模型核+UI):[LingShuWorkspaceReview](#)(unified diff → 文件+hunk、逐块接受、组装补丁、汇总)+ UI [LingShuWorkspaceReviewView](#)(文件树+逐 hunk 接受/拒绝+回退未接受;接 LingShuWorkspacePanel 审查 tab;git 后端 [LingShuWorkspaceReviewGit](#) 在 Services 层=视图不直接碰 Process)。WorkspaceReviewGitTests 对真 git 仓库回退往返通过。

- #5 权限矩阵 UI:[LingShuPermissionMatrixView](#)(资源域×运行模式→裁决只读网格,接设置「系统配置」tab)。
- #7=现有执行记录 / #9 soak 已有。验收:ArtifactVerifier/PermissionMatrix/CompleteVersionSeam/WorkspaceReview Tests + 全量 925 测试 0 失败;app 简单问答 2.1s、多轮续接 2.0s 无回归。接口清单:66 agent 工具 + 31 MCP 控制接口 + 19 对外具身工具。
- (已完成·超越点)灵枢当 MCP server:既有 8917 MCP server(LingShuControlServer+LingShuControlRouter)的 tools/list/tools/call 现额外暴露灵枢独有的具身能力给外部 agent(CC/Codex/Cursor)反向调用 = 给只有终端的 agent 装身体。可替换模块 [LingShuEmbodimentManifest.swift](#)(通用具身名集:计算机控制 8+浏览器 8+语音+外设 2;filter/descriptors/tool(named:) 纯逻辑)+ [LingShuState+Embodiment.swift](#)embodimentCandidateTools()(从各能力源直接装配——注意具身工具不在 agentBuiltinTools 基集、是 mainAgentSession/spawn 调用方追加的)+ router tools/list 追加描述符、callTool default 转发到真 handler。安全:各 handler 系统授权门照旧、8917 仅本机可达且本就有 lingshu_send_prompt 全能力入口(不新增攻击面);开关 lingshu.exposeEmbodiment(默认开)。真假分级:🟢.app live 实锤——重建 .app、curl 8917 tools/list 见 19 个 [灵枢具身] 工具(总 50)、tools/call browser_open 真开了网页标签页、peripherals 真返回;EmbodimentManifestTests 4 例 + 全量 862 测试 0 失败。
- (已完成)差距 5 余项·HTTP live-E2E + 持久连接:transport 改 actor + 握手归 transport(client 只发业务帧,initialize 不再每次自带);持久连接——stdio 进程常驻(LingShuStdioSession 行缓冲读+按 id 取响应)、HTTP Mcp-Session-Id 复用,initialize 跨多次工具调用只握手一次;自愈有界(任何超时/失败即拆连重连,绝不 wedge);开关 lingshu.mcpPersistent(默认开)可切回每次新连。live E2E 实锤:MCPLiveHTTPTests(NWListener 起真 HTTP MCP server → 真 socket 往返 + SSE 解析 + 持久 initCount=1/非持久=+2)、MCPLiveStdioTests(真 python 子进程 mock → 持久进程复用=1/非持久=+2);全量 864 测试 0 失败。详见 Docs/对标前沿 Agent_harness 追平方案.md。
- (已完成·第一刀)本机知识中枢——文件/文档/代码本地索引:灵枢学习/蒸馏本机知识,"找东西/按本机知识答题"更准。隐私模型(用户拍板):本地检索 + 云端脑只发确认片段——索引哪些目录由用户 opt-in(lingshu.localKnowledgeFolders),向量本地算(复用 LingShuLocalTextEmbedding/NLEmbedding,零网络/零上传),只有检索到的片段才作为工具结果进上下文。模块:LingShuTextChunker(纯逻辑分块)+ [LingShuFileKnowledgeIndex](#)(逐块向量+关键词兜底混合检索、按文件折叠、JSON 持久、增量按 mtime)+ [LingShuFileKnowledgeIndexer](#)(遍历 opt-in 目录、扩展名/大小过滤、跳噪声目录、增量、删除按 fileExists 剪枝)+ 四肢 [LingShuState+LocalKnowledge](#)recall_local/index_local_knowledge(挂进 agentBuiltinTools 末尾,全会话共享)。验收:LocalKnowledgeTests(分块/检索/删除/增量/持久化 11 例)+ .app live 实锤(索引 ~/lingshu-kb-test → 驱动 agent recall_local 答出只存于本机文件的代号 ZephyrFox-7794,轨迹含两个工具调用);全量 875 测试 0 失败。③+①+② 已落地(2026-06-21):③ 常驻全局入口 + FSEvents 自动增量—— [LingShuGlobalHotKey](#)(Carbon \Space 免授权全局热键)+ [LingShuQuickAsk](#)(浮动 NSPanel "问/找/做",复用 submitTextWithAttachments)+ MenuBarExtra「快速提问」+

- [LingShuFolderWatcher](#)(FSEventStream 监听 opt-in 目录、去抖 2s 增量重索引,加目录时 restart);① 多源——[LingShuDocumentText](#)(PDF 经 PDFKit 抽取,索引器接入)+[LingShuBrowserHistorySource](#)(sqlite3 读 Safari/Chrome 历史、epoch 转换、复制到临时只读副本避锁)+ 工具 `index_browser_history`(合成源 `path=history://...`,文件遍历剪枝只剪绝对路径不误删合成源);② **dreaming** 蒸馏——索引加召回热度统计(`recordHits/topRecalled/clearHits`,独立 hits 文件)+ [LingShuLocalKnowledgeDistiller](#)(高频命中内容→模型提炼离散事实→remember 进知识图谱→清计数,注入 `distill/remember` 可测)+ **dreaming** 钩子 `distillFrequentLocalKnowledge`。验收:`LocalSourcesTests`(7,含真 CG 生成 PDF 抽取)+`LocalKnowledgeDistillTests`(5)+ **.app live** 实锤 **FSEvents** 自动增量(偷偷加新文件未手动索引→8s 后 `recall_local` 找到);全量 887 测试 0 失败。
- (定调·用户拍板 **2026-06-22**)本机知识 = 灵枢【基础能力】,不加任何开关(不可启停);访问=对话直接用(本体主窗 + 自主模式都走 `agentBuiltinTools` → 已含全部本机知识工具;主+自主系统提示都加了"本机知识是基础能力、涉及本机文件/资料/历史先 `recall_local`"的引导,大脑对话中主动用)。我曾建的独立浮窗面板被否("不是我需要的")→ 对话为主、面板降为可选富交互入口。
 - (已完成·多源+面板富交互 **2026-06-22**) ① 日历源 [LingShuCalendarSource](#)(EventKit 取近窗口事件,工具 `index_calendar,path=calendar://`)② 邮件源 [LingShuMailSource](#)(遍历 `~/Library/Mail` 的 .emlx、纯逻辑 `parseEMLX` 抽主题/发件人/正文+剥 HTML,工具 `index_mail`,需完全磁盘访问,`path=mail://`)③ 照片源 [LingShuPhotoSource](#)(本机 **Vision OCR**+场景分类生成字幕,全 on-device、照片绝不上云=守 [perception-data-zero-retention](#) 零留存,工具 `index_photos,path=真实图片路径`)④ 快速面板富交互 [LingShuQuickAskLinks](#)(纯逻辑从回复抽可点开项:存在的文件/网址/`history://`内层 URL)+ 面板渲染可点击行(文件→默认 app 打开、网址→浏览器)。验收:`LocalSourcesExtraTests`(7,含照片本机 **OCR** 真识别)+ **.app live** 实锤(对话里成功触发 `index_calendar`);全量 894 测试 0 失败。
 - (已完成·通用源增量化 + 面板直接执行 **2026-06-22**,铁律"通用零定制、知识模块内"):通用增量入库管线 [LingShuKnowledgeIngest](#)(Item/Scan/ingest)——所有源都 `scan(knownMtime: )→ingest(owns:stillExists: )` 同一条路:`mtime` 跳未变(邮件不重解析/照片不重 **OCR**)+ 按 `owns` 只剪本源不误删别源(`file=非图片"/"`、`photo=图片"/"`、`calendar://`、`mail://`、`history://`;文件/照片 `stillExists=fileExists` 防误删)。各源加 `scan`、删掉各自 `collect+upsert` 重复;`FileKnowledgeIndexer.reindex` 改为 `scan+ingest` 薄封装。面板 [LingShuQuickAsk](#) 每结果行=打开/访达显示/复制。验证逼出并修真排序 **bug**:大索引(1440 条)下唯一 token 查询被向量噪声淹没→`LingShuFileKnowledgeIndex.search` 加整串精确匹配 +1.0、关键词 0.3/词(守卫测试:50 噪声中目标第一)。验收:`LocalKnowledgeIngestTests`(5)+ `ranking` 守卫 + 全场景 **live** 验证(`Scripts/lk-scenario-validate.py`:文件/照片 OCR/日历真 37 条/历史/邮件/FSEvents/跨源检索通,增量 **live** 实锤"第二次新建 0")+ 全量 901 测试 0 失败。注:S3 自然对话触发 `recall` 偶发不稳=模型工具调用变异([deepseek-agentic-ceiling](#))非机制 **bug**。
 - (通用 **AI** 中枢·总纲 + **P1a** 落地,**2026-06-22**) 方案见 `Docs/通用 AI 中枢推进方案.md`(对标 15 条要求的差距评分 + **P1-P6** 路线:GoalSpec → 能力图谱/缺口分析→通用验收→经验闭环→强弱脑→自我进化,全建在现有内核+工具上、领域只进数据)。诊断:下半身(执行/具身/外围自生长)壮、上半身(认知控制平面)缺。**P1** 完全落地(目标认知,**2026-06-22**):[LingShuGoalSpec](#)(纯类型 `objective/kind/constraints/boundaries/risks/success_criteria/open_questions` + `LingShuGoalSpecParser` 容错解析:剥围栏/取首个{}/缺字段兜空/无 `objective` → `nil`;+ `executionGuidance/acceptanceCriteriaBlock` 消费助手)+[LingShuState+GoalSpec](#)(`deriveGoalSpec` 模型 1-shot、`bindGoalSpec` 绑定记录,开关

lingshu.goalSpec DEBUG 默认开)。入口 **submitTextInput** 分诊之前派生(与 **classifyDispatch** 并发不加 wall-clock 延迟;真唯一入口,非 **runMainAgentTurn**——task 走 **dispatchIsolatedTask** 不经它),存 **goalSpecsByRecord**[记录]。三处消费:① **driveAgentDelivery** 注入执行引导(主/自主/派发都过)② **verifyAgentDeliverable** 注入成功标准(完整性维度据此核对)③ 任务记录落痕(记忆)。验收:**GoalSpecTests**(12:解析真值 + **executionGuidance/acceptanceCriteriaBlock** 组合 + 记录 **Codable** 持久化/向后兼容)+ **.app live** 实锤(chat 型:主会话上下文实见注入「本次目标…恰好三句话」;task 型:记录绑定 **GoalSpec**+抓到"不改目录外文件"边界、真出 **add.py** 已完成);全量 943 测试 0 失败。

- **(P1 真闭环补全,2026-06-22)** 三处:① 持久化——**GoalSpec** 改为 **LingShuTaskExecutionRecord.goalSpec** typed 字段(**Codable** 跨重启,decodeIfPresent 向后兼容;弃内存字典),单一真相经 **goalSpec(for:)** 读;实锤 **UserDefaults.blob.lingshu.task-execution.records** 里记录带完整 typed **goalSpec**。② 长期记忆结构化沉淀——**finishTaskRecord** 终态(完成/直答/未达标)调 **rememberGoalExperienceIfNeeded**:把「目标→成功标准→结果→产出/未达标小结」蒸成可检索经验入知识图谱(陈述句、园丁去重);实锤 **trace**「经验沉淀」+ **recall_memory** 检出历史任务(目标→…→经验闭环)。③ 全入口覆盖 + 配置入口——发布态默认开(去掉 **DEBUG/Release** 分叉)、**setGoalSpecEnabled** + **MCP lingshu_set_goalSpec** + **lingshu_status.goalSpecEnabled**;覆盖策略(口径=离散新目标派生、续接/输入复用或不派生):派生=① 主路径 **submitTextInput** task/chat ② 自主真实目标 **launchAutonomousExecution** ③ 模型 **spawn** 的子任务(**spawn_task/spawn_team/agent_demo** 经 **handleOrchestratorEvent** .spawned 新建记录那支——主输入预映射的不重派生;子任务 **objective** 即清晰目标,执行引导①由 **objective** 等价承担,**GoalSpec** 供验收成功标准②+经验③)。复用/不派生=续接/追问/在岗逐句对话(读 **record.goalSpec**)、后台守候/会议/感知事件(执行中输入,非新目标)。实锤:**agent_demo** 两子会话经 **orchestrator.spawn** → 各自 **objective** 派生 **GoalSpec** 绑定记录。
- **(P2 能力缺口分析·前置落地,2026-06-22)** 执行前判"以当前能力(含可立即自我扩展补齐的)能不能做成、缺什么、怎么补",让"超能力目标不拒绝、能自补的规划自补、真需外部的如实告知"成真。模块:[LingShuGapAnalysis](#)(纯类型 **LingShuGapKind** 八类/**CapabilityGap**{kind,missing,fillPath,blocking}/**GapAnalysis**{feasibleNow,gaps,note} + **LingShuGapAnalyzer** 容错解析 + 嵌能力快照的评估提示,提示固定带 **author_component/discover_skill/acquire_resource/discover_devices**/连 **MCP** 自我扩展元能力)+ [LingShuState+Capabilities.capabilitySnapshot](#)(核心原语+具身+知识+编排+已连 **MCP**/固化技能)+ [LingShuState+GapAnalysis](#)(**deriveGapAnalysis** 模型 1-shot、**bindGapAnalysis** 绑记录、**gapAnalysis(for:)** 读)。typed 字段 **LingShuTaskExecutionRecord.gapAnalysis** 持久化(同 **goalSpec**)。入口:**submitTextInput** 仅 **task** 型目标(triage 后)派生→绑记录;消费:**driveAgentDelivery** 在 **GoalSpec** 引导之上叠加缺口+补齐计划(**executionGuidance**,无缺口不加压)。共用开关 **goalSpecEnabled**。验收:**GapAnalysisTests**(9)+ **GoalSpecTests**(13)+ **.app live** 实锤(「Spotify 加歌」→ **feasibleNow=false** + 2 缺口:tool → **author_component** 自写、**permission** → 需用户授权,typed 持久化进记录);全量 953 测试 0 失败。

- **P2 补齐(同日):**① 覆盖对齐——统一前置认知 **bindPreflightCognition**(并发派生 **GoalSpec** + 缺口分析并绑记录,**GoalSpec** 失败给最小兜底)接进自主真实目标

(launchAutonomousExecution)+ **spawn_task** 子任务(派生在 spawnDetached 前,无竞态),与主输入 task 路径同覆盖。② 主动澄清——
LingShuGoalSpec.executionGuidance 有 open_questions 时指示"执行前先 ask_user 问清";LingShuGapAnalysis.executionGuidance 有需用户提供的阻断前提(needsUserToUnblock:humanConfirmation/permission/funding/device)时指示"先 ask_user 确认拿到,真拿不到给替代不假装完成"。live 端到端实锤:Spotify 任务带阻断 permission 缺口→真 ■ 卡住主动问用户(账号/歌单/凭据/或打开 Web 版你登录);Notion 缺能力→如实"无法接入+给替代",同目标里"找最大文件"(有能力)照常做完。诚实保留:子任务/自主覆盖为代码层(共用 helper),DeepSeek 实测常不真 spawn 故无 spawn live demo;缺口→确定性"补齐→验证→执行"编排仍是模型驱动(注入计划 + 自我扩展工具),未做确定性编排闭环,归后续(与 P6 自我扩展执行重叠)。

- (P3 全类型验收·落地,2026-06-22) 把 GoalSpec 成功标准分类型逐条验收,凡能被宿主侧事实确定性核验的(文件存在 / 命令·测试成功)用文件系统 + 执行记录证据裁决——不靠模型自说自话;不能的(设备/环境/用户确认)如实标 unverifiable(绝不幻觉为已达成);主观的(内容质量)交 LLM 评审官。任一确定性条 unmet → 硬门返工,模型口头"已完成"不能推翻文件系统事实。这把 #11 验收从"动作/文档二元 + 代码确定性门"推进到"按成功标准全类型确定性验收"。模块:
[LingShuAcceptance](#)(纯类型 LingShuCriterionKind 六类[fileExists/commandSucceeds/deviceEffect/environmentChange/userConfirmation/contentQuality]+isDeterministic、AcceptanceCheck{kind,criterion,probe}、CheckStatus[met/unmet/unverifiable]、AcceptanceReport{verdicts,note}+make 纯裁决 /verifierBlock/deterministicFailureReason、LingShuAcceptancePlanner 分类提示+容错解析[解析失败回退 content_quality 不丢条目])+ [LingShuState+Acceptance](#)(acceptanceReport 惰性分类[缓存到记录]+宿主事实裁决、deriveAcceptanceChecks 模型 1-shot、acceptanceFileExists[绝对/相对工作目录/realFiles basename/*.ext]、commandProbeOutcome[扫记录 run_command 配对,成功/失败/未出现三态])。typed 字段
record.acceptanceChecks+record.acceptanceReport 持久化(同 goalSpec/gapAnalysis)。消费:verifyAgentDeliverable 在 realFiles 后跑确定性核验→hasDeterministicFailure 直接返工、否则 verifierBlock 注入评审官逐条核对(unverifiable 如实标不假定达成);共用开关 goalSpecEnabled。验收:
AcceptanceTests(12:分类解析容错/三态裁决/非确定性类型恒 unverifiable 不幻觉/硬门/持久化)+全量 967 测试 0 失败 + .app live 实锤(「斐波那契脚本+保存 fib.txt+中文注释」→三条成功标准被分类成 commandSucceeds[probe=python fib.py]/fileExists[/tmp/p3demo/fib.txt]/contentQuality;裁决:fileExists=**met**[文件系统已核实]、commandSucceeds=**unverifiable**[记录里没匹配该探针命令,未幻觉为达成]、contentQuality=**unverifiable**[交评审官];验收官评审意见真引用 commandSucceeds 逐条核对;acceptanceChecks+report 真持久化进记录读得出)。诚实保留:① commandSucceeds 探针为子串匹配(模型给 "python fib.py" 但实际跑 "python3 /tmp/.../fib.py" 则不匹配→降级 unverifiable[安全,不误判 met])+ 既有可见运行门继续把关,探针更鲁棒匹配是后续优化;② 验收分类是 1-shot 模型调用(首次验收时惰性触发、缓存复用),关 goalSpecEnabled 即零成本跳过;③ "确定性补齐→再执行"编排仍归 P6。
- (P2 真闭环·防伪完成 + 能力获取闭环,2026-06-22) 修根因 bug:复合任务一部分没做成却报「完成」、把「我没能力」当最终答案。根因:runVerificationLoop 只有在有产物/动作时才

验收→缺口型任务整段跳过验收,finalizeMainTurn/编排器 .completed 硬编码
.answered/.completed;gapAnalysis 只作 guidance 文本不参与执行流/验收流/状态
(spec 第 13 条禁项);P2 把"解决问题(补齐能力)"误推给 P6。修复:① 防伪完成闸
LingShuTaskCompletionGate(纯决策:未解除阻断缺口/承认无能力[通用承认语,非领域]/部分达成→禁当完成;可自补未试→needsAcquisition 驱动获取;需用户→waitingForUser;部分→partial;仍卡→blocked)+ LingShuCapabilityAcquisition(获取结果五态分类,最小验证未过≠acquiredVerified)→ 跑在 verifyAndContinue 末尾(主/派发/自主共用,无产物也跑=根因修复)LingShuState+CompletionGate。② 能力获取闭环:可自补阻断缺口未试→runCompletionGate 有界 resume 驱动模型真去获取
(连 MCP/discover_skill/author_component/浏览器)+ 最小验证(获取成功后新能力被真实动作工具调通)→ 写 LingShuCapabilityGraph(有状态:在线/权限/已验证;只有最小验证过才 usable 可复用)LingShuState+CapabilityGraph,已获取并验证的进能力快照供复用。③ 能力需求 LingShuCapabilityRequirement(通用动词 external_system.read/write、local_file.scan、document.generate、browser.operate、device.*、api.call、human.confirm...从 GoalSpec 推导、查图谱)。④ 状态语义:LingShuTaskExecutionStatus 新增 analyzing/acquiringCapability/waitingForUser/ready/partial/verified/failed(影响 finishTaskRecord 收尾/汇总/记忆/续接,非 UI 装饰);typed 字段 record.
{capabilityRequirements, acquisitionAttempts, taskOutcome} 持久化。⑤ 终态由完成闸定:finalizeMainTurn + 编排器 .completed/.failed 读 taskOutcome → finishStatus 映射(派发即便撞顶/停滞被判 failed,若闸早判 waitingForUser/partial 以闸为准、给「需要你…」诚实收尾并指向便于续接)。⑥ 记忆+复用:rememberGoalExperienceIfNeeded 沉淀缺口/补齐路径/成败;续接优先恢复 blocked/partial/waitingForUser(pickResumeTarget + dispatched 指向)。验收:TaskCompletionGateTests(spec 第 14 条:缺能力进获取/需授权 waitingForUser/复合 partial/承认无能力禁 verified/获取分类三态)+CapabilityGraphTests(需求解析/图谱匹配/最小验证写入门/动态注册复用/续接恢复)+全量 987 测试 0 失败 + .app live 实锤(「同步待办到 Notion」→ 派生 capabilityRequirements[external_system.write+human.confirm]、gapAnalysis 4 阻断缺口[含 permission=Notion Token]、acquisitionAttempts 记录 3 次尝试→needsUser、taskOutcome=waitingForUser、绝不再报「完成」)。诚实保留:获取路径选择是模型驱动(spec 第 11 条),代码只强制尝试+确定性裁决+写图谱;DeepSeek 实测会先 flailing 上百步才被停滞/完成闸兜住(模型 agentic 天花板 deepseek-agentic-ceiling),但结果诚实不伪完成是机制保证。追修三个 live 暴露的真 bug(2026-06-22 同日):① 给了凭据仍循环再问(根因:gapAnalysis 静态、缺口永不解除)→ LingShuCapabilityGap.resolved(typed,向后兼容 Codable)+ LingShuGapAnalysis.blockingGaps/hasBlockingGap 只算未解除;resolveUserProvidedGaps 在 submitTaskFollowup 续接 waitingForUser 任务时把需用户阻断缺口标 resolved + 清旧 taskOutcome → 据续接后真实结果重判,不再无限追问。② 内部停滞文本泄漏给用户(「连续 16 步只读取...(无输出)」)→ looksLikeInternalDump 检测,honestDeliveryText 降级时换干净话(不 append 到垃圾文本);完成闸对 .completed/.maxTurnsReached 都给干净收尾。③ ask_user 等待时状态显示「执行中」很怪 → 编排器 .blocked 把记录状态置 waitingForUser(不 finishTaskRecord、保留续接)。续接引导改指向派发会话真有的路径(enter_managed_mode 接管屏幕驱动本机应用/浏览器/author_component,非主会话独有的 computer-use)。992 测试 0 失败。

- (P4 经验闭环·落地,2026-06-22) P1 只被动沉淀经验(靠模型自己 recall_memory),P4 升级成主动复用闭环:目标终态蒸一条结构化可复用经验存持久库 + 新目标执行前据相似度召回最相关历史

- 经验、注入执行引导(复用成功打法/避开旧坑)。模块 [LingShuGoalExperience](#)(纯类型 {objective,kind,outcome,lesson,sourceRecordID})
+[LingShuGoalExperienceMatch](#):relevance[字符二元组 Jaccard,中文友好无嵌入]/mostRelevant[过阈值 0.2+仅排除同一 sourceRecordID 自身,相同历史目标仍召回]/guidanceBlock[无相关返回 base])+ [LingShuState+GoalExperience](#)(库 UserDefaults lingshu.goal.experiences 截留
近 200/recordGoalExperience/recallGoalExperiences/goalExperienceGuidance/distilledGoalLesson)。沉淀:rememberGoalExperienceIfNeeded 终态时除写图谱事实(广义 recall),再存一条结构化经验(distilledGoalLesson:成功→打法+产出、失败/部分→坑+summary、能力补齐→需用户提供 X/已补齐可复用 X,并记录 sourceRecordID 防自引用)。消费:driveAgentDelivery 在 GoalSpec/缺口引导之上叠
goalExperienceGuidance(据本回合目标召回 top2 相关经验注入;落 trace「经验复用·召回相关历史经验」)。图谱事实(广义模型自取)与结构化库(确定性主动注入)并存互补。
[GoalExperienceTests](#)(相似度/召回过滤排序/相同历史目标可召回/仅排同源自身/沉淀→召回闭环/注入/派发组装引导含 **P1** 目标+**P4** 经验回归)+全量测试绿。诚实保留:相似度是启发式(非语义嵌入)。**P4 live** 实锤过(甲完成→沉淀,乙同类→真召回甲经验注入,trace「经验复用」),并 live 暴露+修了派发任务收不到前置引导的真 bug(driveAgentDelivery 只主/自主调用,派发走 orchestrator.drive-session.send 不经它 → **P1** 目标/**P2** 缺口/**P4** 经验三层引导从未注入派发任务)→ 抽 assembledExecutionGuidance(**P1** → **P2** → **P4** 逐层)经 initialMessages 注入派发,三层引导主/自主/派发全覆盖。
- (**P5** 强/中/弱脑分层 + 降级·框架落地,2026-06-22) 派发任务前据复杂度路由到弱/中/强档脑,理想档未配→降级到可用档(对应 #9)。模块 [LingShuBrainRouter](#)(纯:
[LingShuBrainTier](#){weak/medium/strong+rank}、desiredTier[复杂度打分:kind+约束数+标准数+阻断缺口+升级计数]、resolve[据可用档取≤理想的最高/否则最低可用]、route)+ [LingShuState+BrainRouter](#)(tier → [LingShuBrainTierConfig](#) 存 lingshu.brain.tiers、setBrainTierModel、availableBrainTiers、tierModelAdapter[未配该档→回退当前单脑]、routeBrainTier 落 trace「脑分层·脑路由」、routedModelAdapter)接进 dispatchIsolatedTask。
[BrainRouterTests6](#)+全量 1015 测试 0 失败。诚实约束:真三档要用用户配多个脑端点;现单脑(DeepSeek)→ 路由照算+落 trace 但都回退当前脑(框架在、分层空),配多脑即生效。主会话(持久 holder)未按回合重路由(chat 轻);mid-task 失败升档(escalationCount)信号已留,中途重建会话升级编排归后续。
 - (**P6** 自我进化闭环·有界落地,2026-06-22,用户选「有界自进化+人批采纳」) 挖 **P4** 经验库反复失败弱点 → 生成有界改进提案(自写工具/装技能/接连接器/调提示,绝不自改核心 **Swift**) → 默认 pending 不自动采纳 → 人批准才把提案变成真任务去建(走既有 task 派发 → author_component/discover_skill/连接器,沙箱+安全门+完成闸),可禁用回滚。模块 [LingShuSelfImprovement](#)(纯:[LingShuSelfImprovementMiner.detectPatterns](#) [非成功经验按目标相似度贪心聚类,簇内 ≥2=反复弱点]、suggestion 模板有界建议、[LingShuImprovementProposal](#){theme,occurrences,suggestion,status:pending/approved/rejected})+ [LingShuState+SelfImprovement](#)(提案存 lingshu.self.improvements、mineSelfImprovementPatterns、proposeSelfImprovements[去重+不自动采纳+提示气泡]、approveSelfImprovement[→ submitTextInput 派任务建]、rejectSelfImprovement)。[SelfImprovementTests6](#)(聚类挖弱点/阈值/有界建议

提人批/提案不自动采纳+去重/否决/Codable)+全量 1021 测试 0 失败。诚实约束:① 提案触发已自动挂(2026-06-23)——autoMineSelfImprovementsOnFailure(挂 rememberGoalExperienceIfNeeded 末尾):非成功终态经验落库即挖一次反复弱点(挖掘是纯聚类无模型调用,原"乱发模型调用"顾虑不成立),失败成簇 ≥ 2 自动提待批提案(去重、不自动采纳)。② 自我进化总开关(2026-06-23,用户要求):@Published selfEvolutionEnabled 默认关闭(高风险能力),持久化 lingshu.selfEvolution, 门控全部 P6 动作 (proposeSelfImprovements/autoMineSelfImprovementsOnFailure/approveSelfImprovement 均 guard selfEvolutionEnabled);UI 开关在 [LingShuModuleVariantsPanel](#) 顶部,开启前 .alert 弹风险提示(会主动改自身行为/能力边界;但逐条人批+一键回退+安全红线不放松),关闭直接生效;setSelfEvolutionEnabled / MCP lingshu_set_self_evolution / status.selfEvolutionEnabled;live 实锤(默认 False → 开 True 持久 → 关)② 无专门 UI 面板(提案经 chat 气泡+trace 露出,approve/reject 是方法可经 MCP;UI 待补)③ 采纳的真「建」仍是模型驱动 (author_component,受 DeepSeek 天花板)④ live 待做。红线沿用 [skill-self-evolution](#):绝不自动执行未审代码。- (P6+ 无界自进化使能件+模块变体注册表,2026-06-22,用户定调:核心也可改,但必须模块化+一键切换+一键回退→安全模型由「禁止改核心」改成「可逆」) 模块 [LingShuModuleVariant](#)(纯:每槽位多变体+活跃指针+历史栈,register/switchActive [记历史]/rollback[回上一活跃或基线]/activePayload)+ [LingShuState+ModuleVariants](#)(持久化 lingshu.module.variants、ensureModuleBaseline、registerModuleVariant[默认 inactive=自进化产物先不生效、人批后切换]、switchModuleVariant 一键切、rollbackModuleVariant 一键回退、activeModulePayload)。接一个运行时可热切换槽位 guidance.execution(执行策略,追加进 assembledExecutionGuidance)证明热切换不重启:注册 inactive → 一键切 → 运行时引导立即变 → 一键回退立即回基线。ModuleVariantTests6+全量 1026 测试 0 失败。诚实约束(编译型 App 硬事实):运行时可插拔层(提示策略/技能/工具脚本/连接器)真热切换不重启;编译核心 Swift 没法运行时热替换一段编译码,但可「协议变体+特性开关」做到模块化切换/回退(改动过一次构建,切换/回退仍一键瞬时可逆)。

- (P6+ 五条留账全补齐·无界自进化全真,2026-06-23,用户定调:大脑更换暂搁置,要的不是接多脑而是据当前脑动态调整→下一方向) ① 多槽位接进注册表(LingShuModuleSlots.all):除 guidance.execution 再接 guidance.persona(行为人格片段 → mainAgentSession 系统提示尾,additive 不动身份锚)、gate.acquisitionCeiling(数值参数槽 → runCompletionGate 获取上限, acquisitionCeilingOverride 夹 1-5)、core.guidanceAssembly(编译核心组合器键)——注册表泛化到提示/系统提示/数值/编译核心四种槽位类型;启动 ensureAllModuleBaselines 确保各槽有基线。② P6 采纳 → 自动注册变体: approveSelfImprovement 批准后 registerImprovementAsVariant 把改进蒸成 inactive guidance.execution 策略变体(默认不生效,人到面板一键切),闭环=弱点 → 提案 → 批准 → 变体登记 → 人切 → 热生效 → 可回退。③ 编译核心变体(协议变体+特性开关): [LingShuGuidanceComposer](#)(协议 LingShuGuidanceComposing+两编译实现 append[默认/同历史]/prepend[策略前置,呼应"据当前脑调整"]) +LingShuGuidanceComposers.resolve payload 键 → 已编译实现); assembledExecutionGuidance 经 activeGuidanceComposer() 组合经验与策

略;切已编译变体=运行时翻 **active** 键(无构建),新增编译变体才过一次构建。④变体管理 **UI** [LingShuModuleVariantsPanel](#)(挂"系统配置→技能":列槽位→变体→活跃标记+一键切换/回退/删除[removeModuleVariant,基线与活跃不可删];@Published moduleVariantsRevision 驱动刷新)。⑤ **live** 验证:MCP `lingshu_module_variants(list/register/switch/rollback/remove)` +`moduleVariantsPayload` 暴露 **effective** 实际消费值;.app 重打包重启经 8917 实锤:注册 `inactive` → `effective` 不变 → 一键切 → `executionStrategy/guidanceComposer` 热变(不重启) → 一键回退 → 回基线 → `remove` 清测试变体回纯净。配套: `LingShuState+AgentMemorySeed.swift` 从 `AgentBackbone` 拆出(并行 `human-input-envelope` 功能把 `AgentBackbone` 撑过 500 行架构闸,用户授意拆稳定记忆簇 `seededDistilledMemory/identityAnchorMessage/distillConversationMemory`)。 `GuidanceComposerTests6+ModuleVariantTests` 扩至 13+全量 **1044** 测试 **0** 失败。诚实约束:persona/execution 是 additive 提示(模型遵从受脑限);编译核心"新增变体"仍需一次构建(切换/回退一键)。

- (据当前脑动态调整·接通全旋钮,2026-06-23) 澄清:这套主干 **P5**「差距 2 薄基线」早落地——[LingShuHarnessProfile.capability](#)(脑力测评 `brainBenchmarkResult.score` 主导 + 运行净分 `brainScore ±3` 微调) → `tier lean/balanced/guided`(阈值 75/50 带迟滞防翻档) → [harnessKnobPrefix\(\)](#) 前缀进每轮系统提示(放权/适度/强制 `plan`)+ 换脑归零 + `brainScore` 自主完成 +1/兜底 -1 反馈 + `LingShuCapabilityEscalation` 失败反应式加厚。本轮把它接进新变体槽: `effectiveSlotPayload(slotID:)` = 人切的非基线变体优先(治理不变)、否则 `tierDefaultPayload` 据当前脑力档自动给默认(弱脑→获取上限 1/组合器 `prepend` 策略前置/补引导人格;强脑→上限 3/`append`/不加)。 **live** 实锤:跑中 `DeepSeek` 净分 +544 → `lean` → `acquisitionCeiling effective` 自动=3(非旧硬编码 2)。 `testTierDrivenSlotDefaultsAndHumanOverride`(强/弱脑各档+人覆盖盖过脑力档+回退回自动)+全量 **1045** 绿。注:「接多脑(P5 多端点路由)」才是搁置项(用户明确不要,要的是据当前单脑调)。至此 **P1-P6** 认知控制平面全部落地(目标认知→能力图谱/缺口/获取→全类型验收→经验闭环→脑分层→有界自我进化)+ **P6+** 无界自进化全真(可逆模块变体治理),通用中枢上半身补齐。

6. 变更历史 = git

本文件只记当前状态(组件地图 / 不变量 / 缺口),不记流水账——历史看 `git log`。 workflow:提交时写清楚提交说明(改了什么、为什么、影响面);每次开工前先 `git log --oneline -20` 看最近改动。本文件只在「组件地图 / 不变量 / 缺口」发生结构性变化时更新对应行。